Microservices Patterns Handbook

Twelve patterns for building, migrating and operating microservices · resilience · data consistency · messaging · configuration

12 parts · one place to study: DesignGurus courses + classic books + the web, combined, beginner to intermediate

what it is · problem · how it works · trade-offs · real world · interview · self-check · TL;DR

Personal study notes. Course diagrams belong to DesignGurus. Not for redistribution.

Contents

1. **Strangler Fig Pattern**
   - Part 0 — Concepts you need first
   - Part 1 — Where the name comes from
   - Part 2 — The problem in detail
   - Part 3 — The solution
   - Part 4 — Architecture
   - Part 5 — The four implementation steps
   - Part 6 — Worked example 1: routing a single function
   - Part 7 — Worked example 2: a facade over a banking system
   - Part 8 — Key insights: the proxy layer is everything
   - Part 9 — Trade-offs and what actually goes wrong
   - Part 10 — Common beginner misconceptions
   - Part 11 — Check your understanding
   - Part 12 — Interview soundbite (30 seconds)
   - Part 13 — Implementation
   - Related patterns
2. **API Gateway Pattern**
   - Lesson 1 — Introduction to the API Gateway Pattern
   - Lesson 2 — Advantages: the fifteen problems it solves
   - Lesson 3 — Worked example: a library system
   - Lesson 4 — Problems the gateway itself introduces
   - Lesson 4 (cont.) — Performance implications
   - Lesson 5 — System design example: Instagram
   - Consolidation
   - Implementation
   - Related patterns
3. **Backends for Frontends (BFF) Pattern**
   - Part 0 — Concepts you need first
   - Part 1 — The core idea, in plain terms
   - Part 2 — The problem in detail
   - Part 3 — Architecture
   - Part 4 — The benefits, explained
   - Part 5 — Worked example: e-commerce product details
   - Part 6 — Problems with BFF, and how to handle them
   - Part 7 — System design example: Netflix
   - Part 8 — BFF vs. API Gateway
   - Part 9 — Common beginner misconceptions
   - Part 10 — Check your understanding
   - Part 11 — Interview soundbite (30 seconds)
   - Part 12 — Implementation
   - Related patterns
4. **Service Discovery Pattern**
   - Part 0 — Concepts you need first
   - Part 1 — The problem
   - Part 2 — The solution
   - Part 3 — Architecture
   - Part 4 — Client-side vs. server-side discovery
   - Part 5 — Inner workings
   - Part 6 — Worked example (simplified Java)
   - Part 7 — Performance implications and special considerations
   - Part 8 — Security considerations
   - Part 9 — Real-world implementations
   - Part 10 — Common beginner misconceptions
   - Part 11 — Check your understanding
   - Part 12 — Interview soundbite (30 seconds)
   - Part 13 — Implementation
   - Related patterns
5. **Circuit Breaker Pattern**
   - Part 0 — Concepts you need first
   - Part 1 — The metaphor
   - Part 2 — The problem
   - Part 3 — The solution
   - Part 4 — The three states
   - Part 5 — Integration: it's additive, not invasive
   - Part 6 — Configuration
   - Part 7 — Worked example (Java)
   - Part 8 — Performance implications
   - Part 9 — Best practices and trade-offs
   - Part 10 — Use cases
   - Part 11 — Real-world examples
   - Part 12 — System design example: distributed social media platform
   - Part 13 — Common beginner misconceptions
   - Part 14 — Check your understanding
   - Part 15 — Interview soundbite (30 seconds)
   - Part 16 — Implementation
   - Related patterns
6. **Bulkhead Pattern**
   - Part 0 — Concepts you need first
   - Part 1 — Where the name comes from
   - Part 2 — The problem: failure propagation
   - Part 3 — The solution: isolation
   - Part 4 — Architecture
   - Part 5 — Inner workings
   - Part 6 — Worked example: thread-pool bulkheads in Java

- Part 7 — Performance implications
- Part 8 — Use cases
- Part 9 — System design example: music streaming (Spotify-like)
- Part 10 — Bulkhead vs. Circuit Breaker
- Part 11 — Common beginner misconceptions
- Part 12 — Check your understanding
- Part 13 — Interview soundbite (30 seconds)
- Part 14 — Implementation
- Related patterns

7. **Retry Pattern**
   - Part 0 — Concepts you need first
   - Part 1 — The problem
   - Part 2 — The solution
   - Part 3 — Transient vs. permanent: the critical distinction
   - Part 4 — Architecture
   - Part 5 — Worked example (Java)
   - Part 6 — Performance implications and risks
   - Part 7 — Use cases
   - Part 8 — Real-world examples
   - Part 9 — System design examples
   - Part 10 — When NOT to use retry
   - Part 11 — Common beginner misconceptions
   - Part 12 — Check your understanding
   - Part 13 — Interview soundbite (30 seconds)
   - Part 14 — Implementation
   - Related patterns
8. **Sidecar Pattern**
   - Part 0 — Concepts you need first
   - Part 1 — The metaphor and the core idea
   - ⚠️ Part 2 — One correction to the course material
   - Part 3 — The problem: monolithic application management
   - Part 4 — The solution
   - Part 5 — Architecture
   - Part 6 — Worked example
   - Part 7 — Performance implications
   - Part 8 — Special considerations
   - Part 9 — Use cases
   - Part 10 — System design examples
   - Part 11 — Sidecar vs. shared library
   - Part 12 — Common beginner misconceptions
   - Part 13 — Check your understanding
   - Part 14 — Interview soundbite (30 seconds)
   - Part 15 — Implementation
   - Related patterns
9. **Saga Pattern**
   - Part 0 — Concepts you need first
   - Part 1 — The problem
   - Part 2 — The solution
   - Part 3 — The two types
   - Part 4 — Architecture
   - Part 5 — Worked example: online shopping (orchestration)
   - Part 6 — Issues and considerations
   - Part 7 — Use cases
   - Part 8 — System design examples
   - Part 9 — Saga vs. two-phase commit (2PC)
   - Part 10 — Common beginner misconceptions
   - Part 11 — Check your understanding
   - Part 12 — Interview soundbite (30 seconds)
   - Part 13 — Implementation
   - Related patterns
10. **Event-Driven Architecture (EDA) Pattern**
    - Part 0 — Concepts you need first
    - Part 1 — The problem
    - Part 2 — The solution
    - Part 3 — The benefits
    - Part 4 — The components
    - Part 5 — Three structural variants
    - Part 6 — Inner workings
    - Part 7 — Worked example: traffic management (Java)
    - Part 8 — Performance implications and considerations
    - Part 9 — Use cases
    - Part 10 — System design example: a ride-sharing app
    - Part 11 — Common beginner misconceptions
    - Part 12 — Check your understanding
    - Part 13 — Interview soundbite (30 seconds)
    - Part 14 — Implementation
    - Related patterns
11. **CQRS — Command Query Responsibility Segregation**
    - Part 0 — Concepts you need first
    - Part 1 — The problem
    - Part 2 — The solution
    - Part 3 — Architecture: three levels
    - Part 4 — Inner workings
    - Part 5 — Worked example: user profile management
    - Part 6 — Issues, considerations, and performance
    - Part 7 — Use cases
    - Part 8 — System design examples
    - Part 9 — Common beginner misconceptions
    - Part 10 — Check your understanding
    - Part 11 — Interview soundbite (30 seconds)
    - Part 12 — Implementation
    - Related patterns
12. **Configuration Externalization Pattern**
    - Part 0 — Concepts you need first
    - Part 1 — The framing

- Part 2 — The problem
- Part 3 — The solution
- Part 4 — Architecture
- Part 5 — Worked example: Spring Cloud Config
- Part 6 — Considerations and implications
- Part 7 — Real-world use cases
- Part 8 — Common beginner misconceptions
- Part 9 — Check your understanding
- Part 10 — Interview soundbite (30 seconds)
- Part 11 — Implementation
- Related patterns

Strangler Fig Pattern

Category: Migration / modernization **One-liner:** Replace an old system gradually by putting a router in front of it and moving one capability at a time to the new system, instead of rewriting everything and switching over in one go.

Part 0 — Concepts you need first

If these are already familiar, skip to Part 1.

What is a monolith?

A **monolith** is an application built and deployed as one single unit. All the code — user accounts, payments, reporting, email — lives in one codebase, compiles into one artifact, and runs as one process. Changing one line in the reporting module means rebuilding and redeploying *everything*, including payments.

Monoliths are not evil. For a small application a monolith is usually the *right* choice — it's simpler to build, test, and deploy. The problems start when it grows large and many teams need to change it simultaneously.

What is a legacy system?

"Legacy" doesn't just mean old. It means **old software that is still running because it does something essential**. That's the crucial part: it survives precisely because the business depends on it. Nobody keeps unimportant old software running; they delete it.

This creates the trap that defines this whole chapter: the system is **too risky to keep** (it's fragile and expensive) and **too risky to replace** (it's load-bearing).

What is a "big-bang rewrite"?

The instinctive approach to a bad old system: build a complete replacement, then switch everything over on one date. It's called big-bang because everything changes at once.

It fails often, and for a predictable reason. While you spend eighteen months building the replacement, the old system keeps changing — bugs get fixed, features get added, regulations change. You're rebuilding a moving target. On switchover day you discover the hundred behaviours nobody documented, and you have no way back.

What is a proxy or router (in this context)?

A piece of software sitting **between** a client and one or more servers. The client talks only to the proxy; the proxy decides where the request actually goes. The client cannot tell the difference.

This "client can't tell the difference" property is the entire foundation of the Strangler Fig Pattern. If you can put something in the middle that clients can't distinguish from the real system, you can change what's behind it whenever you like.

What is a feature flag?

A configuration switch that turns behaviour on or off **without deploying new code**. Instead of:

```
return newImplementation(x);
```

you write:

```
if (flags.isEnabled("use-new-implementation")) {
    return newImplementation(x);
}
return oldImplementation(x);
```

Now you can change behaviour by editing configuration, and change it *back* in seconds if something breaks. Flags are what make the migration reversible.

Part 1 — Where the name comes from

The **strangler fig** is a real tree. Its seed germinates in the branches of an existing tree. It grows roots downward and branches upward, gradually enveloping its host. Eventually the host tree dies and rots away, leaving the fig standing in its exact shape — hollow in the middle where the original used to be.

Two things in that image matter for software:

1. **The fig grows around the host while the host is still alive.** It doesn't chop the tree down and plant a seedling in the gap. Both exist simultaneously for a long time.
2. **The transition is gradual and continuous.** There is no single moment when the tree becomes a fig.

Applied to software: your new system grows *around* the legacy system, which continues serving real production traffic the entire time it's being displaced.

The course extends the metaphor: picture an ancient, enormous tree in a dense forest. That's your legacy system — rooted deeply in the organisation's architecture, its many branches representing all the functions it supports. Replacing it feels as daunting as replacing that tree.

Part 2 — The problem in detail

Why legacy systems are painful to live with

Technical debt. Over years, patches, fixes, and new features get added haphazardly — often under deadline pressure, often by people who have since left. The result is a complex, hard-to-maintain codebase where no one is confident what a change will affect.

> *Beginner note:* "technical debt" is a metaphor — taking a shortcut now (like debt) lets you move fast today but charges interest later in the form of slower future changes. Some debt is a reasonable trade; the problem is debt that's never repaid.

Obsolete technology. The stack is no longer current. Concretely, this means you can't hire people who know it (or must pay a large premium), the vendor no longer issues support or patches, and the libraries you depend on have unfixed bugs forever.

Integration issues. Modern tools and services don't integrate easily with older systems. The practical consequence is **silos of information** — data trapped in a system that nothing else can conveniently read, so it gets manually exported, copied, and duplicated.

Scalability concerns. The system was designed for the load and deployment model of its era. It may assume a single server, or a fixed number of them. It's typically **not cloud-ready**, meaning it can't simply be run as more copies when traffic grows.

Costly maintenance. As the system ages, maintenance gets more expensive in both time and resources. Every change takes longer because the code is tangled and the people who understand it are fewer.

Security vulnerabilities. It was designed before current security practices existed. It may store passwords insecurely, lack protection against now-standard attacks, or run on an OS that no longer receives patches.

Why legacy systems are painful to replace

Critical operations. These systems are deeply embedded in the organisation's critical operations. Any disruption has significant impact — this is not "the site is slow," it's "we cannot take orders today."

Migration risks. Moving data from the old system to the new risks **data loss or corruption**. Old schemas usually contain years of accumulated irregularity: records that violate current rules, fields reused for different purposes, encoding inconsistencies.

Training and adoption. Staff know the old system. A new one requires training and often meets genuine resistance — people who were fast and confident are suddenly slow and uncertain.

High costs. Building a new system *and making it as reliable as the old one* is expensive. That second part is easy to underestimate: the old system's reliability came from a decade of bugs found and fixed in production. Your new system hasn't had those years.

Unknown unknowns. There are undocumented features and behaviours that users have come to depend on.

> This is the one that kills rewrites. You cannot write a specification for behaviour nobody wrote down and nobody remembers. You discover it when you switch over and someone says "the report used to round differently and now our reconciliation breaks." A rewrite finds these all at once, on the worst possible day.

Part 3 — The solution

Don't replace the system. **Strangle it.**

Build the new system alongside the old one. Put a routing layer in front of both. Migrate one capability at a time, flipping the router as each new piece proves itself in production. When no traffic reaches the legacy system anymore, delete it.

Why this beats a rewrite

Less risky. The old system is still there. Each step has a safety net, and each step is individually revertible. If the new checkout breaks, you route checkout back to the old system in seconds.

Flexible prioritisation. You migrate by **urgency or business need**, not by the dependency order a rewrite would force. If the payment module is the most painful, do it first.

No big-bang releases. Continuous delivery and gradual replacement instead of one catastrophic cutover date. Less potential for massive failures.

Smooth transition. Done well, users may not notice a shift happened at all — it's gradual and each piece is tested before it carries real traffic.

Early feedback loops. Real users hit new code early, so you learn about problems while they're still cheap to fix, rather than discovering everything at once at the end.

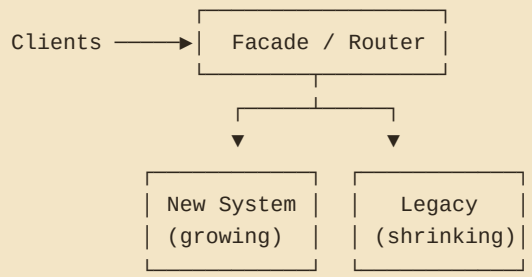
Technology flexibility. The new system can adopt new technology as the industry evolves. You're not locked into whatever was fashionable at project kickoff — a real concern when a migration takes two years.

Part 4 — Architecture

The Facade Interface — the gatekeeper

The central component. Every incoming request hits the facade first, and it makes one decision: **has this functionality been migrated yet?**

- **Yes** → route to the new system
- **No** → route to the legacy system



The course's analogy: **a traffic controller at an intersection**, guiding cars (requests) down the correct road.

The critical property: **clients are unaware there are two systems at all.** They see one address, one API, one system. That's what makes the migration invisible.

> *Beginner note — why "facade"?* A facade is the front of a building. In software, the Facade pattern means putting a simple, unified front over something complicated behind it. Here the complicated thing is "there are two entire systems and the answer depends on which feature you want."

Layered, phased migration

The course describes it as **peeling an onion layer by layer**, each layer being a part of the legacy system replaced by the new one. The migration happens step by step, never all at once:

1. **Identify a piece of functionality that can be cleanly isolated** and redirected.
2. **Rebuild that functionality** in the new system.
3. **Test it fully.**
4. **Flip the facade** to route that functionality to the new system, while everything else still goes to legacy.
5. **Repeat.**

That gradual shift of responsibility from old to new is the crux of the pattern.

Step 1 is the hard one, and beginners underestimate it. "Cleanly isolated" means the feature doesn't share mutable state with everything else. In practice most legacy systems have one giant shared database where everything touches everything, so finding a genuinely separable seam is the real work. Look for capabilities that read a lot and write little, or that own a table nothing else writes to.

Part 5 — The four implementation steps

1. **Identify entry points** — choose the parts of the legacy system you want to replace, and build new services to take over those functions.

2. **Route traffic** — use a router or proxy to direct traffic to either the old system or the new service, based on what's been migrated so far.
3. **Incrementally build and replace** — develop the new services and slowly displace old functionality.
4. **Retire the old** — once all functionality has been taken over, safely decommission the old system.

Part 6 — Worked example 1: routing a single function

The scenario: an old user-management system. You want to replace just `getUserDetails`.

```
class UserServiceRouter:
    def get_user_details(self, user_id):
        if self.use_new_service(user_id):
            return NewUserService().get_user_details(user_id)
        else:
            return OldUserService().get_user_details(user_id)

    def use_new_service(self, user_id):
        # Logic deciding which service to use.
        # For simplicity: use the new service for even user IDs.
        return user_id % 2 == 0
```

Reading this line by line

- `UserServiceRouter` is the facade. **Callers use this class, not the two services directly** — that's what makes the switch invisible to them.
- `get_user_details` has the same signature as the underlying services. The router is a drop-in replacement for whatever callers used before.
- `use_new_service` is the **decision function** — the entire migration policy lives in this one method.
- `user_id % 2 == 0` means "even user IDs." This sends roughly 50% of users to the new service.

Why the even-ID trick is smarter than it looks

This isn't just a toy. It gives you a **percentage rollout dial**. You're not choosing "old or new" — you're choosing *how much* new:

```
return hash(user_id) % 100 < 5     # 5% of users
return hash(user_id) % 100 < 50    # 50% of users
return True                        # everyone
```

And it's **consistent per user**: user 42 always gets the same implementation. That matters enormously — if users flipped between implementations randomly on each request, any difference in behaviour would look like a bizarre intermittent bug, and you'd never reproduce it.

As confidence grows, widen the condition until the old path is dead code — then delete it.

Part 7 — Worked example 2: a facade over a banking system

A fuller example. The legacy system has four operations:

```
public class LegacySystem {
    public void createAccount(String name) { /* old logic */ }
    public void deposit(int amount)        { /* old logic */ }
    public void withdraw(int amount)       { /* old logic */ }
    public int  checkBalance()             { /* old logic */ }
}
```

The new system implements the same four operations with updated logic:

```
public class NewSystem {
    public void createAccount(String name) { /* new logic */ }
    public void deposit(int amount)        { /* new logic */ }
    public void withdraw(int amount)       { /* new logic */ }
    public int  checkBalance()             { /* new logic */ }
}
```

Notice the signatures are identical. That's deliberate and necessary — the facade can only substitute one for the other if they're interchangeable.

Now the facade — the class the outside world actually talks to. It holds an instance of both systems and directs each request:

```
public class BankingFacade {
    private LegacySystem legacySystem;
    private NewSystem newSystem;

    public BankingFacade(LegacySystem legacySystem, NewSystem newSystem) {
        this.legacySystem = legacySystem;
        this.newSystem = newSystem;
    }

    public void createAccount(String name) {
        if (newSystemReadyFor("createAccount")) {
            newSystem.createAccount(name);
        } else {
            legacySystem.createAccount(name);
        }
    }

    public void deposit(int amount) {
        if (newSystemReadyFor("deposit")) {
            newSystem.deposit(amount);
        } else {
            legacySystem.deposit(amount);
        }
    }

    public int checkBalance() {
        if (newSystemReadyFor("checkBalance")) {
            return newSystem.checkBalance();
        } else {
            return legacySystem.checkBalance();
        }
    }

    private boolean newSystemReadyFor(String functionality) {
        // Check whether the new system is ready for this functionality
        return false;   // placeholder — starts false for everything
    }
}
```

How the migration actually proceeds

At the start, `newSystemReadyFor` returns `false` for everything, so **100% of traffic goes to the legacy system**. The facade is in place but changes nothing. This is important: you deploy the facade first and verify it changes nothing, *before* you migrate anything. If something breaks at that point, you know it's the facade, not the new code.

Then you rebuild `createAccount` in the new system, test it, and update `newSystemReadyFor` to return `true` for `"createAccount"`. Now all account creation goes to the new system while deposits, withdrawals, and balance checks still go to legacy.

Repeat for each capability. Gradually the new system takes over.

The key structural insight: the only thing that changes at each cutover is what `newSystemReadyFor` returns. Clients don't change. The two systems don't change. In a real implementation that method reads a **feature flag** from configuration, so flipping a capability doesn't even require a deployment — and flipping it *back* takes seconds.

Part 8 — Key insights: the proxy layer is everything

The router/proxy is the heart of the pattern, and it needs to be genuinely robust — it's now on the critical path for **every single request**. The course lists three kinds of routing logic:

Feature flags. Turn new functionality on and off to test it. The operational superpower here is **instant rollback** — no deploy, no rebuild, just a config change.

User segmentation. Route certain users to new services for **A/B testing or phased rollouts**. Common progressions: internal employees first, then 1% of users, then 10%, then everyone. Or by geography: a small market first, so problems affect fewer customers.

API versioning. Direct requests based on version headers or paths — for example `/v1/orders` to legacy and `/v2/orders` to the new service.

Part 9 — Trade-offs and what actually goes wrong

Running two systems at once. You operate, monitor, and pay for both throughout the migration — which can last years. Two deployment pipelines, two on-call rotations, two sets of dashboards.

The facade is a single point of failure. Everything flows through it. It needs high availability and a carefully considered latency budget, because whatever time it adds is added to every request.

An extra hop. The routing layer adds network latency to every request, however small.

Data consistency — the hard part the pattern doesn't solve. If old and new systems share or split data, you need a strategy: a shared database, synchronisation, or dual writes. This is genuinely the hardest part of most real migrations, and the pattern gives you no help with it.

> *Concrete example of why:* suppose you migrate "create account" to the new system but leave "check balance" on legacy. If the two write to different databases, an account created in the new system is invisible to the legacy balance check. In practice teams usually keep a **single shared database** during migration and split it later — a second strangler migration in its own right.

Duplicated logic. While a capability is in flight, the same business rule exists in two places. Change the rule and you must change both, or they diverge.

Migration fatigue — the most common real-world failure.

> The pattern makes it *safe* to stop halfway, so organisations do. The urgent 60% gets migrated, the pain goes away, priorities shift, and the team runs both systems forever — which is worse than either system alone. **Set a decommission date for the legacy system up front and treat it as a real deliverable.**

Part 10 — Common beginner misconceptions

"The new system must be microservices." No. Strangler Fig is about *how* you migrate, not what you migrate *to*. You can strangle one monolith into a better monolith.

"You migrate the database first." Usually the opposite. Data migration is the riskiest part, so teams typically keep the shared database initially and move behaviour first, splitting data later.

"The facade is temporary." Often it isn't — it frequently becomes the permanent API Gateway. That's fine, as long as it's a deliberate choice.

"Once a capability is migrated you delete the old code immediately." Leave it in place, unreachable but intact, until you're confident. It's your rollback. Delete it once the flag has been at 100% for a sensible period.

"This is only for ancient systems." It works any time you replace a component that's in use — swapping a payment provider, replacing a search engine, moving from one cloud to another.

Part 11 — Check your understanding

1. Why is the strangler fig a better metaphor than "demolish and rebuild"?
2. What single method in the `BankingFacade` example controls the entire migration?
3. Why route by a *consistent* function of user ID rather than randomly per request?
4. Why deploy the facade before migrating anything?
5. What problem does the pattern explicitly *not* solve?
6. What's the most common organisational failure mode, and the countermeasure?

(Answers: 1 — both systems coexist and the transition is gradual and reversible; 2 — `newSystemReadyFor`; 3 — a user must get consistent behaviour, or differences look like intermittent bugs; 4 — to verify the facade itself is harmless before adding a second variable; 5 — data consistency between old and new; 6 — stalling halfway, prevented by committing to a decommission date.)

Part 12 — Interview soundbite (30 seconds)

> "Strangler Fig is how you replace a legacy system without a big-bang rewrite. You put a facade or proxy in front of the old system so clients only ever talk to that, then rebuild functionality one capability at a time behind it. The facade routes each request to either the new service or the legacy system depending on what's been migrated — usually driven by feature flags or user segmentation, so you get percentage rollouts and instant rollback without a deploy. Over time traffic to legacy drops to zero and you delete it. It's named after a tree that grows around its host until the host rots away. The reason it beats a rewrite is that a rewrite has to hit a moving target and discovers all the undocumented legacy behaviour on switchover day, whereas this surfaces problems one capability at a time while you still have a way back. The trade-offs are running both systems in parallel for a long time, the facade becoming a critical single point of failure, and data consistency between old and new, which the pattern doesn't solve. The biggest practical risk is organisational — because it's safe to stop halfway, teams often do, so you commit to a decommission date up front."

Part 13 — Implementation

A. Runnable demo

Full source: **`demos/StranglerFigDemo.java`**

```
cd demos && ./gradlew runCh02
```

The core is the flag check that makes rollout **consistent per user**:

```
boolean useNewSystem(String capability, String userId) {
    int percent = rolloutPercent.getOrDefault(capability, 0);
    if (percent <= 0)   return false;
    if (percent >= 100) return true;
    int bucket = Math.floorMod(userId.hashCode(), 100);   // stable per user
    return bucket < percent;
}
```

`Math.floorMod` (not `%`) because `hashCode()` can be negative and `%` would then yield a negative bucket that never matches.

B. What you should see

```
=== Step 1: facade deployed, nothing migrated ===
  LEGACY:created:alice
  >> all traffic still on LEGACY - facade proven harmless

=== Step 3: same user always gets the same system ===
  user-7 -> LEGACY:created:acct     (x3, never flips)

=== Step 4: widen the rollout ===
    [flags] createAccount -> 25%  ->  actual: 253/1000 users
    [flags] createAccount -> 50%  ->  actual: 493/1000 users
    [flags] createAccount -> 100% ->  actual: 1000/1000 users

=== Step 5: problem found - INSTANT rollback ===
  user-7 -> LEGACY:created:acct
  >> no deploy, no rebuild: config change only
```

Two things worth noting from the real output. **Step 3** proves consistency — user-7 gets the same system every time, so a behavioural difference can never look like an intermittent bug. And the percentages aren't exact (5% gave 44/1000, not 50) because **hash distribution is only approximately uniform** at these sample sizes. That's expected and fine; you're canarying, not metering.

C. Production — routing at the proxy

In reality the facade is usually infrastructure, not application code.

Nginx — route by path:

```
upstream legacy_system { server legacy.internal:8080; }
upstream new_system    { server orders-v2.internal:8080; }

server {
    location /api/orders  { proxy_pass http://new_system;    }   # migrated
    location /api/billing { proxy_pass http://legacy_system; }   # not yet
    location /            { proxy_pass http://legacy_system; }   # default: legacy
}
```

Istio — percentage rollout without touching application code:

```yaml
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: orders
spec:
  hosts: [orders]
  http:
    # Internal staff always get the new system - your first canary group
    - match:
        - headers:
            x-user-group: { exact: internal }
      route:
        - destination: { host: orders, subset: v2 }
    # Everyone else: 5% to the new system
    - route:
        - destination: { host: orders, subset: v1 }
          weight: 95
        - destination: { host: orders, subset: v2 }
          weight: 5
```

Feature flags in application code — LaunchDarkly, Unleash, or a table in your config store:

```java
if (flags.isEnabled("orders-v2", userContext)) {
    return newOrderService.create(order);
}
return legacyOrderService.create(order);
```

D. The part the pattern doesn't solve: data

The hardest real-world problem. Three strategies, in increasing order of difficulty:

1. Shared database (start here). Both systems read and write the same database. No sync problem at all — the new service simply uses the old schema. Split the data later as a separate migration.

2. Dual writes. The new system writes to both databases during transition. Simple to describe, genuinely hard to get right: if the second write fails you have divergence, and there's no transaction spanning both.

3. Change Data Capture (CDC). Stream changes from the legacy database into the new one using Debezium reading the transaction log. The new system's data is eventually consistent with legacy, and crucially **the legacy system needs no modification**:

```yaml
# Debezium connector: legacy Postgres -> Kafka -> new service
connector.class: io.debezium.connector.postgresql.PostgresConnector
database.hostname: legacy-db
table.include.list: public.orders,public.customers
plugin.name: pgoutput
```

E. Implementation pitfalls

1. Random rather than consistent routing. `Math.random() < 0.05` sends the *same user* to different systems on consecutive requests. Any behavioural difference becomes an unreproducible bug report. Always hash a stable identifier.

2. Deploying the facade and migrating a capability in one release. If something breaks you can't tell which change caused it. Deploy the facade first, confirm it's a no-op, then migrate.

3. Deleting legacy code the moment traffic hits zero. That code is your rollback. Leave it unreachable but intact until the flag has been at 100% for a full business cycle.

4. No comparison stage. Before flipping traffic, run **shadow mode** — send the request to both systems, return the legacy response, and log any differences. This is how you find undocumented behaviour *before* it reaches users:

```
Result legacyResult = legacy.process(request);
executor.submit(() -> {                          // async, never blocks the user
    try {
        Result newResult = modern.process(request);
        if (!newResult.equals(legacyResult)) {
            metrics.increment("shadow.mismatch");
            log.warn("Divergence for {}: {} vs {}", request.id(), legacyResult, newResult);
        }
    } catch (Exception e) { metrics.increment("shadow.error"); }
});
return legacyResult;                             // user always gets the proven answer
```

5. Flags that outlive the migration. Every completed migration leaves a dead conditional. Schedule flag removal as part of "done," or the codebase fills with permanent forks.

6. No decommission date. The organisational failure mode from Part 9. Put it in the plan.

Related patterns

- **API Gateway** — the facade is very often literally an API Gateway
- **BFF** — can bridge new frontends to legacy systems during migration
- **Config Externalization** — where the feature flags driving routing actually live

API Gateway Pattern

Category: Edge / routing **One-liner:** A single entry point that sits in front of all microservices, routing, aggregating, securing, and shaping requests so clients never talk to services directly.

Lesson 1 — Introduction to the API Gateway Pattern

The fast-food counter

Picture ordering a burger. You walk to the counter, place your order, pay, and wait. That counter is the API Gateway. It is the **single point at which you, the customer, interact with the restaurant's services** — the kitchen, the billing system, inventory. You never walk into the kitchen to ask the cook for your patty and then go to a separate desk to pay. One counter, one interaction, and everything behind it is somebody else's problem.

What the pattern actually is

In a microservice architecture you have many services each doing their own thing. The API Gateway Pattern is about having **a single entry point for external consumers** to reach all of them.

Three things follow from that single entry point, and it's worth separating them because they're independent benefits:

1. **It makes the system easier to manage** — there is one place where request handling policy lives.
2. **It secures your services** — the services themselves need not be individually exposed.
3. **It provides a unified interface to the external world** — clients see one coherent API rather than the seams between your teams.

Why the pattern exists

The dominant architectural trend of recent years has been the shift from monolithic structures to microservices. The course compares it to the shift from landlines to smartphones in telecommunications — a genuine step change that brought enhanced scalability, increased flexibility, and more. But as with any technological advance, it introduced its own problems.

The biggest of those problems is communication:

- **How do you ensure effective and efficient communication** between a large number of services?
- **How do you manage the growing complexity** that comes with an ever-increasing number of interactions between them?

This is precisely the gap the API Gateway Pattern fills.

The Netflix example

When you open Netflix on your TV or phone, the app needs a lot of different data — the movie list, your user preferences, subtitles, and more. Netflix has a separate microservice responsible for each of those pieces.

Now consider the two possible designs:

- **Without a gateway:** your TV app calls each microservice separately. It must know that there *is* a preferences service, where it lives, how to authenticate to it, and what shape its response takes — and repeat all of that for every other service.
- **With a gateway:** your TV app makes one call to the Netflix API Gateway, which then talks to whichever microservices are needed.

The second is dramatically simpler for the TV app. And note *which* app benefits most — a TV app or a mobile app is something you cannot force users to update. Every piece of service topology you bake into it is frozen in the field for years.

The master orchestrator

The gateway operates like **the conductor of an orchestra**. It oversees and directs the entire ensemble of services, ensuring each plays its part at the right time to produce a harmonious output.

Mechanically, it does three things in sequence:

1. Serves as the **single entry point** for all client requests
2. **Routes** those requests to the appropriate microservices
3. **Gathers the responses** from the services involved and delivers a **unified response** back to the client

The beauty of the pattern is its ability to **mask the underlying complexity from clients**. As a client you do not need to know how many services were involved or how they communicate with each other. You make a request; the gateway handles the rest.

The headline advantages

The introduction gives four, which the next lesson expands considerably:

- **Simplified client** — clients interact with one gateway instead of many services.
- **Security** — authentication and authorization can be added at the gateway level.
- **Performance** — you can add caching, compress responses, and manage load balancing.
- **Cross-cutting concerns** — logging, analytics, and similar concerns are handled in one place.

Lesson 2 — Advantages: the fifteen problems it solves

The framing

Imagine a city full of different services and places to visit. As a tourist, interacting with each service individually gets confusing and overwhelming fast. But if there is a **single information centre** that guides you and helps you interact with all of them, your life becomes far easier. That is what the gateway does for microservices.

What follows is each problem the course identifies, with the gateway's answer. I've kept all fifteen as their own item because each is a distinct argument.

1. Complexity of interactions

Problem: In a microservices architecture there are many services talking to each other. If every service communicates with all the others directly, the number of connections grows very complex, very fast. **Solution:** The gateway becomes the central point of communication. Services talk to the gateway, and the gateway talks to other services — collapsing a mesh of connections into a hub.

2. Inconsistency in service calls

Problem: Different services may require different interaction styles and have different data formats. A client ends up implementing several dialects of your own API. **Solution:** The gateway provides one consistent way to interact with everything. It can **translate calls and data formats**, so the client sees uniformity even where the backend has none.

3. Security and authentication

Problem: Handling security for each service individually is a nightmare. Every service must be independently secured, and that is a great deal of duplicated, easily-inconsistent work. **Solution:** Authentication and authorization are handled **centrally**. Clients authenticate with the gateway, and the gateway ensures callers are who they claim to be before anything reaches a service.

4. Load balancing

Problem: Some services receive far more traffic than others, and that traffic needs managing efficiently. **Solution:** The gateway distributes incoming traffic evenly across services, ensuring **no single service is overwhelmed**.

5. Performance issues

Problem: Services are spread out, and the network calls between them add delays. **Solution:** The gateway can **cache responses** from services, reducing the need for repeated calls and improving performance.

6. Cross-cutting concerns

Problem: Concerns like logging, monitoring, and analytics would otherwise need implementing in **every single service**. **Solution:** Handle them centrally at the gateway, removing that duplication.

7. Managing legacy systems

Problem: You may have old services that must be accessed in specific, awkward ways. **Solution:** The gateway can interact with legacy services while **presenting a modern interface** to clients — the awkwardness stops at the gateway boundary.

8. Version management

Problem: Different clients may need to interact with different versions of a service — again, most acute with mobile and TV clients you cannot force to upgrade. **Solution:** The gateway routes requests to the appropriate version of a service based on the client's needs.

9. Aggregation of results

Problem: A client may need information spread across multiple services. Gathering it could require several network calls, which is inefficient — especially over a mobile connection. **Solution:** The gateway aggregates results from multiple services and returns a **single consolidated response**.

10. Service orchestration

Problem: Some operations require a series of service calls made **in a particular order**. **Solution:** The gateway orchestrates those calls, ensures correct ordering, and handles any transformations needed along the way.

11. Handling partial failures

Problem: In a distributed system some services fail while others keep working. Handling these **partial** failures gracefully is genuinely difficult. **Solution:** The gateway detects failures and either **reroutes** requests or provides **fallback responses**, so the system stays robust rather than failing wholesale.

12. Request transformation

Problem: Different services expect requests formatted in different ways. **Solution:** The gateway transforms each incoming request into the format the target service expects.

13. Client-specific adaptations

Problem: Different clients — mobile, web, and so on — have different requirements. **Solution:** The gateway adapts responses to suit each client's needs. *(Pushed to its logical conclusion, this becomes BFF — one gateway per client type.)*

14. Centralized logging and monitoring

Problem: Collecting logs and monitoring data from every service in a distributed system is challenging. **Solution:** The gateway collects logs and monitoring data, giving you **one point for observability**.

15. Handling asynchronous operations

Problem: Some operations take a long time and should not block the client. **Solution:** The gateway manages those asynchronous operations, giving clients a way to **check progress or receive a callback** on completion.

Taken together, these create a smoother, more reliable interaction layer for clients — the course describes it as a super-efficient helper that takes care of the nitty-gritty details so you get a seamless experience.

Lesson 3 — Worked example: a library system

The scenario

A library system in which various clients — library users, staff, and management — interact with several microservices: a **Catalogue Service**, a **User Service**, and an **Inventory Service**.

Each service corresponds to a distinct area of the library's functionality:

- **CatalogueService** handles requests relating to book searches
- **UserService** manages user profiles
- **InventoryService** keeps track of book stock levels

In the Java implementation each would be an interface with methods matching potential client requests — `CatalogueService`, for instance, might expose `searchByTitle(String title)` and `searchByAuthor(String author)`.

The gateway

A separate class acting as the single point of contact for all client requests. The course's analogy: **a receptionist at the library**, fielding questions from visitors and directing them to the right place.

The crucial design point: `ApiGateway` has a method corresponding to each client request, mirroring the services — but rather than handling those requests itself, **each method delegates the work to the appropriate service**.

```
public class ApiGateway {
    private UserService userService;
    private CatalogueService catalogueService;
    private InventoryService inventoryService;

    public ApiGateway(UserService userService,
                      CatalogueService catalogueService,
                      InventoryService inventoryService) {
        this.userService = userService;
        this.catalogueService = catalogueService;
        this.inventoryService = inventoryService;
    }

    public User getUserDetails(String userId) {
        return userService.getUserDetails(userId);
    }

    public Book searchByTitle(String title) {
        return catalogueService.searchByTitle(title);
    }

    // ...additional methods as needed...
}
```

Walking it through: `ApiGateway` receives a request for user details and passes it to `UserService`. The same applies to a book title search, which is routed to `CatalogueService`.

What to take from the shape, not the detail: gateway methods *delegate*, they never *implement*. The moment a gateway method starts containing business rules rather than forwarding, you are on the road to the distributed monolith described below. Real gateways would wrap each delegation with auth, timeouts, retries, and caching — but the delegation shape stays the same.

Lesson 4 — Problems the gateway itself introduces

The course opens this lesson with "every rose has its thorn" — the gateway solves a great deal and becomes a liability of its own. Five problems, each with its mitigation.

1. Single point of failure

Problem: If the API Gateway goes down, **nothing can communicate**. You have concentrated all traffic through one component, so its availability is now your system's availability. **Solution:** Implement high availability and redundancy for the gateway, and **monitor it diligently**.

2. Potential bottleneck

Problem: All requests pass through the gateway, which could cause delays. **Solution:** Ensure the gateway is highly performant, **scale it horizontally**, and optimise its code.

3. Complexity in the gateway

Problem: The gateway can become too complex as it accumulates routing, security, transformations, and everything else. **Solution:** Keep the gateway's responsibilities clear. **Consider splitting it into multiple gateways** if necessary — per client type, or per functionality.

> This is the failure mode worth naming explicitly: the gateway is where business logic goes to hide. Once teams start putting domain rules in it, every team needs to change it, and it becomes

> the deployment bottleneck microservices were supposed to eliminate. Keep it about **transport**, not **domain**.

4. Maintenance overhead

Problem: As services evolve, maintaining the gateway's routes and rules becomes challenging — the gateway's configuration drifts away from reality. **Solution: Automate as much as possible.** Use CI/CD to ease maintenance.

5. Security risk

Problem: Being the entry point makes it a **prime target for attacks**. The same concentration that makes it useful makes it valuable to an attacker. **Solution:** Implement robust security practices, keep the gateway updated, and monitor for vulnerabilities.

Lesson 4 (cont.) — Performance implications

Distinct from the problems above: these are the costs the gateway imposes on every request.

1. Latency

Implication: The gateway introduces an **additional network hop**. **Solution:** Optimise its performance using techniques like **caching and response compression**.

2. Resource utilization

Implication: The gateway may consume significant resources, affecting performance. **Solution:** Ensure it has adequate resources; monitor utilisation and **scale as necessary**.

3. Content aggregation overhead

Implication: Aggregating responses from multiple services at the gateway is resource-intensive. **Solution:** Optimise the aggregation logic and use **asynchronous operations** where possible to prevent blocking.

4. Rate limiting challenges

Implication: Implementing rate limiting at the gateway is complex, **especially in distributed deployments** — several gateway instances must share a view of each client's usage. **Solution:** Use advanced algorithms built for this, such as **token bucket** or **leaky bucket**, which handle distributed scenarios.

5. Load balancing overhead

Implication: The gateway must distribute requests efficiently to prevent hotspots. **Solution:** Implement smart load balancing algorithms; **consider a service mesh** for more granular control. *(See Sidecar.)*

6. Handling asynchronous operations

Implication: Managing async operations at the gateway is complex and can affect performance. **Solution:** Optimise their handling and provide clients with **clear mechanisms to check status or receive callbacks**.

The overall verdict: the pattern introduces challenges and performance costs, but with careful design, monitoring, and optimisation you can mitigate them and run a smooth, efficient system. The alternative — every client knowing every service — is worse.

Lesson 5 — System design example: Instagram

Instagram is a good illustration because it is genuinely complex: millions of users and a great many features. Think of it as a **bustling city**, with the API Gateway as the **main entrance**, directing traffic and keeping everything running smoothly.

The gateway's responsibilities here

- **Routing** — directing user requests to the appropriate microservice
- **Aggregation** — combining data from multiple services into one consolidated client response
- **Authentication & authorization** — confirming users are who they say they are, and that they have permission to access what they requested
- **Rate limiting** — preventing abuse by limiting how many requests a user can make in a timeframe
- **Load balancing** — distributing requests across multiple instances of a service so none is overwhelmed
- **Logging and monitoring** — tracking what is happening for troubleshooting and performance monitoring

The microservices behind it

- **User Management Service** — user profiles, relationships (followers/following), account settings
- **Photo Upload and Processing Service** — uploading photos, applying filters, resizing for different devices
- **Feed Service** — generating and serving the user's feed of photos from people they follow
- **Notification Service** — notifications for new followers, likes, comments
- **Search Service** — queries for user profiles, hashtags, and locations
- **Comment Service** — comments on photos

Scenario A — uploading a photo *(sequential)*

1. The user uploads a photo via the Instagram app
2. The request hits the API Gateway
3. The gateway **authenticates** the user and checks they are **authorized to upload photos**
4. Once authenticated, the gateway routes the request to the Photo Upload and Processing Service
5. The Photo Service processes the image and sends a response back to the gateway
6. The gateway sends a confirmation back to the user's app

Note that authentication and authorization are **two separate checks** at step 3, and both happen before any service is touched.

Scenario B — viewing the feed *(parallel — the interesting one)*

1. The user opens the app to view their feed
2. The request hits the API Gateway
3. The gateway authenticates the user
4. The gateway makes **parallel requests** to:
    - **Feed Service** — to get the list of recent photos from people the user follows
    - **Notification Service** — to fetch any new notifications
5. The gateway **aggregates** the responses from these services
6. The gateway sends a **single consolidated response** back to the app, showing feed and notifications together

Why the contrast matters

Upload is a single routed call. Feed is a **scatter-gather**: one client request fans out to several parallel service calls and comes back as one merged response.

That fan-out is where the gateway earns its latency cost. Doing the same thing client-side would mean several sequential round trips over a mobile network — far slower than one round trip plus parallel calls inside the data centre, where latency between services is a fraction of the latency to the phone.

The result: Instagram gets a consistent, efficient, and secure way to handle all the interactions needed for a seamless experience across millions of users, and the system stays easier to manage and scale as it grows.

Consolidation

API Gateway vs. BFF

	API Gateway	BFF
Count	One shared entry point	One per client type
Optimised for	All clients generically	One specific client
Client-specific logic	Pollutes it	Belongs there
Risk	Becomes a bottleneck / distributed monolith	Duplication and drift across BFFs

Not mutually exclusive — a common setup is a thin shared gateway (TLS, coarse auth, rate limiting) with BFFs behind it.

When to use

- Multiple microservices **and** multiple client types
- Clients would otherwise make many round trips per screen
- You want cross-cutting concerns (auth, rate limiting, logging) in one place
- You are exposing services to untrusted external consumers

When to avoid

- Very few services — the gateway is pure overhead
- Purely internal service-to-service traffic — a **service mesh** fits better
- One client whose needs diverge sharply from everything else — consider BFF
- Ultra-low-latency paths where an extra hop is unacceptable

Interview soundbite (30 seconds)

> "An API Gateway is a single entry point in front of your microservices. Clients call it instead of calling services directly, and it handles routing, response aggregation, authentication, rate limiting, caching, and logging in one place. The big win is that clients don't need to know your service topology — which matters most for mobile and TV clients you can't force-update, because any topology you bake into them is frozen in the field for years. The classic example is Instagram's feed: one request fans out in parallel to the feed and notification services and comes back as one response, which beats several sequential round trips from the phone. The trade-offs are that it's a single point of failure needing HA and redundancy, it adds a network hop, rate limiting across distributed gateway instances needs token or leaky bucket algorithms,

> and it tends to accumulate business logic until it becomes a distributed monolith — so keep it about transport, not domain, and split it per client type or per domain if it gets too big. If each client type needs a genuinely different API shape, that's Backends for Frontends."

Implementation

A. Runnable demo

Full source: `demos/ApiGatewayDemo.java`

```
cd demos && ./gradlew runCh03
```

The heart of it is scatter-gather with per-service fallbacks:

```
Future<String> fFeed    = pool.submit(() -> feed.fetch(userId));
Future<String> fNotifs  = pool.submit(() -> notifications.fetch(userId));
Future<String> fProfile = pool.submit(() -> profile.fetch(userId));

result.put("feed",          get(fFeed,    "feed-unavailable"));
result.put("notifications", get(fNotifs,  "no-notifications"));
result.put("profile",       get(fProfile, "profile-unavailable"));

private String get(Future<String> f, String fallback) {
    try { return f.get(1, TimeUnit.SECONDS); }
    catch (Exception e) { return fallback; }        // partial failure contained
}
```

All three calls are **submitted before any result is read** — that's what makes them parallel. Reading `fFeed` before submitting the others would serialise them.

B. What you should see

```
=== 1. Without a gateway: client calls each service itself ===
  3 round trips from the client: 288ms
  >> on mobile add ~150ms latency PER trip: ~738ms

=== 2. Gateway, sequential aggregation ===
  1 client round trip, 3 sequential server calls: 279ms

=== 3. Gateway, PARALLEL scatter-gather ===
  1 client round trip, 3 parallel server calls: 129ms
  >> ~the slowest service (120ms), not the sum

=== 4. Auth enforced once, at the edge ===
  rejected: 401 Unauthorized  (no service was touched)

=== 5. Partial failure -> graceful degradation ===
  {feed=feed-data, notifications=no-notifications, profile=profile-data}

=== 6. Rate limiting, also at the edge ===
  429 Too Many Requests after 5 requests
```

Compare 2 and 3: 279ms → 129ms. Parallel aggregation costs the *slowest* service, not the sum. And note **section 4** — an unauthenticated request touched **zero** backend services, which is the security argument for a gateway stated as an observable fact.

C. Production — Spring Cloud Gateway

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-gateway</artifactId>
</dependency>
```

```
spring:
  cloud:
    gateway:
      routes:
        - id: orders
          uri: lb://order-service          # lb:// = resolve via service discovery
          predicates:
            - Path=/api/orders/**
          filters:
            - StripPrefix=2
            - name: CircuitBreaker
              args:
                name: orderCB
                fallbackUri: forward:/fallback/orders
            - name: RequestRateLimiter
              args:
                redis-rate-limiter.replenishRate: 100    # steady-state req/s
                redis-rate-limiter.burstCapacity: 200    # burst allowance
            - name: Retry
              args:
                retries: 2
                statuses: BAD_GATEWAY,SERVICE_UNAVAILABLE
                methods: GET                             # only idempotent methods

        - id: products
          uri: lb://product-service
          predicates:
            - Path=/api/products/**
            - Header=X-API-Version, v2                   # route by version
```

Global cross-cutting filter — auth applied to every route:

```
@Component
public class AuthFilter implements GlobalFilter, Ordered {
    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        String token = exchange.getRequest().getHeaders().getFirst("Authorization");
        if (!tokenService.isValid(token)) {
            exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);
            return exchange.getResponse().setComplete();      // never reaches a service
        }
        return chain.filter(exchange);
    }
    @Override public int getOrder() { return -100; }          // run early
}
```

Kubernetes-native alternative — an Ingress or Gateway API resource does routing, TLS, and rate limiting without you writing or operating a gateway application at all. Prefer this when your needs are routing-shaped rather than aggregation-shaped.

D. Implementation pitfalls

1. Sequential calls that should be parallel. The most common performance bug in gateways. If you `.get()` on each future as you create it, you've written sequential code that looks concurrent.

2. No per-call timeout in aggregation. One slow service makes the whole aggregated response slow. Always bound each future — `f.get(1, TimeUnit.SECONDS)` in the demo — so the slowest dependency can't dictate your latency.

3. Business logic in the gateway. The distributed-monolith trap. Once domain rules live there, every team must change it and it becomes the deployment bottleneck. Keep it to transport concerns: routing, auth, rate limiting, aggregation, observability.

4. In-memory rate limiting behind multiple gateway instances. Three replicas each allowing 100 req/s means the real limit is 300. Use a **shared store (Redis)**, which is why the config above uses `redis-rate-limiter`.

5. Retrying non-idempotent requests. The `methods: GET` line above is deliberate. A gateway retrying a POST can create duplicate orders. *(See Retry.)*

6. Running the gateway as a single instance. It's a single point of failure by design — so run at least two replicas behind a load balancer, and treat its resource limits and monitoring as seriously as any critical service.

7. Losing trace context. The gateway is the ideal place to generate a correlation ID and propagate it downstream. Skip it and you lose the ability to trace a request across services.

Related patterns

- **BFF** — one gateway per client type instead of one shared gateway
- **Circuit Breaker** — the gateway is a natural place to configure breaker rules per backend route
- **Service Discovery** — how the gateway finds service instances; the gateway is the classic server-side discovery router
- **Strangler Fig** — the migration facade is often literally an API Gateway
- **Sidecar** — gateway handles north-south (external) traffic; a mesh of sidecars handles east-west (internal)

Backends for Frontends (BFF) Pattern

Category: Edge / routing **One-liner:** Instead of one shared backend serving every kind of client, give each client type its own backend tailored to exactly what that client needs.

Origin: Coined by **Sam Newman**, to solve the problem of building backend services for structurally different clients — mobile apps, web applications, and other client-side frameworks.

Part 0 — Concepts you need first

What is a "frontend" and a "backend"?

The **frontend** is the part the user sees and interacts with — the mobile app, the website, the smart-TV interface. The **backend** is the server-side software the frontend talks to for data and operations.

A **client** in this chapter means a frontend application. "Client types" means categories of them: mobile, web, TV, watch.

What is over-fetching?

Over-fetching is when a client downloads more data than it actually needs.

Concretely: a mobile app shows a product list with a thumbnail, name, and price. But the API returns the full product object — a 2,000-word description, fifteen high-resolution image URLs, full specifications, the seller's history. The app downloads all of it and displays three fields.

Why it hurts:

- **Bandwidth** — on a metered mobile plan, this is the user's money
- **Latency** — larger payloads take longer over slow connections
- **Battery and CPU** — the phone must parse JSON it discards
- **Memory** — constrained devices hold data they never use

What is an API?

An **API** (Application Programming Interface) is the agreed contract for how one piece of software asks another for something: what you can request, what parameters you supply, what shape comes back.

When this chapter says "one uniform set of APIs for all frontends," it means every client — however different — gets the same endpoints returning the same shapes.

What is aggregation?

Combining data from several sources into one response. If a screen needs product details, reviews, and stock levels from three services, **aggregation** means one component calls all three and returns a single merged result, so the client makes one request instead of three.

Why round trips matter so much on mobile

A **round trip** is one request out and one response back. On a fast office network it might take 5-20ms. On a mobile network it can be **100-300ms or worse**.

Six round trips on a desktop: barely noticeable. Six round trips on mobile: potentially two seconds of nothing happening. This single fact drives most of this chapter.

Part 1 — The core idea, in plain terms

Imagine a company with three digital platforms:

- **A detailed website** — for users on computers, offering extensive information
- **A mobile app** — quick, essential services for on-the-go use
- **A smartwatch app** — core functions only, in a very compact form

These are genuinely different products with genuinely different needs. The website can display a thousand words; the watch can display about eight.

Traditionally, one backend serves all three. The course's analogy: BFF means giving **each section of a diverse restaurant its own kitchen**, rather than forcing one kitchen to cook for wildly different menus.

The reason this matters: the backend complexity appropriate for the website **overwhelms** the simpler needs of mobile and watch. The website's needs win by default, because it's usually the biggest and oldest client, and everyone else gets its leftovers.

Netflix does exactly this — web browsers, mobile apps, and smart TVs each get their own backend service, because each device has different capabilities and interfaces.

Part 2 — The problem in detail

One-size-fits-all

In traditional backend models the backend is a monolithic entity serving multiple frontends, exposing **one uniform set of APIs regardless of each client's unique requirements**.

Consider a desktop app and a mobile app hitting the same API:

	Desktop	Mobile
Processing power	High	Limited
Display	Large	Small
Network	Usually fast, unmetered	Possibly slow, possibly metered

The course asks the right questions: should the mobile app receive the same amount of data as the desktop app? Is it efficient to process all that data in a mobile environment? **Not quite.**

Performance inefficiency and over-fetching

This leads directly to over-fetching, which causes performance issues, longer loading times, and increased bandwidth usage.

Worth being concrete about the cost. A product list screen:

- **Shared API response:** 250 KB (full descriptions, high-res image URLs, full specs)
- **What mobile displays:** name, price, thumbnail — about 4 KB of actual information

The user paid for 250 KB and used 4 KB. On a slow connection that's several seconds of waiting for data that gets thrown away.

Complexity in code management

When a single codebase serves multiple frontends it becomes **bloated and complex** — harder to maintain, troubleshoot, and update. The more complex the system, the more time and resources it consumes, which could have gone to productive work.

In practice this shows up as endpoints crowded with conditionals:

```
if (clientType.equals("mobile")) {
    // trim the description
} else if (clientType.equals("tv")) {
    // use high-res images
} else if (clientType.equals("watch")) {
    // just the title
}
```

Every client's needs tangled into one method that nobody can change safely.

Lack of flexibility and scalability

Traditional backends have a rigid structure that makes it hard to innovate and scale. The course gives two symptoms that should feel obviously wrong:

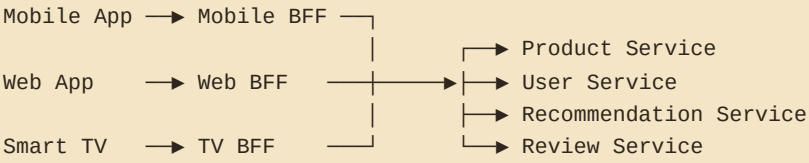
1. **Sifting through a monolithic codebase to change one small feature for one specific frontend**
2. **Scaling the entire backend system because one frontend is experiencing high traffic**

Both are overkill — that's exactly the signal you need BFF.

> *Why symptom 2 matters:* suppose a TV advert drives a spike in mobile traffic. With a shared backend you scale the whole thing — including the capacity serving web and internal tools that aren't busy at all. You pay for capacity you don't need because the parts can't be scaled separately.

Part 3 — Architecture

Three layers:



1. Frontend applications — the user-facing applications: desktop, mobile, web. *The course's analogy: the customers ordering from a menu.*

2. BFFs — each paired with one specific frontend, catering to its unique requirements. *The chefs, preparing each dish to a customer's preference.*

3. Microservices — the underlying services BFFs call to fetch data. *The kitchen staff providing the raw materials.*

The important structural point: **the microservices are shared; only the BFF layer is duplicated.** There is still exactly one Product Service holding product truth. BFFs don't own data — they reshape it.

What a BFF actually does

Tailored data delivery. Each BFF ensures its frontend receives *just* the data it needs. It acts as a **filter**, ensuring the frontend isn't overwhelmed with unnecessary information.

Streamlining communication. The BFF talks to multiple microservices, consolidates the data, and presents it to the frontend in one streamlined shape. The course calls it an **interpreter**,

translating many languages into one the frontend understands.

Handling client-specific logic. Each BFF absorbs logic specific to its frontend, relieving the client of complex computation — a **personal assistant** doing work so the client's job is easier.

> *Why moving logic server-side matters especially for mobile:* you cannot force users to update a mobile app. If display logic lives in the app, fixing it means shipping a release, waiting for app store review, and waiting for users to update — some never will. If it lives in the BFF, you fix it and deploy in minutes.

Part 4 — The benefits, explained

The course lists seventeen. Grouped by theme, with the reasoning behind each.

Fit and user experience

Diverse frontend needs. A mobile app, a web app, and a smart TV app have genuinely different requirements. BFF acknowledges that diversity instead of pretending it away.

Enhanced user experience. By catering to each device's specific needs and capabilities, the user gets the best possible experience on whatever they're using.

Localization and internationalization. Different frontends may serve users in different regions with different languages and cultural contexts. A BFF can serve localized content and region-relevant features.

Performance

Performance optimization. Mobile devices may be on slower networks with less processing power than a desktop. A BFF lets you optimise the data and interactions per device for fast, responsive apps.

Reduced bandwidth usage. Especially important for mobile — the BFF sends only what that specific frontend requires.

Scalability. Each frontend has its own backend, so you can scale parts of the system independently based on their own demand. A mobile traffic spike scales only the Mobile BFF.

Simplicity for frontend teams

Simplified frontend logic. Frontends no longer deal with the complexity of interacting with multiple microservices. They have a single, tailored backend that aggregates and processes data for them.

Consistency. A BFF can provide a **consistent API even while the underlying microservices are constantly changing**. This is a stability layer for frontend developers — the backend team can reorganise services without breaking every app.

Versioning control. Different frontend applications may be at different versions requiring different backend logic. A BFF handles versions gracefully, so each frontend gets logic appropriate to its version. *(Again most acute on mobile, where old versions persist in the wild.)*

Team and delivery

Agility. Teams work independently on their own BFF, allowing faster development and deployment cycles. The mobile team doesn't queue behind the web team.

Easier maintenance and upgrades. You can update the mobile backend without affecting the web backend, and vice versa.

Cross-functional teams. BFF promotes teams that own **end-to-end functionality for one user interface** — frontend and its backend together. Better collaboration, faster delivery.

Capability

Security. Each client type can have its own security considerations. A mobile app may need different authentication mechanisms than a web app.

Error handling and resilience. You can implement error handling specific to each frontend. A web application might handle certain errors gracefully where a mobile app cannot.

Facilitates A/B testing. You can test new features or changes on one client type — say the mobile app — **without affecting the others**.

Feature flagging. Enable or disable features for specific frontends at the backend level, without affecting others.

Handling legacy systems. During a monolith-to-microservices transition, a BFF acts as a **bridge**, letting new microservices-based frontends interact with legacy systems. *(See Strangler Fig.)*

Part 5 — Worked example: e-commerce product details

Three components: one shared **Product Service**, a **Web BFF** (extensive detail), and a **Mobile BFF** (concise, to save data and improve load times).

1. Product Service — the real backend

```
@RestController
@RequestMapping("/products")
public class ProductServiceController {

    @Autowired
    private ProductRepository productRepository;

    @GetMapping("/{productId}")
    public Product getProductById(@PathVariable String productId) {
        return productRepository.findById(productId)
                .orElseThrow(() -> new ProductNotFoundException(productId));
    }
}
```

Reading it:

- `@RestController` marks this a web-request handler returning data (not HTML pages)
- `@RequestMapping("/products")` — every endpoint here starts with `/products`
- `@GetMapping("/{productId}")` — handles `GET /products/123`; `{productId}` is a placeholder
- `@PathVariable` pulls that value out of the URL into the method parameter
- `orElseThrow` — if no product exists, raise a clear error rather than returning null

This service owns the **full, detailed** `Product` and does the heavy lifting. It knows nothing about mobile or web.

2. Web BFF — the detailed view

```java
@RestController
@RequestMapping("/web/products")
public class WebProductController {

    @Autowired
    private ProductServiceClient productServiceClient;

    @GetMapping("/{productId}")
    public ProductDetails getProductDetails(@PathVariable String productId) {
        Product product = productServiceClient.getProductById(productId);
        return convertToWebProductDetails(product);
    }

    private ProductDetails convertToWebProductDetails(Product product) {
        // Convert to a detailed view suitable for web:
        // richer descriptions, full-size images, etc.
    }
}
```

Note the URL prefix `/web/products` — each BFF has its own namespace, so clients can't accidentally call the wrong one.

`ProductServiceClient` is a **Feign client** or any HTTP client used to talk to the Product Service.

> *Beginner note — what's a Feign client?* A library that lets you call another service by declaring an interface, and generates the HTTP-calling code for you. You call `getProductById(id)` like a normal method and it makes the network request behind the scenes.

3. Mobile BFF — the concise view

```java
@RestController
@RequestMapping("/mobile/products")
public class MobileProductController {

    @Autowired
    private ProductServiceClient productServiceClient;

    @GetMapping("/{productId}")
    public SimpleProductDetails getProductDetails(@PathVariable String productId) {
        Product product = productServiceClient.getProductById(productId);
        return convertToMobileProductDetails(product);
    }

    private SimpleProductDetails convertToMobileProductDetails(Product product) {
        // Convert to a concise view suitable for mobile:
        // stripped-down descriptions, smaller images, etc.
    }
}
```

The structural point — read this part twice

Compare the two BFFs. They are **nearly identical**:

1. Both call the **same** `productServiceClient.getProductById(productId)`
2. Both receive the **same** full `Product` object
3. They differ **only in the transformation step** and therefore the return type:

- Web returns `ProductDetails` — rich descriptions, full images
- Mobile returns `SimpleProductDetails` — stripped descriptions, smaller images

A BFF is a projection layer, not a second source of truth. There is one Product Service holding one truth about products. The BFFs each project a different view of it. If you ever find a BFF *storing* data or deciding business rules about what a product *is*, it has stopped being a BFF.

How it works in practice

- The **Product Service** handles all the heavy lifting related to product data
- The **Web BFF** provides a detailed view suitable for web users
- The **Mobile BFF** provides a concise view suitable for mobile users

Each frontend gets exactly what it needs, and the backend logic is tailored to give the best possible experience.

Part 6 — Problems with BFF, and how to handle them

1. Code duplication

Problem: Since each frontend has its own BFF, there may be **duplication across BFFs**, especially where they share common functionality — authentication handling, error formats, logging. **Solution:** Use **shared libraries** for common functionality, or create **internal microservices** that multiple BFFs consume.

2. Increased maintenance

Problem: More BFFs mean **more codebases to maintain**, increasing operational overhead. Three BFFs means three deployment pipelines, three sets of dependencies to patch. **Solution:** Adopt good DevOps practices, automate testing and deployment, and ensure proper monitoring to keep maintenance manageable.

3. Coordination overhead

Problem: Coordinating changes between frontends and their BFFs becomes challenging in large teams. **Solution:** Establish **clear contracts** between frontends and BFFs, and use **Consumer-Driven Contract Testing** to ensure compatibility.

> *Beginner note — consumer-driven contract testing:* the consumer (the app) writes down what it expects from the BFF as an executable test. The BFF's build runs those tests. If the BFF changes something the app depends on, the BFF's build fails — before it reaches production, rather than after.

Performance implications

Network latency. BFF adds an additional network hop, contributing to latency. *Solution:* optimise network paths, use efficient protocols, and ensure **BFFs are geographically close to the frontends they serve**.

Resource utilization. Each BFF may require its own set of resources. *Solution:* optimise BFFs for performance, use **serverless** where appropriate, and scale on demand.

Caching challenges. Caching gets complex when multiple frontends have different requirements — the same underlying data cached in three different shapes. *Solution:* intelligent caching at both BFF and frontend layers, and effective use of `Cache-Control` headers.

Load on backend services. Multiple BFFs can increase load on the shared backend services. Three BFFs each fetching the same product means three calls where there was one. *Solution:* ensure backend services are scalable, monitor their performance, and implement rate limiting if necessary.

> ⚠️ **The duplication trap is the one that actually bites.** Three BFFs slowly drifting apart on auth handling or error semantics is how "tailored" quietly becomes "inconsistent" — and users hit a bug on mobile that doesn't exist on web. Shared libraries are not optional at scale.

Part 7 — System design example: Netflix

Netflix serves smartphones, tablets, smart TVs, laptops, and gaming consoles. Each has distinct characteristics, user interactions, and performance considerations.

Scenario: viewing movie details — the user selects a movie and sees title, description, genre, rating, cast, recommendations, and user reviews.

Different devices prioritise differently:

Device	Priority
Smartphones	Quick loading, less data usage
Smart TVs	Rich visuals, detailed information, high-quality images
Web browsers	Balance between performance and rich content

Three BFFs: **Mobile BFF** (phones and tablets), **TV BFF** (smart TVs), **Web BFF** (browsers).

Mobile BFF

Serves concise movie details, smaller images, and **a subset of recommendations** to optimise for speed and data usage.

- **Authentication** — ensures the user is logged in and has a valid subscription
- **Routing** — directs the request to the appropriate microservices (Movie Details Service, Recommendation Service)
- **Aggregation** — combines data from those services and tailors it for mobile
- **Transformation** — reduces image sizes, shortens descriptions, formats data for quick mobile rendering

TV BFF

Serves rich visuals, detailed movie information, and a comprehensive list of recommendations.

- **Authentication & authorization** — checks not just login status but **whether the device is authorized to play high-definition content**
- **Routing & aggregation** — similar to Mobile BFF, but optimised for TV capabilities and network conditions
- **Transformation** — high-resolution images, full cast lists, elaborate descriptions

Web BFF

Balances performance and content richness.

- **Authentication** — standard login check
- **Routing & aggregation** — tailored for web browsers
- **Transformation** — content optimised for a wide range of browsers and screen sizes

The benefits realised

- **User experience** — each device gets an optimised, tailored experience
- **Performance** — data streamlined per device type, improving load times
- **Maintainability** — frontend teams work closely with their own BFF, leading to faster iterations

The challenges realised

- **Maintenance** — each BFF must be maintained and updated, creating overhead
- **Consistency** — ensuring consistent behaviour across all BFFs is genuinely challenging

> **The TV BFF's extra HD-entitlement check is the clearest single argument for BFF over one shared gateway.** That rule is meaningless for phones and browsers. In a shared gateway it would sit behind a device-type conditional, polluting a component every team must touch. In the TV BFF it's simply that BFF's business.

Part 8 — BFF vs. API Gateway

	API Gateway	BFF
How many	One shared entry point	One per client type
Optimised for	All clients generically	One specific client
Client-specific logic	Pollutes it	Belongs there
Main risk	Bottleneck / distributed monolith	Duplication and drift
Owned by	Usually a platform team	Usually the frontend team

They are not mutually exclusive. A very common production setup is a thin shared gateway handling TLS termination, coarse authentication, and rate limiting, with BFFs sitting behind it. The gateway does what's identical for everyone; the BFFs do what differs.

Think of BFF as API Gateway specialised per client, and note the ownership difference — a BFF being owned by the team that owns the frontend is a large part of why it delivers agility.

Part 9 — Common beginner misconceptions

"Each BFF needs its own database." No. BFFs are generally **stateless**. They call shared microservices and reshape responses. A BFF with its own database has usually turned into a microservice by accident.

"BFF means one backend per screen." No — per **client type**. All mobile screens share the Mobile BFF.

"You need BFF as soon as you have a mobile app." Only when needs genuinely diverge. If mobile and web want nearly the same data, a shared gateway is simpler. Adopt BFF when you're adding device-conditionals to shared code.

"GraphQL makes BFF obsolete." GraphQL lets clients request exactly the fields they want, solving over-fetching without separate backends — a genuine alternative for *shaping*. But it doesn't solve client-specific *logic* (the HD-entitlement check) or independent scaling and ownership. Some teams use both: a GraphQL layer per client type.

"BFFs make the system faster." Not inherently — they add a hop. They make it faster *for the client* by cutting round trips and payload size, which dominates on mobile networks.

Part 10 — Check your understanding

1. What is over-fetching, and why does it matter more on mobile than desktop?
2. In the worked example, what is the **only** thing that differs between Web BFF and Mobile BFF?
3. Why is it wrong for a BFF to own a database?
4. What's the strongest argument for the TV BFF over a shared gateway?
5. Name the main cost of BFF and its standard mitigation.
6. When should you *not* use BFF?

(Answers: 1 — downloading more data than needed; mobile has metered data, slow networks, limited CPU/memory; 2 — the transformation step and its return type, both call the same service for the same object; 3 — BFFs are projection layers, not sources of truth — one service owns each piece of data; 4 — the HD-entitlement authorization check is meaningless to other clients and would pollute shared code; 5 — code duplication across BFFs, mitigated by shared libraries and internal services; 6 — when clients' needs are nearly identical, or the team is too small to maintain several backends.)

Part 11 — Interview soundbite (30 seconds)

> "BFF, from Sam Newman, means giving each client type its own backend instead of sharing one. The problem it solves is over-fetching and one-size-fits-all APIs — a mobile app on a metered connection shouldn't receive a payload designed for a desktop, and it shouldn't need six round trips on a slow network to build one screen. Each BFF calls the same underlying microservices but aggregates and transforms for its client: mobile gets small images and short descriptions, TV gets high-res and full cast lists. Crucially the BFF is a projection layer, not a source of truth — the microservices still own the data. Netflix does this across phones, TVs, and browsers, and their TV BFF has an extra authorization check for HD entitlement, which is a good example of client-specific logic that would pollute a shared gateway. It also lets frontend teams own their backend and ship independently, and lets you scale per client. The costs are code duplication across BFFs, more codebases to maintain, an extra network hop, and keeping behaviour consistent between them — so shared libraries and consumer-driven contract testing matter. It's essentially API Gateway specialised per client rather than one shared gateway, and the two are often combined."

Part 12 — Implementation

A. Runnable demo

Full source: `demos/BffDemo.java`

```
cd demos && ./gradlew runCh04
```

The essential structure — three BFFs, one upstream service, three projections:

```
// Mobile: trim the copy, shrink the image
new MobileProductDetails(
    p.id(), p.name(), p.price(),
    truncate(p.longDescription(), 120),
    p.hiResImages().get(0).replace("4k", "thumb"));

// Watch: two fields, nothing else
new WatchProductDetails(truncate(p.name(), 20), p.price());
```

B. What you should see

```
=== 1. The over-fetching problem ===
  shared API payload:   3,525 bytes
  >> a watch showing name + price downloads ALL of that

=== 2. Each BFF projects the same object differently ===
  Web BFF:      3,525 bytes  (100% of shared)
  Mobile BFF:     251 bytes  (  7% of shared)
  Watch BFF:       31 bytes  (  0% of shared)
  >> mobile saves ~3274 bytes PER PRODUCT

=== 3. All BFFs call the SAME upstream service ===
  ProductService invocations so far: 4
  >> BFFs are projection layers - one source of truth remains

=== 4. Client-specific logic belongs in its own BFF ===
  certified TV:     Full HD payload ... with 3 4K images
  uncertified TV:   SD-only payload ...
```

Mobile receives 7% of the shared payload. Multiply by a 50-item product list and it's the difference between a screen that loads instantly and one that visibly stalls on a train.

Section 3 is the conceptual check: every BFF called the *same* `ProductService`. **They add no data and own no data.** A BFF that starts storing things has stopped being a BFF.

C. Production — a BFF is just a small service

There's no special framework; a BFF is an ordinary application whose job is aggregation and shaping.

```java
@RestController
@RequestMapping("/mobile/v1")
public class MobileBffController {

    private final ProductClient products;
    private final ReviewClient reviews;
    private final InventoryClient inventory;

    @GetMapping("/products/{id}")
    public MobileProductView get(@PathVariable String id) {
        // Fan out in parallel - the BFF exists to save the client round trips
        var pf = CompletableFuture.supplyAsync(() -> products.byId(id));
        var rf = CompletableFuture.supplyAsync(() -> reviews.summaryFor(id))
                    .exceptionally(e -> ReviewSummary.empty());     // degrade, don't fail
        var inf = CompletableFuture.supplyAsync(() -> inventory.stockFor(id))
                    .exceptionally(e -> Stock.unknown());

        CompletableFuture.allOf(pf, rf, inf).join();
        return MobileProductView.from(pf.join(), rf.join(), inf.join());
    }
}
```

GraphQL as an alternative. It solves the *shaping* half of the problem — clients request exactly the fields they need, so one endpoint serves all clients without over-fetching:

```graphql
query MobileProduct($id: ID!) {
  product(id: $id) { id name price thumbnail }     # mobile asks for four fields
}
```

But it does **not** solve client-specific *logic* (the TV HD-entitlement check), independent scaling, or team ownership. A common production setup is **one GraphQL BFF per client type** — shaping from GraphQL, isolation from BFF.

Shared code across BFFs — the mitigation for the duplication problem:

```
bff-common/          <- auth, error format, tracing, resilience config
  ├─ mobile-bff/
  ├─ web-bff/
  └─ tv-bff/
```

D. Implementation pitfalls

1. Giving a BFF its own database. The single most common mistake. BFFs are **stateless projection layers**. Persistent state means you've accidentally created a microservice with duplicated ownership of data.

2. Sequential upstream calls. A BFF exists to cut client round trips. Making three sequential server-side calls just moves the latency rather than removing it. Fan out with `CompletableFuture.allOf`.

3. No fallback on secondary data. If reviews are down, a product page should still render. Note the `.exceptionally(...)` on the review and inventory futures but *not* on the product itself — the primary resource legitimately fails the request; supporting data must not.

4. Copy-pasting auth into every BFF. This is how three BFFs drift into three subtly different security behaviours. Extract to `bff-common` from the start.

5. One BFF per screen. Per **client type**, not per screen. Twenty BFFs is twenty deployment pipelines for no benefit.

6. Business rules migrating into the BFF. "Is this order eligible for a refund?" belongs in the order service, where every client gets the same answer. Only **presentation** decisions belong in a BFF.

7. Versioning by breaking changes. Old mobile app versions live in the wild for years. Version the BFF path (`/mobile/v1`, `/mobile/v2`) and keep old versions running — being able to do this is a major reason BFF exists.

Related patterns

- **API Gateway** — the shared-entry-point alternative, and frequent companion
- **Strangler Fig** — a BFF can bridge new frontends to legacy systems mid-migration
- **Service Discovery** — how BFFs locate the services they aggregate
- **Circuit Breaker** — a BFF calling several services needs protection when one fails

Service Discovery Pattern

Category: Communication **One-liner:** A registry that lets services find each other by *name* at runtime, instead of hardcoding network addresses that change constantly.

Part 0 — Concepts you need first

What is an IP address and a port?

An **IP address** identifies a machine on a network (e.g. `10.0.3.47`). A **port** identifies which program on that machine (e.g. `8080`). Together — `10.0.3.47:8080` — they identify one running service instance.

To call another service, you must know its address and port. That's the entire problem this pattern exists to solve: in modern deployments, **those numbers change all the time**.

What is an "instance"?

One running copy of a service. If your Order Service is busy, you run five copies to share the load — five **instances**, each with its own IP address, all running identical code.

This is **horizontal scaling**: handle more load by adding more copies, rather than by making one copy bigger.

What is a container, and why does it make addresses unstable?

A **container** packages an application with everything it needs to run, so it starts anywhere in seconds. **Kubernetes** is the standard system for running containers across a fleet of machines.

The key consequence: containers are **disposable**. Kubernetes starts them, stops them, moves them between machines, and replaces them during deployments. **Each new one typically gets a new IP address.** An address you looked up ten minutes ago may now point at nothing.

What is auto-scaling?

Automatically adding instances when load rises and removing them when it falls. AWS auto-scaling groups do this. It saves money — but it means **the number of instances and their addresses change throughout the day without anyone touching anything**.

What is load balancing?

Distributing incoming requests across multiple instances so no single one is overwhelmed. If you have five instances and one hundred requests, ideally each handles about twenty.

What is DNS?

The **Domain Name System** translates names into IP addresses — you type `example.com`, DNS returns the address. It's the internet's original name-to-address lookup, and some service discovery systems (notably Kubernetes) work by extending exactly this idea inside your cluster.

Part 1 — The problem

In a **monolith**, components are tightly integrated and deployed as one unit. Component A calling component B is just a **method call inside the same process** — no network, no address, nothing to discover. Service discovery is a non-problem.

In **microservices**, services are loosely coupled and deployed independently. They may live on different servers, in different data centres, even on different continents. Now every call is a network call to an address that must be known.

Three forces make hardcoded addresses unworkable.

1. Dynamic infrastructure

Modern deployments — virtual machines, containers, Kubernetes — give services **no fixed IPs or hosts**. Instances come and go, scale up and down, and get replaced during deployments.

The course's example: an AWS auto-scaling group launches new instances of a service or terminates them based on load, **each time with new IP addresses**. Keeping track of these moving targets is a serious challenge.

2. Undetermined number of instances

Multiple instances of the same service run for load balancing. So: **which instance should a client call?**

- Hardcoding one address means all traffic goes to that one instance — you've lost load balancing entirely, and if it dies, everything fails
- Maintaining a list by hand is error-prone, and becomes untenable as services and instances multiply

3. Coupling

A goal of microservices is that teams deploy and scale independently. But if every client must know every service's exact address, you have **tight coupling**. A location change means updating configuration in **every client that calls it**.

Deployment details leak into client configuration, and the agility you adopted microservices for disappears.

The concrete scenario

An online retail application. A user places an order:

- **Order Service** must call **Inventory Service** to check item availability
- Then call **Payment Service** to process payment

How does Order Service know where those are? What happens when Inventory moves to a new server? What happens when extra Payment instances spin up under load — how does Order Service learn they exist?

Part 2 — The solution

Introduce a **central registry and lookup mechanism**.

Dynamic registration

Each service instance, **when it starts, registers its address (host and port) with a central Service Registry**. When shutting down, it de-registers. The registry therefore always holds a current listing of active instances. Clients no longer need hardcoded addresses.

Lookup by logical name

Services register under a **logical name** — `OrderService`, `UserService`. A client needing to call a service simply asks the registry for the address of `"OrderService"`, and gets back the current location, or several if multiple instances exist.

> **This indirection is the entire trick.** Instance addresses can change freely because clients only ever know the *name*. It's the same idea as a phone book: you look up a person by name, and it doesn't matter that their number changed last year.

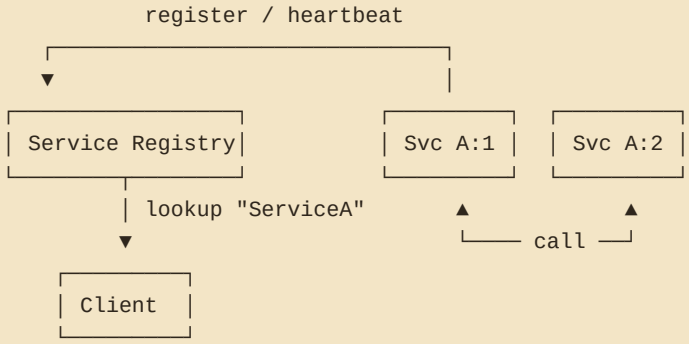
Load balancing and scaling

With multiple instances running, discovery works together with a load-balancing strategy. Scaling horizontally becomes trivial: **register more instances, and clients discover them automatically**.

Decoupled configuration

Clients maintain no configuration of service endpoints, staying decoupled from other services' deployment details. This makes continuous deployment and auto-scaling possible without disrupting clients. The client supplies a service name; **actual routing happens at runtime**.

Part 3 — Architecture



The Service Registry

A database holding all services and their current locations — the **centralized source of truth**, effectively a phone book for your system.

Three responsibilities:

1. Service Registration. When a service comes online it registers itself, providing its name, IP address, port, and potentially metadata such as service version or health endpoint.

2. Service Deregistration. Removing a service — either when it shuts down gracefully, or when the registry detects it's no longer available.

3. Health Checking. The registry periodically pings services or their health endpoints to check they're still running. If a service doesn't respond within a certain timeframe, it's considered failed and removed.

> *Why health checking is essential:* graceful deregistration only covers graceful shutdown. If an instance **crashes**, loses power, or gets partitioned off the network, it never gets the chance to say goodbye. Without health checks the registry would hand out dead addresses forever.

The payoff: the system **adapts dynamically**. A service moves or scales → the registry updates → others still find it. A service dies → it's removed → others can apply fallback strategies.

Part 4 — Client-side vs. server-side discovery

Both use a registry. They differ in **who performs the lookup and load balancing** — and this is the single most likely thing you'll be asked about in an interview.

Client-side discovery — "smart client"

The workflow:

1. **Service Registration** — each instance registers with the registry at startup, providing its address and perhaps a health heartbeat; it unregisters on shutdown.
2. **Service Lookup** — when a client needs to call a service, it **queries the registry directly** for a list of active instances.
3. **Client-side Load Balancing** — the client (or a library on the client side) selects one of the instances, often using round-robin or consistent hashing.
4. **Request Invocation** — the client makes the request **directly to the chosen instance**. Subsequent requests may repeat the lookup or use a cached result.

The client is **"smart"**: it contains logic to query the registry and balance load.

The classic stack: Netflix **Eureka** as the registry, combined with a client load-balancer library such as **Ribbon** or **Spring Cloud LoadBalancer**. The client knows the Eureka location, uses it to find instances, then calls one directly.

Advantages:

- **No extra network hop** — after discovery the client talks to the service directly
- **Intelligent decisions** — because the client knows all instances, it can implement sophisticated, application-specific load balancing (e.g. choose an instance based on request content)
- **Fewer moving parts at runtime** — aside from the registry, no additional routing layer is needed
- **Slightly lower latency** — no intermediate proxy

Disadvantages:

- **Every client needs the discovery logic and library**, increasing complexity
- **Painful in polyglot environments** — with services in multiple languages you need a compatible discovery client for each, or a standardized approach. This is a real barrier
- **Two network calls per request** — one to the registry, then the actual request (though caching usually eliminates the first)
- **Couples clients to the discovery mechanism** — if the registry changes, many clients may need updating, which is a maintenance burden

The standard mitigation: clients **cache** registry responses. This avoids a lookup on every call *and* lets the system keep working if the registry is temporarily down.

Server-side discovery — "smart router, dumb client"

The workflow:

1. **Service Registration** — identical; instances register at startup, deregister on shutdown.
2. **Request to Load Balancer** — the client sends its request to a **known, fixed endpoint** — a load balancer, gateway, DNS name, or virtual IP. **The client never queries the registry.**
3. **Service Discovery & Routing** — the load balancer receives the request and queries the registry for healthy instances.
4. **Load Balancing** — the load balancer selects an instance using its own algorithm.
5. **Forwarding** — it forwards the request to the chosen instance. From the client's perspective it simply made a request to a single endpoint; it has no idea any of this happened.

The intermediary is "smart" and the client is simple.

Advantages:

- **Much simpler clients** — one call to a fixed address, no discovery code, no special libraries
- **Language-agnostic** — any client that can make a normal HTTP request works, because discovery happens in the network infrastructure. This is the decisive advantage in a polyglot estate
- **One network call per request** — no separate lookup
- **Abstraction** — where services actually run can change without impacting clients
- **Often provided out of the box** — Kubernetes and cloud load balancers give you this for free

Disadvantages:

- **An extra hop** through the load balancer, adding slight latency
- **The load balancer must be highly available and scalable** — it's now critical to every request
- **Operational overhead** if your environment doesn't already provide one
- **The client loses control** over which instance it talks to — it can't tailor routing per request and must trust the router's decisions

Choosing between them

	Client-side	Server-side
Who queries the registry	The client	The load balancer / gateway
Network hops	Fewer (direct call)	One more (through the router)
Client complexity	Higher — needs a library	Very low
Polyglot friendliness	Poor	Excellent
Routing control	Fine-grained, app-specific	Whatever the router does
Typical stack	Eureka + Ribbon (Spring Cloud)	Kubernetes Services, cloud LBs

Practical guidance:

- In the **Spring Cloud / Netflix OSS** world, client-side (Eureka + Ribbon) is straightforward
- On **Kubernetes**, the platform's built-in discovery via DNS means you lean server-side — you simply call `http://my-service` and Kubernetes routes it
- **They're not mutually exclusive** — you might use client-side for internal service-to-service calls while having an API gateway (server-side style) for external traffic

In modern systems — Kubernetes, service meshes — **server-side discovery is the norm**, because the platform abstracts discovery behind DNS or a gateway.

Part 5 — Inner workings

Self-registration vs. third-party registration

Self-registration: services register and deregister *themselves* when they come online or go offline.

- *Upside:* simplicity, and the service has full control
- *Downside:* the service carries the added responsibility of keeping its status up to date

Third-party registration: a **service manager** handles registration. It monitors services and updates their status in the registry.

- *Upside:* takes the burden off the services entirely
- *Downside:* introduces another component that must itself be managed

Health checks — two models

Self-reporting (the heartbeat model). Services send a periodic signal to the registry saying they're still active. If the registry doesn't receive a signal within a given timeframe, it assumes the service is offline and removes it.

Active monitoring. The registry proactively checks services by sending a ping or similar request. No response → marked offline.

> The difference matters under load: heartbeats put work on the services and scale with instance count; active monitoring puts work on the registry and scales with *its* capacity to poll everyone.

Part 6 — Worked example (simplified Java)

The registry — a phone book

```java
import java.util.concurrent.ConcurrentHashMap;

public class ServiceRegistry {
    private ConcurrentHashMap<String, String> services = new ConcurrentHashMap<>();

    public void register(String serviceName, String serviceAddress) {
        services.put(serviceName, serviceAddress);
    }

    public String discover(String serviceName) {
        return services.get(serviceName);
    }
}
```

Reading it: keys are service names, values are addresses. `register` adds a service; `discover` retrieves an address by name.

Why `ConcurrentHashMap`? Registration and lookup happen from many threads simultaneously — many services registering while many clients look things up. A plain `HashMap` would corrupt under concurrent modification. `ConcurrentHashMap` is designed for exactly this.

A service registering itself

```java
public class Service {
    private String name;
    private String address;

    public void register(ServiceRegistry registry) {
        registry.register(name, address);
    }
}
```

This is **self-registration**. When the service starts it calls `register` to add itself. In a more sophisticated system this happens automatically — via a service manager, or as part of the deployment process.

A client discovering and calling

```
public class ClientService {
    private ServiceRegistry registry;

    public void callService(String serviceName) {
        String serviceAddress = registry.discover(serviceName);

        if (serviceAddress == null) {
            throw new RuntimeException("Service not found: " + serviceName);
        }

        // Make a request to the service
    }
}
```

The client uses the registry to find the address of the service it wants. If not found, it throws. If found, it proceeds with the request.

What this toy version is missing

The course is explicit that real implementations are more complex. Specifically:

- **Health checks** — services must regularly report in to keep their registration current
- **A distributed registry** — to handle high traffic and provide redundancy
- **Failure handling in `callService`** — retrying the request, or falling back to a backup service
- **A *list* of instances, not one address** — `Map<String, String>` stores exactly one address per service name, so this design **cannot load balance at all**. A real registry stores `Map<String, List<String>>` and returns all healthy instances so the client can choose

That last point is the most important gap to notice — it's the difference between a demo and something usable.

Part 7 — Performance implications and special considerations

1. Latency overhead

Every registry lookup is an extra step. In client-side discovery this is usually small because clients **cache** — fetching the instance list once and reusing it for many requests, refreshing every ~30 seconds or on a cache miss. The lookup is normally a fast local network call taking milliseconds.

To minimise latency:

- Cache and reuse discovery information for a short duration rather than querying per request
- Design the registry to respond quickly (in-memory indexes)
- Use asynchronous discovery updates — pushed updates or long-polling
- In high-performance scenarios, bake a small local endpoint cache into each instance at startup to avoid even the initial lookup delay

2. Scalability of the registry

It must handle registrations from potentially thousands of instances plus frequent lookups. It can become a bottleneck.

- **Clustering** — run the registry as a cluster of multiple nodes (e.g. three instances of Eureka or Consul) sharing load and providing redundancy. Configure clients with multiple registry URLs

- **Sharding** — partition registries by service domain. Uncommon; one cluster usually suffices for a large organisation
- **Efficient heartbeats** — thousands of instances heartbeating every 30 seconds is genuine load. Eureka lets you tune the heartbeat interval and use **self-preservation mode** to avoid overwhelming the network during an outage or partition
- **Lightweight protocols** — Consul uses an efficient **gossip** mechanism for health; Zookeeper uses an event-driven model with persistent connections

3. Fault tolerance and high availability

If the registry is down, services may be unable to find each other — especially with client-side discovery.

- **Run multiple registry instances** — at least two or three in production. Keep data in sync via consensus (Consul, Zookeeper) or peer-to-peer sync (Eureka)
- **Client-side caching for resilience** — many clients continue using the last known good information if the registry becomes unreachable. Spring's Eureka client does this by default, allowing the system to keep functioning assuming topology hasn't changed drastically
- **Failover of registry nodes** — configure clients with multiple registry addresses so they try the next when one fails
- **Monitor the registry itself**, not just the services it tracks
- **Data persistence** — Consul, etcd, and Zookeeper persist to disk and recover state after restart. Eureka keeps state in memory and relies on clients re-registering. Persistence helps recovery if all nodes reboot, but may be overkill if services simply re-register on startup

Consistency vs. availability — the CAP trade-off

> *Beginner note — CAP theorem:* in a distributed system, when a network partition splits your nodes apart, you must choose between **consistency** (never serve wrong data, even if that means serving nothing) and **availability** (always answer, even if the answer may be stale). You cannot have both during a partition.

	Zookeeper / Consul	Eureka
Choice	**CP** — consistency	**AP** — availability
Behaviour	Avoids serving possibly-stale data; may become unavailable until consensus is restored	Continues serving whatever it has, even if stale
On partition	May refuse to answer	**Self-preservation** — won't purge entries, so some info stays available

Be aware of your registry's behaviour and tune it. For example, turn **off** self-preservation in small setups to get quicker removal of dead instances.

> *Which is right?* For service discovery, **availability usually beats consistency**. A slightly stale instance list is survivable — you try an address, it fails, you retry another. A registry that refuses to answer means nothing can talk to anything. This is why Eureka chose AP.

4. Load balancing strategies

- **Round Robin** — each new request goes to the next instance cyclically. Distributes evenly assuming instances have similar capacity
- **Random** — pick randomly each time; roughly balances over many requests and is trivially simple

- **Least Connections / Load** — track which instance has fewest active requests and send there, balancing *actual* load. Requires more sophisticated tracking; usually done in server-side LBs or service meshes
- **Hashing (sticky sessions)** — consistently send the same client to the same instance, for caching or session affinity. E.g. hash of user ID modulo instance count
- **Geographic / AZ affinity** — in multi-datacenter or multi-availability-zone deployments, prefer an instance in the client's zone for lower latency. Discovery metadata carries the zone

Ribbon defaults to round-robin on the client side; a cloud ELB might do round-robin or least connections by configuration.

5. Failover handling

What happens when a call to an instance fails? With discovery you have multiple instances, so try another.

- **Client retry** — on timeout or error, query the registry again (or use the cached list), pick another instance, and retry. Ribbon and Spring Cloud LoadBalancer have retry mechanisms
- **Circuit breakers** — Hystrix or Resilience4j can trip if a service fails repeatedly and stop calling it for a cooldown. Combined with discovery this can be **per-instance**: if instance X of Service A is failing, stop sending to X while continuing with Y and Z. Discovery is what ensures Y and Z are available *(see Circuit Breaker)*
- **Health-based removal** — the registry should promptly mark failing instances unhealthy so lookups stop returning them. Proactive, and better than reacting at request time
- **Timeout tuning** — keep timeouts short. Fail fast and try another instance; that's viable precisely because you have a discovered pool

6. Consistency of data

The registry's view of which instances are up can lag reality. If a service crashes there's a window before the registry notices (the next heartbeat period), during which a client may be handed an instance that just died. To minimise impact:

- **Reduce heartbeat intervals** — faster detection, at the cost of more network chatter
- **Client pre-flight health check** before critical requests — adds overhead, rarely done
- **Tolerate the failed attempt and retry another instance** — usually the right answer
- In strongly consistent systems like Zookeeper, watchers are notified almost immediately when a client disconnects. But under high churn, an AP approach like Eureka may give better overall availability

7. Data volume and registry size

In very large deployments — hundreds of services, thousands of instances — the registry holds a large dataset. Most registries **index by service name** so lookups by name are fast. Ensure adequate RAM/CPU. Some registries support **tagging and filtering** — asking for instances of a service with a tag like `version: v2`, which is useful during rolling deployments of new versions.

8. Transactional consistency — subtle but important

If a service registers itself **before it is fully ready to serve traffic**, a client may discover and call it while it's still warming up — loading caches, opening connection pools, JIT-compiling.

> **Best practice:** register only once fully initialised, and **de-register immediately when shutting down, before actually going offline.** Design your startup and shutdown sequences with this in mind. This is a very common source of confusing intermittent errors right after a deployment.

Part 8 — Security considerations

The registry is critical infrastructure — it knows where everything is and helps route traffic. If compromised, an attacker can misroute traffic or map your entire topology.

Service spoofing (fake registrations). An unauthorized or malicious instance registers itself pretending to be a legitimate service. Other services then discover and communicate with it, letting it **intercept or manipulate data**. *Mitigation:* authentication and authorization for registration — the registry verifies a registering service is allowed to register under that name. Consul provides **ACLs and tokens**; Eureka can be secured with **HTTP basic authentication or mutual TLS**.

Unauthorized registry access. The registry holds sensitive information — internal service addresses, health status. An attacker who can query it freely gets **a map of all your services**. *Mitigation:* restrict access. Don't expose Eureka publicly; run it on an internal network. Require authentication on its API and UI. Consul ACLs can require a token to read. Kubernetes DNS is naturally cluster-isolated, and RBAC can further limit which services read which resources.

Communication security. Traffic between services and the discovery service may cross untrusted networks. *Mitigation:* **TLS** for all registry traffic, especially in multi-datacenter setups. If using an API gateway for server-side discovery, secure gateway-to-service and client-to-gateway calls too.

Least privilege. Not every service needs to know about every other service. *Mitigation:* segment the registry or use ACL policies so a given service discovers only what it needs. A "frontend" service might need `accounts` and `orders` but should not see internal services.

Audit and monitoring. Enable logging or audit trails. Watch for unusual patterns — a service registering from an unknown IP, or a spike in failed authentication against the registry.

DDoS and abuse. A discovery service flooded with registration requests or queries may slow or crash. *Mitigation:* capacity planning, and rate limiting on discovery operations.

Securing server-side components. If you use server-side discovery, the gateway or load balancer accepts client requests *and* talks to the registry — harden it accordingly.

> Treat service discovery infrastructure as **security-sensitive critical infrastructure**.

Part 9 — Real-world implementations

Netflix — Eureka. Netflix is often cited as a microservices pioneer. Hundreds of services register with Eureka, and Netflix applications query it to find instances. This lets Netflix scale services up and down and move them across data centres without clients knowing the details. Proven at streaming scale.

Kubernetes — the most widespread use today. Kubernetes has a **Service**: a stable virtual IP and DNS name for a set of pods. When new pods come up or old ones die, Kubernetes updates its internal endpoints list, and through kube-proxy ensures traffic to the Service is load-balanced to current pods. Any pod can simply look up `http://service-x` and reach it. Under the hood, Kubernetes maintains an **etcd**-based registry and a cluster DNS service (**CoreDNS**) answering queries from current registry data.

> This is why "we're on Kubernetes" is usually a complete answer to service discovery. You already have it, and building a second layer on top is usually a mistake.

Airbnb — SmartStack. Before Kubernetes was popular, Airbnb built SmartStack: **HAProxy** on every host plus **ZooKeeper** as the registry. Each service registered via a local agent, and the local HAProxy updated its configuration to route accordingly — effectively giving every application a local load balancer for other services. They built it because manually tracking service pools had become untenable.

Consul (Stripe, PagerDuty). HashiCorp Consul provides DNS-based discovery plus an HTTP API. Its ability to work across multiple data centres makes it popular in enterprises where Kubernetes isn't the orchestrator, or where a platform-agnostic solution is needed.

Service meshes — Lyft's Envoy. Envoy was built at Lyft to solve discovery, routing, and observability. Instead of each application doing discovery, **Envoy sidecars receive updates from a central control plane** about where every service is. When Service A calls Service B, it actually calls its local Envoy, which knows all instances of B and routes intelligently. **Discovery logic leaves application code entirely and moves into infrastructure.** *(See Sidecar.)*

Use cases

- **Microservices architectures** — managing inter-service connections becomes a nightmare at scale without it
- **Cloud-based systems** — services get allocated dynamically across servers and locations based on demand and cost; they re-register wherever they land

Design example 1 — e-commerce

Services: User, Inventory, Shopping Cart, Order. Without discovery, if Inventory moves to a new server, User and Shopping Cart must update their connections. With discovery, Inventory registers its new location and the others simply query the registry.

Design example 2 — streaming platform

Services for user profiles, video streaming, payments, recommendations. Streaming must scale hard at peak viewing times, adding and removing instances dynamically. Without discovery this causes broken connections to instances that no longer exist. With it, instances register and deregister as they come and go, and callers always get a valid one.

Part 10 — Common beginner misconceptions

"Service discovery replaces load balancing." No — they're complementary. Discovery tells you *what instances exist*; load balancing decides *which one to use*. Client-side discovery bundles both into the client; server-side puts both in the router.

"The registry is a single point of failure, so the pattern is bad." It is a SPOF, which is why you run it as a cluster *and* cache on clients. A registry outage should degrade you (stale data), not stop you.

"You need to install Consul or Eureka." Not if you're on Kubernetes — you already have discovery via DNS. Adding another registry usually creates two competing sources of truth.

"Registering at startup is enough." Registration must be paired with health checking and prompt deregistration, or you accumulate dead entries.

"DNS alone solves this." DNS has caching and TTL semantics that were designed for a world where addresses changed rarely. Clients (and language runtimes) cache DNS aggressively, sometimes forever. That's why Kubernetes pairs DNS with a stable virtual IP rather than returning raw pod addresses.

Part 11 — Check your understanding

1. Why doesn't a monolith need service discovery?
2. What is the "indirection" at the heart of the pattern?
3. Give the decisive advantage of server-side discovery in a polyglot organisation.
4. Why does Eureka choose availability over consistency?
5. Why must a service register only *after* it's fully initialised?
6. What's the most important gap in the toy Java registry example?

(Answers: 1 — components call each other in-process, there's no network address to find; 2 — clients know logical names, not addresses, so addresses can change freely; 3 — it's language-agnostic, since any client that can make an HTTP request works, with no per-language discovery library; 4 — a stale instance list is survivable via retry, but a registry that won't answer stops everything; 5 — otherwise clients discover and call it while it's still warming up, causing intermittent errors after deploys; 6 — it maps one name to one address, so it can't return multiple instances and therefore can't load balance.)

Part 12 — Interview soundbite (30 seconds)

> "Service discovery solves the problem that in a dynamic environment services have no fixed addresses — containers and auto-scaling groups get new IPs constantly, so hardcoding endpoints couples every client to every service's deployment details. Instances register with a service registry at startup under a logical name and deregister on shutdown, with health checks or heartbeats removing dead ones. Clients then look up by name, so addresses can change freely. There are two flavours. Client-side, where the client queries the registry and load-balances itself — Netflix Eureka plus Ribbon — which avoids an extra hop and allows smart routing, but needs a discovery library in every language you use. And server-side, where the client just calls a fixed load balancer or gateway that does the lookup — simpler and language-agnostic, but adds a hop and makes the LB critical. Kubernetes does server-side for you via CoreDNS and an etcd-backed registry, which is why on Kubernetes you usually shouldn't add another layer. The main considerations are running the registry as an HA cluster, caching on clients so a registry outage degrades rather than kills you, and the CAP trade-off — Consul and Zookeeper are CP, Eureka is AP and serves stale data rather than going down, which is usually the right call here. And you should secure it, because anyone who can write to the registry can spoof a service and intercept traffic."

Part 13 — Implementation

A. Runnable demo

Full source: `demos/ServiceDiscoveryDemo.java`

```
cd demos && ./gradlew runCh05
```

The single most important line, and the fix for the toy example's fatal flaw:

```
// A LIST of instances per name - Map<String,String> could never load balance
private final Map<String, Map<String, Registration>> services = new ConcurrentHashMap<>();
```

Eviction by heartbeat TTL — this is what catches crashed instances that never got to deregister:

```
void evictExpired() {
    long now = System.currentTimeMillis();
    services.forEach((name, m) -> m.entrySet().removeIf(e ->
        now - e.getValue().lastHeartbeat > ttlMs));
}
```

And the caching that makes a registry outage survivable:

```
List<Instance> instancesOf(String serviceName) {
    try {
        List<Instance> fresh = registry.discover(serviceName);
        cache.put(serviceName, fresh);
        return fresh;
    } catch (IllegalStateException e) {
        return cache.getOrDefault(serviceName, List.of());   // last known good
    }
}
```

B. What you should see

```
=== 2. Client-side discovery + round robin ===
  call 1 -> 10.0.0.2:8080
  call 2 -> 10.0.0.3:8080
  call 3 -> 10.0.0.1:8080      <- cycles evenly

=== 4. An instance dies (stops heartbeating) ===
    [registry] EVICTED 10.0.0.3:8080 (no heartbeat)
  >> crashed instance removed WITHOUT graceful deregistration

=== 5. The stale-entry window ===
  a client discovering NOW still gets: [..., 10.0.0.4:8080]
  >> unavoidable. The fix is not prevention but RETRY ON ANOTHER INSTANCE

=== 6. Registry outage - cache keeps the system alive ===
    [client] registry DOWN - using cached list (3 instances)
  still routing to 10.0.0.2:8080
  >> running services survive; only NEW registrations are blocked

=== 7. Filtering by metadata (canary routing) ===
  all instances: [10.0.0.2, 10.0.0.1, 10.0.0.4, 10.0.0.5]
  v2.0 only:     [10.0.0.5:8080]
```

Sections 5 and 6 are the ones that teach something. Section 5 shows the stale-entry window is genuinely unavoidable — there is always a gap between an instance dying and the registry noticing, so the answer isn't better detection, it's **retry against another instance**. Section 6 shows why client-side caching matters: the registry is down, and traffic still flows.

C. Production — you probably already have this

Kubernetes (do this by default). Discovery is built in; you don't install anything:

```yaml
apiVersion: v1
kind: Service
metadata:
  name: order-service
spec:
  selector:
    app: order-service          # matches pods by label
  ports:
    - port: 80
      targetPort: 8080
```

Any pod can now call `http://order-service` and CoreDNS resolves it, with kube-proxy load balancing across healthy pods. **Readiness probes are the health check** — this is the piece people skip:

```yaml
readinessProbe:                 # "am I ready to receive traffic?"
  httpGet: { path: /actuator/health/readiness, port: 8080 }
  initialDelaySeconds: 10
  periodSeconds: 5
livenessProbe:                  # "am I alive, or should I be restarted?"
  httpGet: { path: /actuator/health/liveness, port: 8080 }
```

> **Readiness is what enforces the Part 7.8 rule**: a pod that isn't ready is removed from the Service endpoints, so no traffic reaches it while it's still warming up.

Spring Cloud + Eureka (client-side, when you're not on Kubernetes):

```yaml
eureka:
  client:
    serviceUrl.defaultZone: http://eureka-1:8761/eureka,http://eureka-2:8761/eureka  # failover
    registryFetchIntervalSeconds: 30      # cache refresh
  instance:
    leaseRenewalIntervalInSeconds: 10     # heartbeat frequency
    leaseExpirationDurationInSeconds: 30  # TTL before eviction
```

```java
@LoadBalanced                           // makes RestTemplate resolve service names
@Bean RestTemplate restTemplate() { return new RestTemplate(); }

// Call by LOGICAL NAME, not address
restTemplate.getForObject("http://order-service/orders/{id}", Order.class, id);
```

Graceful shutdown — deregister *before* going offline, so the registry never hands out a dying instance:

```yaml
server.shutdown: graceful
spring.lifecycle.timeout-per-shutdown-phase: 30s
eureka.client.shouldUnregisterOnShutdown: true
```

D. Implementation pitfalls

1. Storing one address per service name. The flaw in the chapter's toy example. If your registry maps name → single address, you have a naming service, not a discovery service — no load balancing, no failover.

2. Registering before you're ready. Register only after caches are warm and connection pools are open. Otherwise clients discover you mid-startup and get errors that look random and appear only

after deploys. Use a **readiness probe**, not a liveness probe, to gate traffic.

3. No client-side caching. Without it your registry becomes a hard dependency for every request — a single point of failure that stops everything. With it, an outage is an inconvenience.

4. Heartbeat TTL too long. A 90-second TTL means up to 90 seconds of traffic routed to a dead instance. Too short and you evict healthy instances during a GC pause or network blip. 10s heartbeat / 30s expiry is a reasonable starting point.

5. Not deregistering on shutdown. Without graceful shutdown, every deploy leaves a window where clients get connection-refused errors from instances that have already stopped.

6. Adding a registry when you're on Kubernetes. Running Eureka inside Kubernetes gives you two competing sources of truth that disagree during rollouts. Use the platform's discovery.

7. Treating discovery as a substitute for retry. Discovery tells you what instances *were* alive. It cannot guarantee they still are. Always pair with retry-on-another-instance *(see Retry)*.

8. Leaving the registry unauthenticated. Anyone who can write to it can register a malicious instance under a legitimate service name and intercept traffic. Enable ACLs or mTLS.

Related patterns

- **API Gateway** — typically the server-side discovery router
- **Circuit Breaker** — per-instance breakers over a discovered pool
- **Retry** — retry against a *different* discovered instance
- **Sidecar** — service mesh sidecars (Envoy) absorb discovery entirely
- **Config Externalization** — services need discovery to find the config server too

Circuit Breaker Pattern

Category: Resilience **One-liner:** Wrap calls to a remote dependency in a component that stops calling it once it's clearly failing — failing instantly instead of hanging, and probing periodically to see whether it recovered.

Origin: Popularised by **Michael Nygard** in his book *Release It!*

Part 0 — Concepts you need first

What is a thread, and why do we care?

A **thread** is one line of execution inside a program. A typical web server keeps a **thread pool** — a fixed set of threads, say 200 — and assigns one to each incoming request. When the request finishes, the thread returns to the pool for reuse.

The critical property: a thread waiting on a network call is doing nothing else. It's blocked, occupying a slot, until it gets a response or gives up.

This single fact explains why the Circuit Breaker exists. If you have 200 threads and all 200 are blocked waiting on a slow service, your server can accept no requests at all — for *anything*. It looks completely dead, even though its own code is perfect.

What is a timeout?

The maximum time you'll wait for a response before giving up. If you call a service with a 30-second timeout and it never responds, your thread is stuck for the full 30 seconds.

Timeouts are necessary but not sufficient. They stop you waiting *forever*, but if a service is down and you keep calling it, every request still burns its full timeout before failing. A hundred requests a second against a dead service with a 30-second timeout means three thousand threads tied up. Timeouts bound each individual wait; the circuit breaker prevents the wait from happening at all.

What is a "cascading failure"?

When the failure of one component causes the failure of others that depend on it, spreading through the system like falling dominoes. The key insight is that **the components that fail second were never broken** — they were killed by waiting on something that was.

What is a fallback?

A substitute response used when the real one is unavailable — cached data from earlier, a sensible default, or a degraded version of the feature. It's how you turn a hard failure into a mild one.

What does "fail fast" mean?

Returning an error *immediately* rather than after a long wait. Counter-intuitively this is often better for users: two seconds of waiting followed by an error is worse than an instant, clear "this feature is unavailable right now."

Part 1 — The metaphor

An **electrical circuit breaker** in your home monitors current. When there's a dangerous overload it **trips**, cutting the circuit to prevent fire or damage to appliances. Later you flip it back on. If

the fault is still there, it trips again immediately.

Two features carry over precisely:

1. **It cuts the connection to prevent wider damage** — better to lose one circuit than the house.
2. **Restoration is a deliberate test**, not automatic — you flip it back and see what happens.

A software circuit breaker trips to stop calls to a failing service, letting the system **fail fast and degrade gracefully** rather than grinding to a halt.

Part 2 — The problem

Distributed applications constantly call remote things — other microservices, databases, external APIs. Every such call risks failure or a slow response. And if a service becomes unresponsive while the application keeps calling it, the results are disastrous: **requests pile up, threads hang, and the failure cascades into a wider outage.**

Cascading failures, step by step

Service A depends on Service B. B goes down, or — worse — becomes very slow.

1. Calls from A to B **hang until they time out**
2. During each wait, A's resources — threads, memory — are **tied up doing nothing**
3. This happens frequently, and across many threads simultaneously
4. A becomes overwhelmed — it **runs out of threads or memory**
5. **A now fails**

> Read step 5 carefully: **A is broken even though nothing is wrong with A's own code.** Its own logic is fine, its own database is fine. It died of waiting. That's the cascading failure, and preventing it is the entire purpose of this pattern.

And it doesn't stop at A. Whatever calls A now waits on A, and the same thing happens to them. One slow service at the bottom can take down everything above it.

> **Slow is worse than down.** If B refuses connections instantly, A gets an immediate error and moves on. If B accepts connections and then takes 30 seconds, A's threads are held hostage. Counter-intuitive, but a service that's merely slow does far more damage than one that's cleanly dead.

Unresponsive services and wasted work

Waiting on slow services with long timeouts means users experience delays and threads stay blocked.

Blindly retrying a known-failing operation wastes CPU cycles and **overloads the failing service even more**. The course's example: if dozens of components keep retrying against a database that's already struggling, it just makes things worse.

> This is the vicious circle: the service is slow because it's overloaded → clients time out and retry → the retries add *more* load → it gets slower → more timeouts and retries. Without intervention a service can be prevented from ever recovering, purely by the traffic of clients trying to reach it.

When a fault is unlikely to resolve quickly — a network outage, a crashed service — **continuously attempting the operation is pointless**. It doesn't solve the problem, and it prevents the application

from handling the failure gracefully.

What's needed: a way to detect failure quickly and stop sending traffic to the failing component *before* it drags down the entire system.

Part 3 — The solution

Wrap calls to an external service in a **Circuit Breaker** component that monitors the outcomes of those calls, and stops making them once failures cross a threshold.

Two things happen when it trips, and the second is easy to overlook:

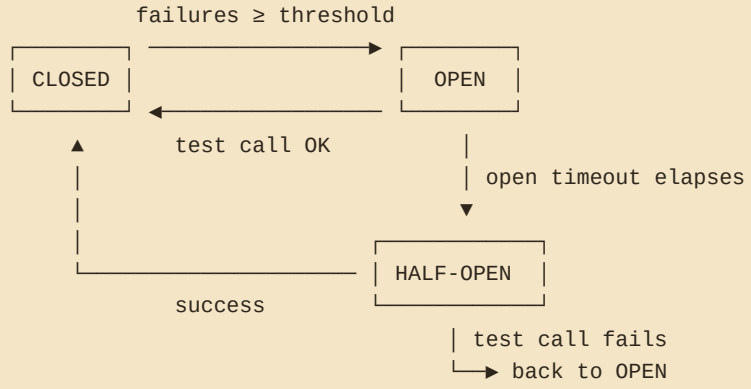
1. Your application stops waiting. Calls are rejected immediately, so you preserve your own responsiveness and resources. Threads stay free for work that can actually succeed.

2. The failing service stops receiving load. An open circuit gives the struggling service **a chance to actually recover** — breaking the vicious circle above.

The pattern prefers **fast failure and graceful degradation** over long waits and resource exhaustion.

Part 4 — The three states

This is the core of the pattern, and what interviewers ask about.



> *Naming warning — this trips people up constantly:* **CLOSED means traffic flows** and **OPEN means traffic is blocked.** It feels backwards until you remember it's an electrical circuit: a closed circuit is a complete loop that current flows through; an open circuit is broken, so nothing flows.

CLOSED — normal operation

The circuit is closed, so traffic flows to the service as usual. The breaker **monitors the number of recent failures**. If failures exceed a configured **failure threshold** within a certain time window, it transitions to Open.

While closed, **successful calls generally reset the failure counter**, so an occasional transient error won't trip the breaker immediately. This matters: you want to distinguish "one blip" from "this service is genuinely broken."

OPEN — tripped

The breaker has detected too many failures and opened the circuit. Calls are **blocked immediately** — the application gets an error or fallback response **without any attempt at the real operation**.

Open state lasts for a configured **timeout period** (the open interval). The purpose is twofold: avoid useless load, and give the troubled service time to recover.

HALF-OPEN — trial mode

After the open timeout expires, the breaker allows **a limited number of test requests** through to the real service. The course describes it as cautiously probing: *"Are you OK now?"*

- **A test request succeeds** → the service is likely back; the breaker resets to **Closed** (usually resetting failure counters)
- **A test request fails** → the breaker flips **immediately back to Open**, and the timeout period starts over

Only a few requests are let through in half-open — this is deliberate, **to avoid blasting a recovering service with traffic**. Often one success is enough to close the circuit; some implementations require several consecutive successes to be sure.

> Half-open is the cleverest part of the design. Without it you'd need either a human to reset the breaker, or a blind reopening of full traffic that would instantly re-kill a service that's only just getting back on its feet.

The per-call logic

Every call through the breaker performs a quick state check:

- **Closed** → attempt the call and record the outcome
- **Open** → skip the call and immediately return a failure or fallback
- **Half-Open** → allow the call (perhaps limiting concurrency), then adjust state based on the result

The breaker acts as a **safety valve**: it ensures a failing component doesn't drag down the rest.

Part 5 — Integration: it's additive, not invasive

Adding a circuit breaker **does not require a system overhaul**. It's a protective layer wrapped around existing service calls, leaving the underlying service logic untouched.

Every request is made *through* the breaker, and the breaker decides whether to forward it based on current state and recent outcomes.

Practical rollout advice from the course: introduce it gradually. Start with **critical services whose failure would significantly impact the system**, then expand to protect other services as the benefits become evident.

Part 6 — Configuration

Parameter	What it controls
Failure Threshold	The limit of failed requests before the breaker switches from Closed to Open
Timeout Period	How long the breaker stays Open before moving to Half-Open — the recovery time given to the service
Number of Requests in Half-Open	How many requests are allowed through while testing whether the service recovered

These values must **strike a balance** between protecting the service, allowing it time to recover, and keeping it as available as possible for handling requests.

The right settings depend on:

- The nature of the service and the system
- The expected load on the service
- How critical the service is to overall functionality

These parameters shouldn't be static

The course makes a point worth remembering: they **can and should be adjusted dynamically** in response to system state and performance. In times of high demand you might:

- **Increase the failure threshold** — let the service endure more failed requests before tripping
- **Reduce the timeout period** — attempt recovery sooner, so the service can handle more requests during peak times

---

Part 7 — Worked example (Java)

A basic implementation with no external libraries, to illustrate the core algorithm.

```java
public class CircuitBreaker {
    private enum State { CLOSED, OPEN, HALF_OPEN }

    private State state = State.CLOSED;
    private int failureCount = 0;
    private final int failureThreshold;
    private final long openTimeout;      // how long to stay open before half-open
    private long lastFailureTime = 0;

    public CircuitBreaker(int failureThreshold, long openTimeout) {
        this.failureThreshold = failureThreshold;
        this.openTimeout = openTimeout;
    }

    public synchronized <T> T call(Callable<T> remoteCall) throws Exception {

        // 1. If OPEN, has the timeout passed?
        if (state == State.OPEN) {
            long timeSinceFailure = System.currentTimeMillis() - lastFailureTime;
            if (timeSinceFailure < openTimeout) {
                // Still within the open window — fail fast, don't call the service
                throw new IllegalStateException(
                    "Circuit is OPEN. Failing fast without calling remote service.");
            } else {
                // Timeout has passed — allow one trial call
                state = State.HALF_OPEN;
            }
        }

        // 2. Attempt the call (allowed when CLOSED or HALF_OPEN)
        try {
            T result = remoteCall.call();
            // Success: reset the counter and close the circuit
            failureCount = 0;
            state = State.CLOSED;
            return result;
        } catch (Exception e) {
            // Failure: count it and record when it happened
            failureCount++;
            lastFailureTime = System.currentTimeMillis();
            if (failureCount >= failureThreshold) {
                state = State.OPEN;   // trip
            }
            throw e;   // propagate (or return a fallback here)
        }
    }
}
```

Walking through the mechanics

The state check comes first. Before doing anything else the breaker asks whether it's Open. If so, and the timeout hasn't elapsed, it throws immediately — **the remote service is never contacted**. That's the whole point: no network call, no thread blocked, microseconds instead of seconds.

`Callable<T>` makes it generic. `Callable` is a Java interface for "some operation that returns a value and may throw." Accepting one means this breaker can wrap *any* operation — an HTTP call, a database query, anything — rather than being tied to one specific service.

`synchronized` handles concurrency. Many threads may call this simultaneously. Without synchronization two threads could both read `failureCount` as 2, both increment it, and you'd lose a count. *(This is also the example's main performance weakness — see below.)*

Success resets everything. Note that success sets `state = CLOSED` unconditionally. From Half-Open, that's the recovery path. From Closed, it stays closed and clears accumulated failures.

Failure records the time. `lastFailureTime` is what the Open-state check measures against to know when the timeout has elapsed.

Using it

```
CircuitBreaker cb = new CircuitBreaker(3, 10000);  // trip after 3 failures, stay open 10s

try {
    String data = cb.call(() -> unreliableService.getData());
    // use data...
} catch (Exception e) {
    useCachedData();   // fallback
}
```

What happens in sequence:

1. `getData()` fails three times in a row → the circuit opens
2. Any attempt within the next 10 seconds throws **immediately**, without calling `getData()` at all
3. After 10 seconds the next call is allowed through (Half-Open)
4. If it succeeds → circuit closes, normal operation resumes
5. If it fails → circuit re-opens and blocks for another 10 seconds

What this example simplifies

The course notes production breakers are more sophisticated. Specifically:

- **A naive counter, not a rolling window.** Here, three failures trip the breaker *whenever* they happen — even three failures spread across an hour of otherwise perfect traffic. Real implementations use a **rolling time window** ("more than 50% of calls in the last 10 seconds failed"), which is far more meaningful.
- `synchronized` **serialises every call.** Every request through this breaker acquires the same lock, so it becomes a bottleneck under load. Production libraries use **atomic operations and non-blocking synchronization** instead.
- **All exceptions count as failures.** A validation error from the remote service would trip the breaker even though the service is perfectly healthy. Real breakers distinguish which exceptions indicate the *service* is unwell.
- **No fallback built in.** The breaker throws and the caller handles it. Most frameworks let you register a fallback function directly.

Part 8 — Performance implications

The breaker adds a small overhead per call — checking state and updating counters. **This overhead is negligible** compared to the alternative.

The comparison that matters: without a breaker, a failing call ties up a thread for **several seconds** until timeout. With one, the failure is handled in **milliseconds**.

> A tiny check is a small price to pay for avoiding a meltdown.

For extremely performance-sensitive scenarios there are options like asynchronous calls or optimised breaker logic, but generally the benefits far outweigh the overhead.

Tuning the failure threshold

- **Too low** → the circuit trips on every minor glitch, causing unnecessary interruptions and **flapping** (rapid on/off cycling)
- **Too high** → the breaker waits too long to trip, failing to protect the system until a lot of errors have already occurred

The course's calibration: tripping after **1** failure is too sensitive; tripping after **100** is too late. Pick a threshold that **distinguishes normal transient failures from a real problem**.

Tuning the open timeout

- **Too short** → the breaker goes Half-Open and tests the service **before it has recovered**, causing another immediate failure and creating noisy open → half-open → open **flapping**
- **Too long** → your application refuses calls longer than necessary, degrading user experience even after the dependency is healthy again

The timeout should roughly correspond to **how long you expect the service might need to recover**.

Finding the right values

This requires monitoring and tweaking. Good practice: monitor **how often the breaker opens, how long it stays open, and whether it's tripping too frequently.** Metrics like failure counts, open events, and half-open trial outcomes should feed dashboards. With that data you adjust.

In a high-traffic system, consider a **percentage-based threshold** rather than an absolute count, so occasional spikes don't trip it.

Part 9 — Best practices and trade-offs

Fallbacks. When a breaker opens, have a fallback strategy rather than just returning an error — cached data, a default value, or redirecting to a simpler alternative. This is what makes the system **degrade gracefully instead of failing completely**. Many frameworks let you specify a fallback function to execute when open.

Combine with retries and timeouts. The Circuit Breaker doesn't replace other resilience patterns — it works alongside them. You still implement **timeouts** for calls, and may **retry** a few times on certain failures before counting one as a failure. They work in tandem: retry a couple of times in case it's a transient network glitch, but let the breaker stop you trying after repeated failures that indicate a real outage. **Bulkheads** and rate limiting complement both.

Monitoring and alerts. A tripped breaker usually means something is wrong with a downstream service, so **set up alerts**. Have the breaker log or emit an event whenever it opens or closes; your monitoring can catch these and alert on-call engineers or trigger automated recovery.

> Without this, the pattern has a hidden danger: the system quietly protects itself, users see a degraded feature, and **nobody realises anything is broken** because there's no flood of errors. The breaker suppresses exactly the signal that would normally raise the alarm.

Thread-safety and concurrency. In multi-threaded server applications, internal counters and state transitions **must be thread-safe** — you don't want two threads concurrently deciding they're both "first" to open the circuit. Use atomic counters, synchronized sections, or a library. High-performance libraries use non-blocking synchronization and atomic operations to minimise overhead.

Avoiding misuse. Apply breakers to calls that are **potentially unreliable or high-latency**, like network calls. There's **no benefit** on in-memory operations that are very unlikely to fail. Be cautious in systems where failures are very localised or quickly self-correcting — **an aggressive**

breaker constantly tripping on ephemeral errors does more harm than good. The pattern must be tuned to your system's reliability needs.

Part 10 — Use cases

Microservices architecture

An e-commerce application split into user management, product catalog, shopping cart, and order processing. Order processing relies on the product catalog to validate availability and user management to validate credentials.

If the **product catalog starts failing**, without safeguards those failures propagate into order processing, degrading the user experience or causing a total outage. With a breaker in front of the catalog, order processing detects failures quickly, stops sending requests, and falls back to a **cached product list** — or fails gracefully with a relevant message.

External API integration

External APIs are **outside your control** and can be unpredictable: downtime, latency spikes, rate limiting policies.

A weather forecasting application pulling from several external weather APIs puts a breaker on **each one**. If one starts failing or becomes slow, the breaker isolates that specific API so its problems don't affect overall performance. The application can then **switch to another API** or provide degraded service until the faulty one recovers.

Database access

Database issues — network problems, resource contention, server failures — cause slow responses or errors.

- **Reads:** in a read-heavy system, a breaker monitors database health. On detecting an increasing error rate or latency, it trips and **redirects reads to a read replica or a cache**, keeping the service available
- **Writes:** the same can apply, but **with more caution**. On increased errors the breaker could trip and temporarily **buffer write operations** — but this must be handled carefully, as **data consistency is at risk** and the buffer itself could become a bottleneck

---

Part 11 — Real-world examples

Netflix and Hystrix. Netflix famously implemented this pattern through their open-source library **Hystrix**. In their microservices web, if a backend service becomes slow or fails, Hystrix isolates that failing endpoint, stopping the failure cascading and returning fallback responses so the user experience is only minimally affected.

> The canonical illustration: **if the recommendations service is down, Netflix can still stream video.** The app omits the recommendations row or shows a default message, rather than failing entirely. That's the whole pattern in one sentence — a non-essential dependency failing must not take down the essential function.

E-commerce product pages. A large site with services for product info, inventory, pricing, and reviews. If the **reviews service** starts failing, a breaker on those calls trips so the product page **skips reviews** rather than slowing the entire page. Product details and pricing load normally, and

the page shows something like "Reviews are unavailable at the moment" — **graceful degradation**. Without the breaker, the page might hang or crash trying endlessly to fetch reviews. Similar strategies apply to payment gateways and shipping calculators: if the dependency is down, fall back quickly ("Sorry, we can't process payments right now") instead of making users wait and wonder.

Cloud and service meshes. Many cloud architectures incorporate breakers to improve stability when calling remote services, and service mesh technologies provide breaker mechanisms **at the infrastructure level**, automatically preventing one overloaded microservice from overloading others. In an **API Gateway** you might configure breaker rules per backend route, so an erroring backend gets rejected for a short period — protecting the backend and freeing the gateway for other routes.

Enterprise integration. Even outside microservices, any integration with an external system — a SOAP or REST API, a database, a messaging system — benefits. A middleware system calling a slow legacy SOAP service uses a breaker to **avoid tying up all its worker threads** on that one slow dependency. The pattern also appears in reactive programming and stream processing for **backpressure**: if a downstream consumer is too slow or failing, the upstream producer trips a breaker and stops feeding it until it recovers.

Libraries: Hystrix (Java, now in maintenance mode), **Resilience4j** (Java), **Polly** (.NET), plus built-in support in cloud platforms and service meshes.

Part 12 — System design example: distributed social media platform

Services: **User Management**, **Post Management**, **Feed Generation**, and more — each running on multiple nodes for high availability and load balancing.

Breakers apply in several places:

In front of User Management. Services relying on it — Post Management, Feed Generation — quickly detect when it's struggling. They then **reduce the load on it** by providing degraded service: displaying **cached user information** or **simplified post feeds**.

Between Post Management and its database. If the database experiences problems, the breaker trips and Post Management starts **serving cached posts**, or **temporarily stops accepting new posts**.

In front of the external email service used for notifications. If it starts failing, the breaker prevents the notification feature from impacting the rest of the system.

Distributed breaker state

An interesting design consideration: use a **shared circuit breaker per service, stored in a distributed cache**. This maintains consistency across all nodes — if a breaker trips on one node, it trips on all of them.

This must be **balanced against the additional complexity and potential performance impact** of consulting a distributed cache on every call.

> The trade-off in plain terms: with per-node breakers, each node must independently discover the service is broken, so each burns its own set of failures learning the same lesson. With shared state, one node's discovery protects everyone — but now every call depends on a cache lookup, and the cache becomes something else that can fail.

Monitoring

Complement the breakers with **comprehensive monitoring and alerting** — monitor the state of all breakers and alert developers or administrators when one trips, ensuring quick detection and remediation.

> **The goal is not to eliminate failures** — they're inevitable in any system. The objective is to **manage failures effectively**, preventing them from cascading into system-wide outages.

Part 13 — Common beginner misconceptions

"Open means working." The most common mix-up. Open = blocked, Closed = flowing. Think electrical circuits.

"The circuit breaker retries for you." No. It *stops* calls. Retry is a separate pattern; they're used together but do opposite things — retry tries harder, the breaker gives up sooner.

"It fixes the broken service." It doesn't. It protects the *caller* and gives the failing service breathing room. Someone still has to fix the actual problem — which is why alerting matters.

"One breaker per application." No — one breaker **per dependency**. If your service calls three others, you need three independent breakers. A shared one would mean a failure in the reviews service blocks your calls to payments.

"If I have timeouts I don't need this." Timeouts bound each individual wait; the breaker prevents the waiting from happening at all once the service is known bad. You need both.

"It makes my system slower." The state check is microseconds. Compared to a 30-second timeout on every call to a dead service, it makes failure handling thousands of times faster.

Part 14 — Check your understanding

1. Why does CLOSED mean traffic flows?
2. Why is a slow service more dangerous than a completely dead one?
3. What two separate benefits does an open circuit provide?
4. Why does Half-Open allow only a *few* requests through?
5. What's the hidden risk of a circuit breaker working well?
6. Why does the example's `synchronized` become a problem at scale?

(Answers: 1 — it's an electrical metaphor; a closed circuit is a complete loop that current flows through; 2 — a dead service refuses instantly so callers move on, while a slow one holds caller threads hostage until timeout; 3 — the caller stops wasting resources waiting, and the failing service stops receiving load so it can recover; 4 — to avoid overwhelming a service that has only just started recovering; 5 — it suppresses the errors that would normally alert you, so problems go unnoticed without explicit alerting on breaker state; 6 — every call acquires the same lock, so the breaker itself becomes a bottleneck under high concurrency.)

Part 15 — Interview soundbite (30 seconds)

> "A circuit breaker stops you repeatedly calling a dependency that's already failing. The problem is cascading failure: if Service B hangs, Service A's threads block waiting on it, and A runs out of threads and dies even though A's own code is fine. Slow is actually worse than down, because a dead service refuses instantly while a slow one holds your threads hostage. The breaker wraps the call and has three states — Closed, where traffic flows and failures are counted; Open, once failures cross a threshold, where calls are rejected instantly with a

> fallback; and Half-Open, after a timeout, where a few trial requests probe whether it recovered. Success closes it, failure re-opens it, and it's only a few requests so you don't re-kill a service that's just getting back up. It does two things at once: your service stops wasting resources waiting, and the failing service stops receiving load so it can actually recover — that second part breaks the vicious circle where retry traffic prevents recovery. Netflix's Hystrix is the classic example: if recommendations is down they still stream video and just drop that row. Tuning matters — too sensitive and it flaps, too lenient and it doesn't protect you, so you generally want a percentage-based threshold over a rolling window rather than a raw count. And you must alert on breaker state, because a breaker working well hides the errors that would otherwise tell you something's broken."

Key takeaways: prevent cascading failures · fail fast and recover · tune and monitor · use in combination with other patterns.

Part 16 — Implementation

A. Runnable demo — watch the state machine work

Full source: `demos/chapters/06-circuit-breaker/CircuitBreakerDemo.java`

```
cd demos && ./gradlew runCh06
```

```java
import java.util.concurrent.Callable;
import java.util.concurrent.atomic.AtomicInteger;

public class CircuitBreakerDemo {

    // ---------- The circuit breaker ----------
    static class CircuitBreaker {
        enum State { CLOSED, OPEN, HALF_OPEN }

        private State state = State.CLOSED;
        private int failureCount = 0;
        private final int failureThreshold;
        private final long openTimeoutMs;
        private long lastFailureTime = 0;
        private final String name;

        CircuitBreaker(String name, int failureThreshold, long openTimeoutMs) {
            this.name = name;
            this.failureThreshold = failureThreshold;
            this.openTimeoutMs = openTimeoutMs;
        }

        synchronized <T> T call(Callable<T> action, Callable<T> fallback) throws Exception {
            if (state == State.OPEN) {
                if (System.currentTimeMillis() - lastFailureTime < openTimeoutMs) {
                    log("REJECTED (circuit OPEN) -> fallback");
                    return fallback.call();          // fail fast, no remote call
                }
                transitionTo(State.HALF_OPEN);       // timeout elapsed: allow a probe
            }

            try {
                T result = action.call();
                if (state == State.HALF_OPEN) {
                    log("probe SUCCEEDED");
                }
                failureCount = 0;
                transitionTo(State.CLOSED);
                return result;
            } catch (Exception e) {
                failureCount++;
                lastFailureTime = System.currentTimeMillis();
                log("call failed (" + failureCount + "/" + failureThreshold + "): " + e.getMessage());
                if (state == State.HALF_OPEN || failureCount >= failureThreshold) {
                    transitionTo(State.OPEN);
                }
                return fallback.call();
            }
        }

        private void transitionTo(State next) {
            if (state != next) {
                log("state " + state + " -> " + next);
                state = next;
            }
        }

        private void log(String msg) {
            System.out.printf("    [%s] %s%n", name, msg);
        }
    }
```

```
    // ---------- A service that fails for a while, then recovers ----------
    static class FlakyService {
        private final AtomicInteger calls = new AtomicInteger();
        private volatile boolean healthy = true;

        String getData() {
            calls.incrementAndGet();
            if (!healthy) {
                throw new RuntimeException("service unavailable");
            }
            return "real-data";
        }

        void breakIt()  { healthy = false; }
        void healIt()   { healthy = true; }
        int callCount() { return calls.get(); }
    }

    public static void main(String[] args) throws Exception {
        FlakyService service = new FlakyService();
        CircuitBreaker cb = new CircuitBreaker("payments", 3, 2000); // 3 failures, 2s open

        System.out.println("\n--- Phase 1: service healthy ---");
        for (int i = 0; i < 3; i++) {
            System.out.println("  result = " + cb.call(service::getData, () -> "CACHED"));
        }

        System.out.println("\n--- Phase 2: service breaks ---");
        service.breakIt();
        for (int i = 0; i < 6; i++) {
            System.out.println("  result = " + cb.call(service::getData, () -> "CACHED"));
        }
        System.out.println("  >> service was actually called "
                + service.callCount() + " times so far");

        System.out.println("\n--- Phase 3: still broken, circuit stays open ---");
        int before = service.callCount();
        for (int i = 0; i < 3; i++) {
            cb.call(service::getData, () -> "CACHED");
        }
        System.out.println("  >> extra real calls during OPEN: "
                + (service.callCount() - before) + "  (should be 0)");

        System.out.println("\n--- Phase 4: wait for timeout, service recovers ---");
        Thread.sleep(2100);
        service.healIt();
        System.out.println("  result = " + cb.call(service::getData, () -> "CACHED"));
        System.out.println("  result = " + cb.call(service::getData, () -> "CACHED"));
    }
}
```

B. What you should see

```
--- Phase 1: service healthy ---
  result = real-data          (x3, circuit stays CLOSED)

--- Phase 2: service breaks ---
    [payments] call failed (1/3): service unavailable
    [payments] call failed (2/3): service unavailable
    [payments] call failed (3/3): service unavailable
    [payments] state CLOSED -> OPEN
    [payments] REJECTED (circuit OPEN) -> fallback
  >> service was actually called 6 times so far

--- Phase 3: still broken, circuit stays open ---
  >> extra real calls during OPEN: 0

--- Phase 4: wait for timeout, service recovers ---
    [payments] state OPEN -> HALF_OPEN
    [payments] probe SUCCEEDED
    [payments] state HALF_OPEN -> CLOSED
  result = real-data
```

The line that proves the pattern works is `extra real calls during OPEN: 0`. Three requests came in; the failing service received none of them. That's both benefits at once — the caller didn't wait, and the sick service got left alone.

C. Production — Resilience4j

Nobody hand-writes breakers in production. **Resilience4j** is the current standard for Java (Hystrix is in maintenance mode).

Dependency (Maven):

```
<dependency>
  <groupId>io.github.resilience4j</groupId>
  <artifactId>resilience4j-spring-boot3</artifactId>
  <version>2.2.0</version>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-aop</artifactId>
</dependency>
```

Configuration (`application.yml`**)** — this is where the real learning is:

```yaml
resilience4j:
  circuitbreaker:
    instances:
      paymentService:
        # Use a percentage over a rolling window, not a raw count
        slidingWindowType: COUNT_BASED
        slidingWindowSize: 20              # consider the last 20 calls
        minimumNumberOfCalls: 10           # don't judge until we have 10 samples
        failureRateThreshold: 50           # open if >50% of them failed
        waitDurationInOpenState: 10s       # how long to stay OPEN
        permittedNumberOfCallsInHalfOpenState: 3
        automaticTransitionFromOpenToHalfOpenEnabled: true
        # Treat slow calls as failures too - crucial, see pitfall 3
        slowCallRateThreshold: 50
        slowCallDurationThreshold: 2s
        recordExceptions:
          - java.io.IOException
          - java.util.concurrent.TimeoutException
        ignoreExceptions:
          - com.example.ValidationException   # never trip on a 400
```

Usage:

```java
@Service
public class PaymentClient {

    @CircuitBreaker(name = "paymentService", fallbackMethod = "paymentFallback")
    public PaymentResult charge(Order order) {
        return restClient.post()
                .uri("/payments")
                .body(order)
                .retrieve()
                .body(PaymentResult.class);
    }

    // Signature must match, plus a trailing Throwable
    private PaymentResult paymentFallback(Order order, Throwable t) {
        log.warn("Payment circuit open for order {}: {}", order.getId(), t.toString());
        return PaymentResult.deferred(order.getId());   // queue for later
    }
}
```

Monitoring — expose breaker state so you can alert on it:

```yaml
management:
  endpoints.web.exposure.include: health,circuitbreakers,prometheus
  health.circuitbreakers.enabled: true
```

Then alert on the Prometheus metric `resilience4j_circuitbreaker_state`.

D. Implementation pitfalls

1. Using a raw failure count instead of a rate over a window. Three failures scattered across an hour of healthy traffic shouldn't trip anything. Always use `slidingWindowSize` + `failureRateThreshold`, and set `minimumNumberOfCalls` so a single early failure at 100% rate doesn't open the circuit.

2. Counting business errors as failures. A 400 or a validation exception means the *request* was bad, not the *service*. Put those in `ignoreExceptions` or you'll trip the breaker on a client bug and take

down a healthy dependency.

3. Forgetting `slowCallDurationThreshold`. A service returning 200 OK after 30 seconds never "fails," so a failure-only breaker never opens — while your threads are consumed exactly as badly as if it were down. Slow-call detection is what catches the most dangerous case.

4. No timeout on the underlying call. The breaker can only react to a call that *returns*. Always set connect and read timeouts on your HTTP client; otherwise a hung socket blocks forever and the breaker never sees a failure to count.

5. One breaker shared across dependencies. Name breakers per downstream service. A shared instance means the reviews service failing blocks your payment calls.

6. Fallbacks that call other services. Your fallback runs when things are already broken. It must be local and fast — cache, default, or queue. A fallback making a network call is a second failure waiting to happen.

7. Not alerting on state. Covered above but it's the one people skip: a breaker doing its job hides errors. Alert on the open transition or you'll find out from a customer.

Related patterns

- **Bulkhead** — isolates resource pools; the breaker limits *how long* you try, the bulkhead limits *how much* damage one failure can do
- **Retry** — handles transient faults; the breaker stops retries becoming an attack on a downed service
- **Service Discovery** — per-instance breakers across a discovered pool
- **API Gateway** — a natural place to configure breaker rules per backend route
- **Sidecar** — service meshes provide breakers at the infrastructure level, with no application code

Bulkhead Pattern

Category: Resilience **One-liner:** Partition a system into isolated compartments with their own dedicated resources, so a failure or overload in one cannot consume the resources everything else needs.

Part 0 — Concepts you need first

What is a thread pool?

A fixed collection of worker threads a program keeps ready. When work arrives, a free thread picks it up; when the work finishes, the thread returns to the pool.

Pools are fixed-size because threads are expensive — each needs memory (typically ~1 MB of stack), and having thousands means the operating system spends more time switching between them than doing work.

The consequence that drives this entire chapter: the pool is a *shared, finite resource*. If something exhausts it, **every** piece of work in that program stops — not just the work that caused the problem.

What does "resource exhaustion" mean?

Using up all of a limited resource so nothing else can get any. The resources that matter here:

- **Threads** — all busy, so no new work can start
- **Memory** — allocated, so new allocations fail
- **Database connections** — connection pools are also fixed-size and also exhaustible
- **Network bandwidth / file handles / CPU**

What is a "noisy neighbour"?

A component that consumes disproportionate shared resources and degrades everything sharing them. The term comes from cloud hosting, where one tenant's heavy usage slows other tenants on the same physical machine. In your own application, one slow dependency can be a noisy neighbour to every other feature.

Why does one slow dependency block unrelated work?

This is the crux, so it's worth walking through concretely.

Your service has **one thread pool of 200 threads**. It calls two dependencies: Inventory (usually busy) and Shipping (usually quiet).

Inventory becomes slow — 30 seconds per call instead of 50 milliseconds.

1. Requests needing Inventory arrive and each takes a thread
2. Each thread blocks for 30 seconds waiting
3. At any moderate request rate, all 200 threads end up blocked on Inventory
4. A request arrives needing only Shipping — **there is no thread available to serve it**

Shipping is perfectly healthy. Its requests still fail. They fail because Inventory ate all the threads. That's the failure this pattern prevents.

Part 1 — Where the name comes from

A ship's hull is divided into **watertight compartments** called bulkheads. If the hull is breached, water floods **one compartment** rather than the whole ship. The ship stays afloat, buying valuable time for rescue.

Without bulkheads, a single breach floods everything and the ship sinks rapidly. The failure — one hull breach — propagates across the entire ship and causes total system failure. That's a **cascading failure in a physical system**.

In distributed systems, a bulkhead is a mechanism that **isolates different parts of the system so that if one part fails, it doesn't cause the rest to fail.**

Part 2 — The problem: failure propagation

In an ideal world services would work perfectly all the time. In reality they fail — bugs, resource exhaustion, network issues, and more. Each service has its own responsibilities, resources, and failure modes.

When a service fails, it causes a **ripple effect**: the failure of one service leads to the failure of others that depend on it.

The e-commerce walkthrough

A microservices-based e-commerce platform with separate services for user management, inventory management, payment processing, and so on.

1. The **inventory service** fails due to a database outage
2. The **user service**, which relies on inventory to display product availability, **also starts failing**
3. The **payment service**, which checks inventory before processing payments, **likewise fails**
4. In a short time, **the entire platform is unavailable**

All because of a failure in one service. This is a classic cascading failure.

Part 3 — The solution: isolation

The essence of the Bulkhead pattern is **isolation**. Like the watertight compartments in a ship, it partitions system components into isolated units. Each is designed to operate **independently of the others**, so if one fails, the others continue functioning.

Think of bulkheads as **failure containment**. By isolating parts of the system, you limit the **scope** of any potential failure. A failure in one part doesn't automatically mean failure of the whole — unaffected parts still provide some level of service, maintaining availability as much as possible.

Applied to the e-commerce example

If the inventory service were isolated into its own bulkhead, its failure wouldn't directly impact the user and payment services. They could continue operating, providing limited functionality — browsing products, viewing past orders — despite inventory being down.

The half people miss

There's a second, subtler benefit. By preventing the inventory failure from **consuming system-wide resources** like CPU or memory, the pattern ensures those resources **remain available for the remaining services**.

> Isolating *resources* matters as much as isolating *code*. Two services can be perfectly separate in the source tree and still kill each other if they draw from the same thread pool. The bulkhead is about the resources, not the modules.

Not a magic solution

The course is careful here, and it's worth internalising: bulkheads don't make failures disappear. If a ship has a significant hull breach it might eventually sink despite its bulkheads. Likewise, if a distributed system experiences a major failure like a total network outage, it may become unavailable despite bulkheads.

What they do is **buy time and maintain some level of service**, significantly mitigating the impact.

Part 4 — Architecture

The architecture revolves around two concepts: **isolation** and **resources**.

What is a bulkhead, concretely?

At a high level, a bulkhead is simply a **partition** within the system. It could be:

- A separate **service**
- A group of related **processes**
- A set of **threads** — a thread pool, the most common implementation
- A dedicated **CPU or memory area**

The key requirement is that each operates **independently** of the others.

What are the resources?

Anything the bulkheads need to function: CPU time, memory, network bandwidth, database connections. Each bulkhead gets a **dedicated share**, ensuring a failure in one doesn't consume all the system's resources.

Step 1 — Divide the system into bulkheads

How you divide depends on your system's specifics and which failures you want to isolate. Common approaches:

- Isolate services that are **prone to failure**
- Isolate services handling **particularly resource-intensive tasks**
- Group services by **function**, or by the **type of users they serve**

In the e-commerce example, you'd create separate bulkheads for user management, inventory management, payment processing, and so on. Each operates independently with its own resources.

The aim: if one service fails **or becomes a resource hog**, the others aren't directly affected.

Step 2 — Allocate resources

Allocation should be **proportional to the needs and importance** of each bulkhead. A resource-intensive or critical service gets more than a less demanding or less critical one.

In the e-commerce platform you might allocate more to **payment processing** (which handles financial transactions) than to **user management** (which mainly handles profiles). The specific allocation depends on:

- Expected load on each service

- Its performance requirements
- The consequences of its failure

This is the critical mechanism: if a bulkhead fails and starts consuming resources excessively, the allocation **guarantees other bulkheads still have resources available**.

Step 3 — Manage interactions between bulkheads

Since each bulkhead operates independently, interactions between them must be carefully managed or failures propagate anyway.

Keep interactions to a minimum. The more a bulkhead depends on other bulkheads, the higher the chance of a failure spreading.

When interactions are necessary, isolate them. If the user management service must interact with inventory, it could do so via a **message queue** or an **API gateway**. That way, if inventory fails, the failure doesn't directly impact user management — the interaction stays isolated and manageable.

> Why a message queue helps: with a direct call, the caller waits for a response and its thread is held while the callee struggles. With a queue, the caller drops a message and moves on immediately. The failure becomes "messages accumulate in a queue" instead of "caller's threads are all blocked."

Part 5 — Inner workings

Isolation in practice

Maintaining independence involves careful design and implementation choices. Each bulkhead might:

- Run in its **own process or container**
- Have its **own database and cache**
- Use its **own thread pool**

But the course makes a point that's easy to skip past and shouldn't be:

> Achieving isolation is **not just about technical measures**. It also requires an **organisational commitment** to maintaining independence. Changes to one bulkhead should not require changes to others. Interdependencies must be minimised and carefully managed.

In other words, you can build perfect technical isolation and then destroy it over six months by letting teams add convenient direct dependencies between compartments. The isolation has to be defended, not just built.

Resource allocation — fairness and efficiency

The aim is **fairness** (each bulkhead gets the resources it needs) and **efficiency** (resources aren't wasted). In practice this involves:

- Setting **resource quotas or limits** per bulkhead
- Scheduling resources fairly and efficiently
- **Dynamically adjusting** allocation based on real-time conditions

For instance, if one bulkhead is experiencing a surge while another is idle, the system might temporarily reallocate some resources from the idle one to the busy one, improving overall performance and responsiveness.

Failure detection and mitigation

The ultimate test is how the pattern handles failures. When one occurs within a bulkhead, the system needs to **detect it, isolate it, and mitigate its impact**. This typically involves monitoring, alerting, and automatic recovery:

- The system **continuously monitors** the health and performance of each bulkhead
- If a problem is detected, it **generates alerts** and starts **automatic recovery**
- If a bulkhead starts consuming resources excessively, the system might automatically **throttle its resource usage or restart it**
- If it remains unresponsive, the system might **reroute traffic** to other bulkheads or **bring up new instances** to handle load

It's not a silver bullet

The pattern can't prevent failures from happening and can't guarantee 100% uptime or performance. It's one tool in the toolbox, to be complemented with **redundancy, replication, load balancing, and caching**.

Part 6 — Worked example: thread-pool bulkheads in Java

The scenario

An **Order Service** responsible for creating orders. It must communicate with two others:

- **Inventory Service** — often gets high traffic
- **Shipping Service** — usually has less load

Without bulkheads: a spike in requests to Inventory could clog the Order Service, preventing it from completing its tasks involving Shipping.

Creating separate thread pools

Java's `ExecutorService` is well suited to creating thread pools:

```
ExecutorService inventoryExecutor = Executors.newFixedThreadPool(100);
ExecutorService shippingExecutor  = Executors.newFixedThreadPool(50);
```

Reading it: `newFixedThreadPool(100)` creates a pool with exactly 100 threads. Submit more than 100 tasks and the extras queue until a thread frees up.

The pool sizes should be determined by the load each service is expected to handle. Inventory gets the larger pool (100) because it typically faces more traffic; Shipping gets 50.

These two pools are the bulkheads. They are separate compartments; threads in one cannot be consumed by work destined for the other.

Using them

```
public void createOrder(Order order) {
    inventoryExecutor.submit(() -> {
        inventoryService.checkAndUpdateInventory(order);
    });

    shippingExecutor.submit(() -> {
        shippingService.scheduleShipping(order);
    });
}
```

`createOrder` initiates tasks that interact with both services. Each task is **submitted to its own executor**, ensuring a slowdown in one service doesn't affect the other.

The bulkhead in action

Now replay the failure scenario:

1. A significant number of requests start slowing down the **Inventory Service**
2. Threads in `inventoryExecutor` get stuck or slowed — potentially all 100 of them
3. **The 50 threads in `shippingExecutor` are completely unaffected**, because they're a separate pool
4. `createOrder` can **continue scheduling shipping** even while inventory checks are delayed

The system uses its resources efficiently and provides a faster response where it can.

Compare this to the single-pool version: all 200 threads blocked on Inventory, and shipping requests failing for no reason of their own. **The only change is drawing from two buckets instead of one.**

What the example deliberately leaves out

The course itself raises these as questions leading into the next section:

- **Error handling** — what if a service doesn't respond at all, or throws an exception?
- **Timeouts** — a stuck thread needs a bound, or it's stuck forever *(pair with Circuit Breaker)*
- **Pool sizing** — how do you actually determine those numbers?

Also worth noticing: `submit()` returns a `Future` that this code ignores, so **failures are silently swallowed**. Real code would capture and handle the result.

Part 7 — Performance implications

Benefits

Efficient resource utilization. Even when one service is sluggish or non-responsive, the system continues processing other tasks. The risk of the entire system freezing or significantly slowing due to one faulty component is mitigated.

Load balancing. Allocating different thread pools to various tasks or services distributes the system's load. The distribution can be fine-tuned based on the nature and expected load of each.

Response time. By ensuring a delay in one part doesn't affect others, overall response time improves. Even if some requests are delayed by an issue in one component, others are served swiftly.

Considerations

Determining bulkhead sizes. One of the real challenges. Ideally sizes are decided by expected load and the nature of the tasks — but striking the right balance often involves **a degree of trial and error**.

> A practical starting point: size each pool to the concurrency that dependency can actually handle. If Inventory can serve 100 concurrent requests before degrading, a pool larger than 100 just queues work inside the dependency instead of inside you — which is worse, because you lose visibility.

Monitoring and adjusting. Over time, as load and task nature evolve, initial allocations need adjustment. Monitor performance and adjust sizes as needed. Tools like Java's **VisualVM** help monitor thread usage and identify bottlenecks.

Error handling. Even with bulkheads, some requests may fail or be excessively delayed if a service goes down or becomes unresponsive. Handle these gracefully — implement a **timeout**, or use another resilience pattern like the **Circuit Breaker**.

Scalability concerns. This one deserves emphasis:

> The Bulkhead pattern improves system **resilience** but **does not directly address scalability**. If load increases significantly, simply creating more threads leads to **excessive context switching** and eventually **decreases** performance. For scale, use horizontal scaling (more machines) or asynchronous processing.

⚠️ **This is the point beginners get wrong most often.** Bulkheads bound damage; they don't add capacity. Reaching for bigger thread pools as a solution to load makes things slower, not faster, because the CPU spends its time switching between threads rather than running them.

Part 8 — Use cases

E-commerce platforms

Customer actions span searching for products, adding items to carts, and placing orders. These are interconnected but can be performed independently.

If the **search service** is overwhelmed by a traffic spike or a bug causing excessive load, without bulkheading this could slow the entire platform — **affecting even cart and ordering services**. Isolating services into separate bulkheads ensures an issue in one doesn't impact the others.

Online gaming platforms

A platform offering various games, each running as a separate service. **Player requests to one game shouldn't affect the performance of other games.** Isolating game services prevents a faulty or overloaded game from affecting the entire platform.

Financial systems

Distinct operations: account management, transaction processing, and reporting. These are interrelated but can function independently.

> **A delay in generating reports shouldn't affect the speed of transaction processing.** This is the cleanest illustration of the principle — reporting is typically heavy and slow, transactions are latency-critical, and there is no good reason for the former to interfere with the latter.

Part 9 — System design example: music streaming (Spotify-like)

A large-scale music streaming platform with services for music streaming, user management, playlist management, and recommendation generation.

Bulkheading services (coarse-grained)

User management isolated from music streaming. Even if user management becomes overloaded due to a sudden surge in new users, this doesn't affect streaming performance. **Users can still listen to music uninterrupted.**

Playlist management isolated from recommendations. Even if the recommendation service becomes slow due to the complexity of processing and analysing user data, playlist management is unaffected and users can still create and manage their playlists.

Bulkheading at a lower level (fine-grained)

Bulkheading can also be applied **within a single service**. Inside the music streaming service you could separate:

- the task of **fetching a song** (from the song database)
- the task of **streaming the song** to the user

into **separate thread pools**. So if fetching songs becomes slow due to a delay accessing the song database, this **won't slow down the streaming of songs already fetched**.

> This two-level view is the most useful takeaway from the chapter. Bulkheads apply *between* services **and** *within* a single service. The fine-grained version is often what actually saves the user experience — a user whose music keeps playing while search is broken barely notices anything is wrong.

In both cases, applying the pattern dramatically increases resilience: while one service or task is slow or failing, others continue functioning normally, providing an uninterrupted experience.

Part 10 — Bulkhead vs. Circuit Breaker

These two get confused constantly. They're complementary, not alternatives.

	Bulkhead	Circuit Breaker
Mechanism	Partitions resources	Stops calls after repeated failures
Protects against	Resource exhaustion spreading	Wasted calls to a known-dead service
When it acts	Always — it's structural	On detecting failures — it's reactive
Analogy	Watertight compartments	A fuse that trips
Limits	*How much* damage one failure can do	*How long* you keep trying

Use both. The bulkhead caps the blast radius; the breaker stops you feeding the fire. In the Order Service example, the bulkhead means Inventory can only ever consume its own 100 threads — and a breaker on Inventory means those 100 threads stop being wasted once it's clearly down.

Part 11 — Common beginner misconceptions

"Bulkhead means microservices." No. You can bulkhead inside a single monolith by giving different workloads different thread pools — and that's often the highest-value place to start.

"Bigger pools are safer." The opposite past a point. Threads cost memory and context-switching. The pool should match what the dependency can handle, not what you wish it could.

"It's the same as rate limiting." Related but different. Rate limiting caps how many requests you *accept*; bulkheading caps how many resources a category of work can *consume*.

"Isolation is a one-time build." It erodes. Teams add convenient cross-dependencies over time. It needs ongoing defence — which is why the course calls it an organisational commitment.

"Bulkheads help with load." They help with *failure*. For load you scale horizontally.

Part 12 — Check your understanding

1. Why can a slow Inventory Service cause Shipping requests to fail, when Shipping is healthy?
2. What are the two things a bulkhead isolates, and which do people forget?
3. Why does a message queue between bulkheads help contain failure?
4. Why doesn't the Bulkhead pattern help with scalability?
5. Give an example of fine-grained bulkheading inside a single service.
6. How does Bulkhead differ from Circuit Breaker?

(Answers: 1 — they share one thread pool, and Inventory's blocked threads consume all of it; 2 — code and resources, and people forget resources; 3 — the caller drops a message and moves on instead of holding a thread waiting for a struggling callee; 4 — more threads cause excessive context switching and degrade performance, so capacity comes from horizontal scaling; 5 — separate thread pools for fetching songs versus streaming already-fetched songs; 6 — bulkhead limits how much damage a failure can do and is always active, breaker limits how long you keep trying and activates on detected failures.)

Part 13 — Interview soundbite (30 seconds)

> "Bulkhead comes from ship design — watertight compartments so one hull breach doesn't sink the whole ship. In software you partition resources so a failing dependency can't consume everything. The classic case is an Order Service calling Inventory and Shipping from one shared thread pool: if Inventory gets slow, every thread ends up blocked waiting on it, so Shipping calls start failing even though Shipping is perfectly healthy. The fix is separate thread pools sized per dependency, so Inventory's problems are bounded to its own hundred threads. You can apply it between services or inside one service — for example separating song fetching from song streaming, so a slow database doesn't interrupt playback of what's already loaded. It pairs with circuit breakers: bulkhead caps how much damage a failure can do, the breaker caps how long you keep trying. The main gotchas are that sizing the pools is genuinely trial and error and needs monitoring, and that it improves resilience but not scalability — adding threads to handle more load just causes context switching, so capacity has to come from horizontal scaling."

Part 14 — Implementation

A. Runnable demo — see thread starvation, then see it fixed

This program runs the *same* workload twice: once with a shared pool, once with bulkheads. Full source: `demos/chapters/07-bulkhead/BulkheadDemo.java`

```
cd demos && ./gradlew runCh07
```

```java
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicInteger;

public class BulkheadDemo {

    static final int POOL = 10;      // small pool so the effect is visible
    static final int REQUESTS = 20;

    // Inventory is slow (2s). Shipping is fast (50ms) and perfectly healthy.
    static String slowInventory() throws InterruptedException {
        Thread.sleep(2000);
        return "inventory-ok";
    }

    static String fastShipping() throws InterruptedException {
        Thread.sleep(50);
        return "shipping-ok";
    }

    // ---------- WITHOUT bulkheads: one shared pool ----------
    static void withoutBulkheads() throws Exception {
        ExecutorService shared = Executors.newFixedThreadPool(POOL);
        AtomicInteger shippingDone = new AtomicInteger();
        long start = System.currentTimeMillis();

        for (int i = 0; i < REQUESTS; i++) {
            shared.submit(() -> { slowInventory(); return null; });
        }
        // Shipping work arrives after the inventory flood
        for (int i = 0; i < 5; i++) {
            shared.submit(() -> {
                fastShipping();
                shippingDone.incrementAndGet();
                return null;
            });
        }

        shared.shutdown();
        boolean finished = shared.awaitTermination(3, TimeUnit.SECONDS);
        System.out.printf("  shipping completed in 3s: %d/5   (finished=%s, %dms)%n",
                shippingDone.get(), finished, System.currentTimeMillis() - start);
    }

    // ---------- WITH bulkheads: separate pools ----------
    static void withBulkheads() throws Exception {
        ExecutorService inventoryPool = Executors.newFixedThreadPool(POOL);
        ExecutorService shippingPool  = Executors.newFixedThreadPool(5);
        AtomicInteger shippingDone = new AtomicInteger();
        long start = System.currentTimeMillis();

        for (int i = 0; i < REQUESTS; i++) {
            inventoryPool.submit(() -> { slowInventory(); return null; });
        }
        for (int i = 0; i < 5; i++) {
            shippingPool.submit(() -> {
                fastShipping();
                shippingDone.incrementAndGet();
                return null;
            });
        }

        Thread.sleep(3000);
```

```
        System.out.printf("  shipping completed in 3s: %d/5   (%dms)%n",
                shippingDone.get(), System.currentTimeMillis() - start);
        inventoryPool.shutdownNow();
        shippingPool.shutdownNow();
    }

    public static void main(String[] args) throws Exception {
        System.out.println("\n=== WITHOUT bulkheads (shared pool of " + POOL + ") ===");
        withoutBulkheads();

        System.out.println("\n=== WITH bulkheads (separate pools) ===");
        withBulkheads();
    }
}
```

B. What you should see

```
=== WITHOUT bulkheads (shared pool of 10) ===
  shipping completed in 3s: 0/5   (finished=false, 3001ms)

=== WITH bulkheads (separate pools) ===
  shipping completed in 3s: 5/5   (3000ms)
```

That's the whole pattern in two lines of output. Same workload, same total resources. In the first run, twenty slow inventory tasks occupy all ten threads, and five trivial 50ms shipping tasks get **zero** of them. In the second, shipping has its own pool and finishes immediately — while inventory is just as slow as before.

Nothing was made faster. Something was made *isolated*.

C. Production — Resilience4j Bulkhead

Two flavours, and choosing correctly matters:

Type	Mechanism	Use when
`SemaphoreBulkhead`	Caps **concurrent calls** on the caller's own thread	Default choice — cheap, no extra threads
`ThreadPoolBulkhead`	Runs calls on a **separate thread pool**	You need true thread isolation, or async execution

Configuration:

```
resilience4j:
  bulkhead:                          # semaphore-based
    instances:
      inventoryService:
        maxConcurrentCalls: 25       # at most 25 in flight
        maxWaitDuration: 100ms       # then reject rather than queue forever
      shippingService:
        maxConcurrentCalls: 10
        maxWaitDuration: 100ms

  thread-pool-bulkhead:              # thread-pool-based
    instances:
      reportGeneration:
        maxThreadPoolSize: 4
        coreThreadPoolSize: 2
        queueCapacity: 20
```

Usage:

```
@Bulkhead(name = "inventoryService", fallbackMethod = "inventoryFallback")
public Inventory check(String sku) {
    return inventoryClient.get(sku);
}

private Inventory inventoryFallback(String sku, BulkheadFullException e) {
    // Bulkhead full — shed load rather than queue
    return Inventory.unknown(sku);
}
```

Kubernetes-level bulkheads — the same idea applied to whole services:

```
resources:
  requests: { cpu: "500m", memory: "512Mi" }
  limits:   { cpu: "1000m", memory: "1Gi" }   # this pod cannot starve its neighbours
```

Database connection pools are bulkheads too — and often the most valuable one:

```
spring.datasource.hikari:
  maximum-pool-size: 20
  connection-timeout: 3000     # fail fast when the pool is exhausted
```

D. How to size the pools

The most-asked question, and there's a real formula to start from — **Little's Law**:

```
concurrency = throughput × latency
```

If a dependency serves **50 requests/second** at **200ms** average latency, then `50 × 0.2 = 10` concurrent calls are in flight at steady state. Size the bulkhead a bit above that — say 15 — to absorb bursts.

The key insight: don't size it larger than the dependency can actually handle. If Inventory degrades beyond 100 concurrent requests, a bulkhead of 500 doesn't help you — it just moves the queue from your process into theirs, where you can't see it or shed it.

Then measure and adjust. `maxWaitDuration` short (~100ms) means you **shed load** rather than accumulate it, which is what you want.

E. Implementation pitfalls

1. Pools sized by guesswork. Start from Little's Law, then tune with real metrics. "200 because that's the default" is how you end up with a pool that can't protect anything.

2. Unbounded queues. `Executors.newFixedThreadPool` uses an **unbounded** `LinkedBlockingQueue`. Under sustained overload it accumulates tasks until you get an `OutOfMemoryError` — the bulkhead prevented thread exhaustion and gave you memory exhaustion instead. Use a bounded queue with an explicit rejection policy:

```
new ThreadPoolExecutor(10, 10, 0L, TimeUnit.MILLISECONDS,
    new ArrayBlockingQueue<>(50),                    // bounded
    new ThreadPoolExecutor.AbortPolicy());           // reject when full
```

3. Ignoring the `Future`. `submit()` swallows exceptions into a `Future` nobody reads — failures vanish silently. Either use `execute()` with an `UncaughtExceptionHandler`, or actually check the result.

4. Forgetting the database connection pool. Teams bulkhead their HTTP calls beautifully and leave one shared 20-connection Hikari pool, where one slow query starves every endpoint. The connection pool is usually the *scarcest* resource you own.

5. Bulkheading without timeouts. A bounded pool full of threads blocked forever is still a dead pool. Bulkhead limits *how many* can be stuck; only a timeout limits *how long*.

6. Too many small pools. Every pool holds idle threads at ~1 MB of stack each. Twenty pools of fifty threads is a thousand threads and roughly a gigabyte before you serve a request. Bulkhead by *dependency*, not by endpoint.

7. Treating it as a scaling tool. Repeated because it's the most common error: bigger pools do not mean more capacity. Past the CPU's ability to run them, more threads means more context switching and *less* throughput.

Related patterns

- **Circuit Breaker** — the natural companion; stop calling what has already failed
- **Retry** — retries inside a bounded bulkhead can't exhaust the whole system
- **Sidecar** — service meshes can enforce bulkheads at the infrastructure level
- **Event-Driven Architecture** — message queues between components are a natural isolation boundary

Retry Pattern

Category: Resilience **One-liner:** Automatically re-attempt operations that failed for temporary reasons, spacing attempts out with backoff and randomness, so brief glitches don't become user-visible errors.

Part 0 — Concepts you need first

What is a "transient" failure?

Transient means temporary — it goes away on its own without anyone fixing anything. The opposite is **permanent**: it will fail identically no matter how many times you try.

The distinction is the single most important idea in this chapter, because **retrying a transient failure is useful and retrying a permanent one is pure waste**.

Transient (retry helps)	Permanent (retry is useless)
Network dropped a packet	The password is wrong
Server briefly out of connections	The request body is malformed
Momentary CPU spike	You lack permission
Load balancer restarted an instance	The record doesn't exist

What is "backoff"?

Waiting between retry attempts instead of retrying instantly. **Exponential backoff** means each wait is longer than the last — typically doubling: 1s, 2s, 4s, 8s.

Why wait at all? Because retrying instantly against a struggling service adds load at the worst possible moment, and gives it no time to recover.

What is "jitter"?

Deliberate randomness added to the wait time. Instead of every client waiting exactly 4 seconds, each waits a random duration around 4 seconds.

It sounds pointless until you see the problem it solves — see the thundering herd below.

What is the "thundering herd" problem?

Imagine a service has a two-second blip. During that blip, **ten thousand clients** fail. Every one of them starts a 4-second backoff at almost the same instant.

Four seconds later, **all ten thousand retry simultaneously**. The service, which had just recovered, is hit by a synchronised wall of traffic and falls over again. Now they all back off 8 seconds, and the same thing happens again, harder.

The clients have accidentally organised themselves into a denial-of-service attack. **Jitter breaks the synchronisation** by spreading those retries across a window.

What does "idempotent" mean?

An operation is **idempotent** if performing it multiple times has the same effect as performing it once.

- `set balance to 100` — **idempotent**. Do it five times, balance is 100.
- `add 100 to balance` — **not idempotent**. Do it five times, you've added 500.

This matters enormously for retries, because a retry might be the *second* execution of something that already succeeded.

Part 1 — The problem

In a distributed microservices architecture, **failures are inevitable**. Services constantly communicate over networks and depend on numerous components, any of which can malfunction. Failures stem from unreliable networks, transient server outages, overloaded load balancers, software bugs, even operator mistakes.

No matter how well you design, **it's virtually impossible to build a system that never fails**. Instead you design for resilience, ensuring a small hiccup doesn't snowball into a major outage.

Transient failures specifically

These are short-lived errors that resolve on their own given a bit of time. A brief network glitch, a momentary loss of connectivity, a temporary service unavailability. Often these issues are **ephemeral** — try again after a short delay and it succeeds.

The classic example: a database connection failure caused by a spike in concurrent users. At the peak moment the connection is refused. **A few seconds later, after some connections close, a retry goes straight through.** Nothing was fixed; the condition passed.

Other common cases: network timeouts, transient HTTP 5xx errors, temporary resource exhaustion.

Why this needs solving

Without a strategy for such hiccups, microservices **propagate errors to end-users or upstream systems unnecessarily**. A tiny blip in connectivity could trigger a user-facing error — or require manual intervention — even though **the system might have recovered milliseconds later**.

That's the frustration the pattern removes: the failure was real, but it was over before the user finished reading the error message.

Part 2 — The solution

If an operation fails due to a temporary issue, **wait a short period and try again**, on the assumption the issue may have resolved in the interim.

Many distributed systems fail in **partial** ways — a subset of requests fail while others succeed — or suffer short-lived outages. Often, trying the same request again simply causes it to succeed.

A good illustration: if a service didn't respond because of a momentary spike, a retry a few moments later **might hit a less-busy instance** and succeed. With multiple instances behind a load balancer, the retry isn't even necessarily going to the same place that failed.

The effect: retries **mask sporadic failures**, increasing the reliability the user perceives.

Why it works

Retries assume the failure was transient or non-deterministic. Network blips clear up, threads get freed, services restart — so a subsequent attempt has a good chance of succeeding where the first failed. This dramatically reduces the error rate seen by higher-level services and end users. Instead of giving up immediately when a downstream call fails, a service can retry and often complete the operation **without any manual fix**.

Part 3 — Transient vs. permanent: the critical distinction

Retry should only engage for errors **likely to be temporary**.

Worth retrying: network timeouts · HTTP 5xx server errors · 502 Bad Gateway from a load balancer · database deadlocks · connection refused · throttling / rate-limit errors

Not worth retrying: HTTP 400 (input validation failure) · HTTP 401 (authentication error) · business logic errors · configuration errors · fatal exceptions

> **No amount of retrying will fix a bad request or invalid credentials.** Those errors are permanent *for that request*. Retrying them just repeats the failure and potentially worsens system load.

Therefore a well-designed retry mechanism **checks the error type or exception type**, retrying only transient conditions while letting permanent errors fail fast.

Retry vs. fail-fast

An alternative strategy is **fail-fast**, typically managed via a Circuit Breaker.

	Retry	Fail-fast (Circuit Breaker)
Behaviour	Keeps trying, hoping for eventual success	Stops trying after detecting a pattern of failures
Favours	Eventual success	Conserving resources
Cost	Extra wait time and work	Gives up on things that might have worked
Right for	Transient faults	Long-lasting or permanent faults

For transient faults, retries are ideal — there's a high chance the next attempt succeeds. For a service that will be down for hours, **blindly retrying wastes time and resources**, and circuit breakers or graceful degradation (serving defaults or cached data) are more appropriate.

Use retries for hiccups; fall back or fail fast for lasting errors. They're complements, not competitors.

Part 4 — Architecture

The pattern is implemented as a **wrapper around an operation** that intercepts failures and repeats according to a policy.

Retry policy and parameters

Defines **when and how** to retry:

- The **maximum number of attempts**
- **Which exceptions or error codes** are considered retryable
- The **delay strategy** between attempts

Policies range from simple — *"retry up to 3 times with 2 seconds between tries"* — to sophisticated — *"retry 5 times with exponentially increasing delays, add randomness, and give up on certain error types."*

Retry loop (the invoker)

The mechanism that executes the operation and applies the policy. It calls the operation, catches any failure, checks whether a retry should happen per the policy, waits the specified delay, then

calls again. This continues until either the operation succeeds or max retries is reached. If it never succeeds, the invoker **propagates the final failure**, often after logging it as a persistent error.

Backoff strategy

To avoid hammering the failing resource, incorporate a wait between attempts rather than retrying in a tight loop.

Strategy	Behaviour	When
Constant delay	Always wait the same, e.g. 5s	Simple cases; predictable recovery time
Exponential backoff	Each wait longer than the last: 1s, 2s, 4s	The general default
Capped exponential backoff	As above with a maximum	Prevents absurdly long waits

Exponential backoff **mitigates load on struggling services** by spacing attempts out, and increases the likelihood that by the time the next attempt runs, the transient issue has resolved.

The cap matters: without it, five doublings from one second gives you a 32-second wait, and eight gives you over four minutes. Implementations typically cap the maximum delay.

Jitter (randomization)

If many clients or threads experience an error at around the same time, **they might all retry in unison after the same backoff interval**. This produces the thundering herd — a burst of synchronized retries hitting the service simultaneously, potentially causing **a second failure spike**.

Jitter introduces **random variation in the delay**. Instead of every client waiting exactly 4 seconds, each waits a random time around 4 seconds — somewhere between 2 and 4, or with a random offset added. This **spreads out the bursts** so load on the downstream service stays more even.

There are several implementations — **full jitter**, **equal jitter**, **decorrelated jitter** — but the core idea is identical: introduce randomness so retries are **de-synchronized across clients**.

> It seems counter-intuitive to add randomness to a system deliberately, but it has been shown to significantly reduce contention and improve recovery times. This is one of those cases where the unintuitive answer is simply correct.

Circuit breaker integration

Retries deal with transient failures; circuit breakers handle persistent ones by cutting off calls to an unresponsive service after a threshold. In practice a breaker **prevents a retry loop from endlessly hitting a downed service**.

For example, if a service has been failing for the last 50 calls, the breaker opens and further calls fail immediately without even trying — giving the service time to recover. After a cooldown, a half-open state allows a test call, and success closes the circuit again.

The balance: allow 3 quick retries on a request, but if multiple requests in a row all fail — signalling a likely bigger problem — open the circuit to stop further traffic for a short period.

Idempotency and side effects

A critical design consideration. If an operation has side effects — **charging a credit card, sending an email, incrementing a counter** — retrying could inadvertently perform the action **multiple times** unless you design against it.

Ideally, retried operations should be **idempotent**. Many APIs already are: deleting a resource, or performing a `GET`. For non-idempotent actions — creating an order, making a payment — you must take special care.

The technique: use **unique transaction identifiers or tokens** (idempotency keys) so the server can recognise a duplicate attempt and not process the action twice.

> The failure mode to picture: your payment call succeeds on the server, but the response is lost on the way back. Your client sees a timeout — indistinguishable from "it never happened" — and retries. **Without an idempotency key, the customer is charged twice.** The client genuinely cannot tell the difference between "lost request" and "lost response," which is why the server must be the one to deduplicate.

Placement in the architecture

Where should retries happen?

Client-side of a service-to-service call — the caller handles retrying. Service A calls Service B; A's client library or HTTP client implements the retry policy. B remains stateless and unaware of retries; it just sees repeated incoming requests.

API gateway or service mesh — infrastructure automatically retries failed requests between services.

⚠️ **The key rule: avoid duplicating retries at multiple layers.**

> If every hop in a chain of microservices did its own retries, **a single failure could multiply into a storm of requests.** Three layers each retrying three times means one original request becomes twenty-seven. Good practice is to **centralize or delegate retries to one layer** — only at the edge service, or only at the immediate caller — to prevent cascading retry explosions. In practice this means disabling internal retries in lower-level clients and handling it once at a higher layer, or using a coordinated approach like a **token bucket** to limit overall retry rates.

Part 5 — Worked example (Java)

```java
public class RetryUtil {

    public static <T> T executeWithRetry(Callable<T> task, int maxAttempts,
            long baseDelayMillis, double backoffFactor, boolean useJitter) throws Exception {

        int attempt = 1;
        Exception lastException = null;
        Random random = new Random();

        while (attempt <= maxAttempts) {
            try {
                return task.call();                 // success — return immediately
            } catch (Exception e) {
                lastException = e;                  // remember it for the final throw

                if (attempt == maxAttempts) {
                    break;                          // no retries left
                }

                // Work out how long to wait
                long delay = baseDelayMillis;
                if (backoffFactor > 1.0) {
                    // exponential: base * factor^(attempt-1)
                    delay = (long) (baseDelayMillis * Math.pow(backoffFactor, attempt - 1));
                }
                if (useJitter) {
                    // "full jitter": random value between 0 and the computed delay
                    delay = (long) (random.nextDouble() * delay);
                }
                // (a maximum cap could be applied here)

                try {
                    Thread.sleep(delay);
                } catch (InterruptedException ie) {
                    Thread.currentThread().interrupt();   // restore the interrupt flag
                    break;                                // stop retrying
                }

                attempt++;
            }
        }
        throw lastException;    // all attempts exhausted
    }
}
```

The parameters

Parameter	Meaning
`maxAttempts`	Total tries **including the initial attempt**. `3` means 1 try + 2 retries
`baseDelayMillis`	Fixed delay if constant; the initial delay if exponential
`backoffFactor`	The multiplier. `2.0` with base 100ms gives 100ms, 200ms, 400ms. Set `1.0` for a constant interval
`useJitter`	Applies full jitter — a random value between 0 and the computed delay

Reading the logic

`Callable<T>` makes it reusable. `Callable` is Java's interface for "an operation that returns a value and may throw." Accepting one means this utility wraps *any* operation — a network call, a database query — rather than being tied to one.

Success returns immediately. `return task.call()` exits the loop on the first success. No further attempts, no delay.

The exception is saved, not swallowed. `lastException = e` keeps the most recent failure so it can be thrown to the caller at the end. The caller learns *why* it ultimately failed.

The attempt check comes before the delay. If this was the last attempt, `break` immediately — there's no point sleeping before giving up.

`Math.pow(backoffFactor, attempt - 1)` — on attempt 1 that's `factor^0 = 1`, so you wait `baseDelay`. On attempt 2 it's `factor^1`, so double. And so on.

The `InterruptedException` handling is correct and worth copying. When a thread is interrupted, catching the exception clears the interrupt flag. `Thread.currentThread().interrupt()` restores it so code higher up still knows the thread was asked to stop. Forgetting this is a classic Java bug that makes applications refuse to shut down.

Using it

```
String result = RetryUtil.executeWithRetry(
    () -> fetchData(url),
    3,        // up to 3 attempts
    1000,     // 1 second base delay
    2.0,      // exponential backoff, doubling
    true      // use jitter
);
```

This attempts the fetch and, on failure, retries up to 2 more times — waiting about 1s before the first retry and about 2s before the second, with randomness in those waits. If all attempts fail it throws after roughly 3 seconds of total waiting.

In many cases that one retry is all that's needed to ride out a transient glitch and succeed on the second try.

Design choices, and one to change

The course is explicit about these:

- **Generic `Callable<T>`** keeps the logic reusable across any operation type
- **Configurable backoff and jitter** allows switching strategies: constant with no jitter (`1.0`, `false`), exponential without jitter (`2.0`, `false`), or exponential with jitter (`2.0`, `true`). This flexibility is crucial for tuning to different scenarios
- **`maxAttempts` prevents infinite loops** — vital, since an unbounded retry loop could endlessly hammer a failing service

And one deliberate simplification you should not carry into production:

- **It catches broad `Exception`**, treating *any* failure as retryable. In a real scenario you should catch **only specific transient exceptions** — `IOException`, timeouts — and let others bubble up without retry. **Your error-filtering logic belongs here**, and without it you'll cheerfully retry validation errors three times.

A refinement on jitter: full jitter (`random × delay`) can produce a near-zero wait, which partially defeats the purpose of backing off. Many production implementations randomise between **50% and 100%** of the delay instead, keeping a meaningful minimum gap.

Worth adding: logging each retry attempt, metrics, and a circuit-breaker check that aborts retrying if a higher-level circuit is already open.

Part 6 — Performance implications and risks

Increased load on dependencies

Every retry is an **additional request**. When failures are genuinely rare this extra load is negligible and well worth the improved reliability.

But if a downstream service is failing *because it's overloaded*, retries make the problem worse. If Service A is hammering a slow Service B and starts retrying aggressively, B gets even more requests. In the worst case, many clients retrying a struggling service create a **retry storm that amplifies load exponentially**, potentially overwhelming it completely.

This is precisely why **backoff is crucial** (breathing room between attempts) and **limited attempts** matter — and why circuit breakers or throttling should be layered on for protection.

Increased latency for end users

Retrying lengthens overall response time. A single failed attempt might add a few seconds of delay.

Sometimes that's acceptable — **a slightly slower successful response beats an immediate error**. But overdone it degrades experience, and chained retries across a call chain accumulate.

The countermeasure: **set an overall timeout for the operation** alongside retries. For example decide that an API call should either succeed **within 5 seconds including all retries**, or not at all, to maintain a responsive SLA. Without this, retry policies silently blow past your latency budget.

Wasted work and resource consumption

Every retry that ultimately fails was wasted effort — CPU, bandwidth, memory that produced no result. If the underlying issue isn't transient, **all retries fail and all that work is for nothing**, plus it strains the system.

Hence: detect non-transient failures quickly. If a response says "permission denied" or "invalid request," a retry won't help. Your retry logic should inspect error codes and keep a **blacklist of non-retryable errors**.

A subtler point worth remembering: **if the first attempt already took a long time** — nearly hitting the client timeout — starting another attempt that will likely also time out is counterproductive. Better to propagate an error than tie up threads on hopeless retries.

Cascading effects in complex systems

Service A calls B, B calls C. If **both** A and B have their own retry policies, a failure in C causes B to issue multiple attempts, **and** A to issue multiple attempts of its call to B. This multiplies load on both B and C.

Avoid uncoordinated retries at every layer. Decide **which layer retries** and let others fail fast. A common practice: the edge or client-facing service handles retries, while intermediate services propagate errors. Monitoring should detect when excessive retries occur, since it often indicates a downstream incident.

Retry storms and thundering herd

If a popular service glitches, many instances of many services may all start retrying at once. Backoff and jitter avoid synchronisation — **jitter specifically tackles the thundering herd** by de-synchronising retry timing.

Other mitigations: **retry budgets** (allowing a certain number of retries per second), and **adaptive algorithms** that lengthen backoff while errors continue, avoiding constant pressure.

Idempotency revisited

Beyond correctness, duplicate side effects have a performance cost too — sending the same email twice uses twice the email-sending resource. **Do not retry non-idempotent operations blindly** unless you have deduplication keys or check-pointing to guard against repeated side effects.

The mitigation checklist

- Implement **exponential backoff** — don't hammer rapidly
- Add **jitter** — avoid synchronised retries
- **Limit attempts**, and limit total time spent retrying
- Use **circuit breakers** or fail-fast switches where failure is likely persistent
- **Distinguish error types** — retry only transient ones
- **Monitor and tune** — frequent retries signal a struggling dependency or a bad policy
- **Ensure idempotency**, or implement server-side request deduplication for critical actions
- **Test under failure conditions** — simulate a dependency going down or exhibiting high latency, and verify your retry logic actually improves resilience rather than overwhelming the system

---

Part 7 — Use cases

Web services

The internet is unreliable, and transient faults are constant. When interacting with a remote web service and hitting a transient fault, retrying is usually beneficial because the **cost asymmetry** favours it: the cost of an unsuccessful request is high (an error message to the user, or an expensive fallback process), while the cost of a retry is typically just the time and resources to send another request.

Database operations

Many database operations are inherently uncertain — network issues, concurrency conflicts, temporary unavailability.

The clearest example: you try to save a record while **another user is updating the same record**. Rather than throwing an error, retry after a short delay. This allows the conflicting transaction to complete and **paves the way for your operation to succeed**.

Microservice architectures

When one service depends on another, the failure of the latter can bring down the former. Retrying gives the failing service **a chance to recover** and makes your service more resilient — ensuring a temporary hiccup doesn't become a full-blown outage.

Part 8 — Real-world examples

Cloud service SDKs (AWS, Google Cloud). Cloud providers build retry strategies into their client libraries. AWS SDKs automatically retry failed requests — a transient 5xx from a service, or a throttling error — using **exponential backoff and jitter**. This matters because network calls to cloud APIs occasionally time out or hit rate limits. Google Cloud's libraries have built-in retries for idempotent requests; a storage upload that times out is retried behind the scenes. These SDKs are

battle-tested to balance retries and backoff, so **developers benefit from the pattern without even realising it**.

Netflix. As part of their overall resilience engineering, Netflix's internal libraries retry calls to microservices or data stores if a request fails or a host is unresponsive. If one instance fails to respond, the client quickly **retries on another instance** — perhaps after a brief backoff — instead of erroring out immediately. Combined with circuit breakers, this ensures sporadic failures don't disrupt the viewing experience.

Payment processing (Stripe, PayPal). Financial APIs frequently encounter transient failures communicating with banking networks or third-party systems. Rather than immediately failing a transaction and upsetting a customer, payment systems retry behind the scenes.

Critically, they **encourage the use of idempotency keys** on API requests. If a network issue occurs, the client can safely retry with the same key, and the server **knows not to duplicate the charge**. This design **explicitly embraces the Retry Pattern — it assumes clients will retry on failure.** The result: a temporary glitch between merchant and gateway doesn't lose the order; the merchant's system retries automatically and payment goes through **without double-charging**.

Message queues and event processing. In systems built on RabbitMQ or Kafka, retry appears as **requeueing messages** or using **dead-letter queues**. If a consumer fails to process a message due to a temporary error — say it can't reach a database — it doesn't acknowledge the message, so the message returns to the queue to be tried again after a delay. This is the Retry Pattern in an **asynchronous** context.

A real example: in order processing, if an attempt to reserve inventory fails due to a **lock timeout**, the message can be retried a minute later, by which time the lock may be free. Many large e-commerce platforms rely on this in their event-driven architectures to ensure eventually consistent outcomes.

Part 9 — System design examples

E-commerce checkout. The user has filled their cart and clicks Buy. The system must charge their credit card. But the **payment gateway is temporarily unavailable**. Do you tell the user their purchase failed? Or retry, hoping the gateway returns shortly?

With the Retry Pattern you choose the latter, handling the temporary glitch gracefully. *(And this is precisely where idempotency keys are non-negotiable — the double-charge scenario above.)*

IoT device data collection. A system collects data from IoT devices and stores it in a database. Devices send data periodically. But the **database is temporarily unavailable** due to a network glitch or scheduled maintenance. Should the system discard the data?

Retry provides a resilient solution: handle the temporary issue and **ensure no data is lost**.

Part 10 — When NOT to use retry

- The error is clearly **non-transient** — a configuration error or fatal exception. Retrying just delays the inevitable
- The downstream service is **known to be down for an extended period** (a planned outage). A retry loop only burns resources; a circuit breaker or fallback response is preferable
- **Strict latency requirements** — real-time systems may prefer to fail fast rather than retry and violate their latency SLA
- The operation is **extremely expensive**, or has **side effects that can't be repeated** safely

In such cases, prefer alternatives: **manual compensation**, **eventual reconciliation** for asynchronous processes, or simply **alerting a human**.

> **Use retries where they make sense — transient, recoverable errors — and avoid them where they don't: permanent failures, or scenarios where retries could cause harm.**

Part 11 — Common beginner misconceptions

"More retries means more reliability." Only up to a point. Beyond a few attempts you're mostly adding latency and load. If three attempts with backoff didn't work, the fourth probably won't either.

"Retry and circuit breaker are alternatives." They're complements doing opposite things. Retry handles the blip; the breaker handles the outage. Use both.

"Jitter is a micro-optimisation." It's the difference between a graceful recovery and a self-inflicted outage at scale. At ten thousand clients it's essential, not decorative.

"My operation is a GET, so idempotency doesn't matter." True for that call — but check the whole path. A GET that triggers a server-side counter increment or audit write is not as idempotent as it looks.

"Retries are free if they usually succeed." They're free when failures are rare. They're most expensive exactly when the system is already struggling — which is when they fire most.

"I'll retry at every layer to be safe." This is the most damaging mistake in the chapter. Layered retries multiply, turning one failure into an exponential request storm.

Part 12 — Check your understanding

1. Why is retrying an HTTP 401 pointless?
2. Explain the thundering herd, and how jitter fixes it.
3. Why can't a client distinguish "lost request" from "lost response," and what follows from that?
4. Why must you set an overall operation timeout alongside a retry policy?
5. If A retries B and B retries C, how many requests can one failure become at 3 attempts each?
6. What's the one line in the example code you should change before production use?

(Answers: 1 — authentication failure is permanent for that request; no number of attempts fixes wrong credentials; 2 — many clients fail simultaneously, back off by the same amount, and retry in unison, re-overloading the service; jitter randomises delays so retries spread out; 3 — both look like a timeout, so the server must deduplicate using an idempotency key rather than the client guessing; 4 — otherwise cumulative retry delays silently exceed your latency budget; 5 — nine, since each of A's 3 attempts triggers 3 of B's; 6 — catching broad `Exception`, which retries permanent errors too.)

Part 13 — Interview soundbite (30 seconds)

> "Retry handles transient failures — network blips, timeouts, throttling, a connection refused during a spike — things that fix themselves in seconds. You wrap the call in a policy: max attempts, which errors are retryable, and a delay strategy. Two details matter most. First,

> exponential backoff, so you're not hammering a service that may be failing precisely because it's overloaded. Second, jitter — random variation in the delay — because otherwise every client that failed at the same moment retries at the same moment, and you get a thundering herd that causes a second outage. You also need idempotency: the client can't distinguish a lost request from a lost response, so retrying a payment without an idempotency key double-charges the customer. That's why Stripe's API is built around them — it explicitly assumes clients will retry. And you have to distinguish transient from permanent, because retrying a 400 or a 401 is pure waste. The biggest architectural mistake is retrying at every layer: if A retries B and B retries C, one failure becomes nine requests, so you pick one layer to retry and everything else fails fast. Retry pairs with circuit breakers — retry for hiccups, trip the breaker for real outages — and you set an overall operation timeout so the retries can't blow your latency budget."

Key takeaways: retry transient only · backoff plus jitter · limit attempts · make it idempotent · retry at one layer · test under failure.

Part 14 — Implementation

A. Runnable demo — backoff, jitter, and the thundering herd

Full source: `demos/chapters/08-retry/RetryDemo.java`

```
cd demos && ./gradlew runCh08
```

```java
import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicInteger;

public class RetryDemo {

    // ---------- Which errors are worth retrying ----------
    static class TransientException extends RuntimeException {
        TransientException(String m) { super(m); }
    }
    static class PermanentException extends RuntimeException {
        PermanentException(String m) { super(m); }
    }

    // ---------- The retry utility ----------
    static <T> T executeWithRetry(Callable<T> task, int maxAttempts,
                                  long baseDelayMs, double factor,
                                  boolean useJitter) throws Exception {
        Random random = new Random();
        Exception last = null;

        for (int attempt = 1; attempt <= maxAttempts; attempt++) {
            try {
                T result = task.call();
                if (attempt > 1) System.out.println("      succeeded on attempt " + attempt);
                return result;
            } catch (PermanentException e) {
                System.out.println("      permanent error, NOT retrying: " + e.getMessage());
                throw e;                                  // fail fast
            } catch (Exception e) {
                last = e;
                if (attempt == maxAttempts) break;

                long delay = (long) (baseDelayMs * Math.pow(factor, attempt - 1));
                delay = Math.min(delay, 5000);            // cap
                if (useJitter) {
                    // "equal jitter": 50-100% of the computed delay
                    delay = delay / 2 + (long) (random.nextDouble() * delay / 2);
                }
                System.out.printf("      attempt %d failed (%s), waiting %dms%n",
                        attempt, e.getMessage(), delay);
                Thread.sleep(delay);
            }
        }
        throw last;
    }

    // ---------- A service that fails N times then works ----------
    static class FlakyService {
        private final AtomicInteger calls = new AtomicInteger();
        private final int failuresBeforeSuccess;

        FlakyService(int n) { this.failuresBeforeSuccess = n; }

        String call() {
            int n = calls.incrementAndGet();
            if (n <= failuresBeforeSuccess) throw new TransientException("timeout");
            return "OK after " + n + " calls";
        }
    }

    public static void main(String[] args) throws Exception {
```

```
        System.out.println("\n=== 1. Transient failure, recovers on 3rd attempt ===");
        FlakyService flaky = new FlakyService(2);
        System.out.println("      result: " +
                executeWithRetry(flaky::call, 5, 100, 2.0, false));

        System.out.println("\n=== 2. Permanent failure — should NOT retry ===");
        try {
            executeWithRetry(() -> { throw new PermanentException("HTTP 401"); },
                    5, 100, 2.0, false);
        } catch (PermanentException e) {
            System.out.println("      gave up immediately (correct)");
        }

        System.out.println("\n=== 3. Backoff schedule, no jitter ===");
        try {
            executeWithRetry(() -> { throw new TransientException("503"); },
                    5, 100, 2.0, false);
        } catch (Exception ignored) { }

        System.out.println("\n=== 4. Same schedule WITH jitter ===");
        try {
            executeWithRetry(() -> { throw new TransientException("503"); },
                    5, 100, 2.0, true);
        } catch (Exception ignored) { }

        System.out.println("\n=== 5. Thundering herd: 1000 clients retry at once ===");
        demonstrateHerd(false);
        demonstrateHerd(true);
    }

    // Show how many clients land in the same 100ms bucket
    static void demonstrateHerd(boolean useJitter) {
        Random random = new Random();
        Map<Long, Integer> buckets = new TreeMap<>();
        long baseDelay = 1000;

        for (int i = 0; i < 1000; i++) {
            long delay = baseDelay;
            if (useJitter) {
                delay = delay / 2 + (long) (random.nextDouble() * delay / 2);
            }
            buckets.merge(delay / 100 * 100, 1, Integer::sum);
        }

        int worst = buckets.values().stream().mapToInt(Integer::intValue).max().orElse(0);
        System.out.printf("      jitter=%-5s -> busiest 100ms bucket holds %d of 1000 clients%n",
                useJitter, worst);
    }
}
```

B. What you should see

```
=== 1. Transient failure, recovers on 3rd attempt ===
      attempt 1 failed (timeout), waiting 100ms
      attempt 2 failed (timeout), waiting 200ms
      succeeded on attempt 3
      result: OK after 3 calls

=== 2. Permanent failure — should NOT retry ===
      permanent error, NOT retrying: HTTP 401
      gave up immediately (correct)

=== 3. Backoff schedule, no jitter ===
      attempt 1 failed (503), waiting 100ms
      attempt 2 failed (503), waiting 200ms
      attempt 3 failed (503), waiting 400ms
      attempt 4 failed (503), waiting 800ms      <- doubling

=== 4. Same schedule WITH jitter ===
      attempt 1 failed (503), waiting 74ms
      attempt 2 failed (503), waiting 133ms
      attempt 3 failed (503), waiting 291ms      <- randomised
      attempt 4 failed (503), waiting 512ms

=== 5. Thundering herd: 1000 clients retry at once ===
      jitter=false -> busiest 100ms bucket holds 1000 of 1000 clients
      jitter=true  -> busiest 100ms bucket holds ~205 of 1000 clients
```

Section 5 is the one to internalise. Without jitter, **all 1000 clients retry in the same 100ms window** — a synchronised wall of traffic hitting a service that just recovered. With jitter the worst bucket holds about a fifth of that. Same number of retries, spread over time instead of stacked.

C. Production — Resilience4j Retry

```yaml
resilience4j:
  retry:
    instances:
      inventoryService:
        maxAttempts: 3                   # 1 initial + 2 retries
        waitDuration: 200ms
        enableExponentialBackoff: true
        exponentialBackoffMultiplier: 2
        enableRandomizedWait: true       # <- jitter
        randomizedWaitFactor: 0.5
        retryExceptions:
          - java.io.IOException
          - java.util.concurrent.TimeoutException
          - org.springframework.web.client.HttpServerErrorException   # 5xx
        ignoreExceptions:
          - org.springframework.web.client.HttpClientErrorException   # 4xx — never retry
```

```java
@Retry(name = "inventoryService", fallbackMethod = "inventoryFallback")
@CircuitBreaker(name = "inventoryService")     // order matters — see pitfall 6
public Inventory check(String sku) {
    return inventoryClient.get(sku);
}
```

Spring's own `@Retryable` is an alternative if you're not using Resilience4j:

```java
@Retryable(
    retryFor = { IOException.class },
    noRetryFor = { ValidationException.class },
    maxAttempts = 3,
    backoff = @Backoff(delay = 200, multiplier = 2, random = true))
public Inventory check(String sku) { ... }

@Recover
public Inventory recover(IOException e, String sku) { return Inventory.unknown(sku); }
```

D. Implementing idempotency keys

The most important production concern, and the part usually missing from tutorials. The client sends a stable key; the server guarantees one execution per key.

Client:

```java
String idempotencyKey = UUID.randomUUID().toString();   // generated ONCE, before the first attempt

executeWithRetry(() ->
    http.post("/payments")
        .header("Idempotency-Key", idempotencyKey)      // same key on EVERY retry
        .body(payment)
        .execute(),
    3, 200, 2.0, true);
```

> **The critical detail:** generate the key **outside** the retry loop. A key generated inside means each attempt looks like a brand-new request and the customer gets charged three times — which is exactly the bug the mechanism exists to prevent.

Server:

```java
@PostMapping("/payments")
public ResponseEntity<PaymentResult> pay(
        @RequestHeader("Idempotency-Key") String key,
        @RequestBody Payment payment) {

    // Atomic "insert if absent" — the DB unique constraint does the work
    Optional<PaymentResult> existing = idempotencyStore.find(key);
    if (existing.isPresent()) {
        return ResponseEntity.ok(existing.get());     // replay the original response
    }

    PaymentResult result = paymentService.charge(payment);
    idempotencyStore.save(key, result, Duration.ofHours(24));
    return ResponseEntity.ok(result);
}
```

Store keys in Redis with a TTL, or a table with a unique index on the key. The unique constraint is what makes it safe under concurrent duplicate requests.

E. Implementation pitfalls

1. Retrying non-idempotent operations without a key. The headline risk: double charges, duplicate orders, duplicate emails.

2. Generating the idempotency key inside the retry loop. See above — silently defeats the whole mechanism.

3. Retrying 4xx errors. A 400 or 401 will fail identically every time. Always configure `ignoreExceptions`.

4. No cap on exponential backoff. Base 1s with multiplier 2 and 10 attempts ends at a **17-minute** wait. Always set a maximum.

5. No overall deadline. Individual timeouts plus retries multiply: a 5s timeout with 3 attempts plus backoff is potentially 20 seconds. Set a budget for the whole operation:

```
resilience4j.timelimiter.instances.inventoryService.timeoutDuration: 3s
```

6. Wrong ordering of retry and circuit breaker. Resilience4j's default order is **Retry(CircuitBreaker(call))** — the breaker is *inside*, so retries happen around it and the breaker sees each attempt. That's usually what you want. If you invert it, the breaker counts a whole exhausted retry sequence as a single failure and opens far too slowly.

7. Retrying at every layer. If the HTTP client retries 3×, the service retries 3×, and the gateway retries 3×, one user request becomes **27** requests to a service that's already struggling. Pick one layer. Disable retries everywhere else explicitly — most HTTP clients have them on by default.

8. Retrying immediately with no delay. A tight retry loop against a struggling service is indistinguishable from an attack.

9. Not logging retries. If retries are invisible, a dependency degrading from 0.1% to 15% failure looks totally healthy from the outside — until it tips over. Emit a metric per retry.

Related patterns

- **Circuit Breaker** — the essential companion; stops retries becoming an attack on a downed service
- **Bulkhead** — bounds the resources retries can consume
- **Service Discovery** — retry against a *different* discovered instance
- **Saga** — compensation for when retrying isn't safe or sufficient
- **Event-Driven Architecture** — requeueing and dead-letter queues are retry in an async context

Sidecar Pattern

Category: Cross-cutting concerns **One-liner:** Attach a helper process alongside each service instance to handle concerns like logging, metrics, security, and networking — so the service itself contains only business logic.

Part 0 — Concepts you need first

What is a "cross-cutting concern"?

Functionality needed by **many parts of a system** but not part of any one part's core purpose.

Your Order Service exists to manage orders. That's its business logic. But it also needs to: log what it does, emit metrics, authenticate callers, encrypt traffic, retry failed calls, and report traces. **Every service needs those**, and none of them is about orders.

Cross-cutting concerns are a problem because the obvious solutions are both bad:

- **Copy the code into every service** → duplication, and changes require touching everything
- **Put it in a shared library** → better, but see the polyglot problem below

What is a process?

A running program with its own memory space. Two processes are genuinely separate — one crashing doesn't corrupt the other's memory.

What is a container?

A package containing an application plus everything it needs to run — libraries, configuration, the lot. It runs isolated from other containers on the same machine. **Docker** is the common tool for building them.

What is a Kubernetes Pod?

The smallest deployable unit in Kubernetes. Crucially, **a Pod can contain more than one container**, and containers in the same Pod:

- Are always **scheduled onto the same machine** together
- Share a **network namespace** — they can talk to each other over `localhost`
- Can share **storage volumes**
- Are **created and destroyed together**

This is the mechanism that makes the Sidecar pattern practical. Without co-scheduling and shared networking, a "helper alongside the service" would just be another remote service.

What is a proxy?

Software that sits between two parties and relays traffic. A **transparent** proxy relays traffic without the application knowing it exists — which is exactly how production sidecars work.

What is a service mesh?

An infrastructure layer handling all service-to-service communication. Concretely, it's **a sidecar proxy deployed next to every service**, plus a central **control plane** that configures them all. **Istio** (using **Envoy** as the proxy) is the best-known example.

The polyglot problem — why shared libraries aren't enough

Suppose you write an excellent library handling retries, metrics, and mTLS. Now your company has services in Java, Go, Python, and Node.

You must write and maintain **four versions**. They will drift. A bug fixed in the Java one lingers in Python for months. Adding a fifth language means a fifth implementation before that team can ship.

A sidecar solves this because it's a separate process speaking network protocols. It doesn't care what language the service beside it is written in.

Part 1 — The metaphor and the core idea

The name comes from a **motorcycle sidecar**: attached to and operating alongside the motorcycle, sharing the journey but remaining a distinct unit. The course also offers Batman and Robin, or Sherlock Holmes and Dr Watson — an aide that handles specific tasks, operates autonomously, and yet is part of a single larger entity.

The pattern encapsulates **modularity and separation of concerns**: deploy components of an application as **separate processes running alongside the main application**. Each sidecar is dedicated to a specific function and provides its service to the main application.

This lets developers **encapsulate the logic and complexity of those functions in the sidecar**, reducing the complexity of the main application.

The strength is its simplicity and modularity. Keeping each function isolated in its own environment makes it easier to develop, manage, and scale each component independently. That gives cleaner, more maintainable code — and makes it possible to **use different technologies for different tasks within the same application**.

⚠️ Part 2 — One correction to the course material

The course states that the sidecar and main application "run in the same process" and are "deployed together in the same container." **That is not how this works in practice**, and since you're using these notes as your reference, it's worth having right:

- The sidecar runs as a **separate process, in its own container**
- Both containers are **co-located in the same Kubernetes Pod** (or equivalent scheduling unit)
- They **share the Pod's network namespace** — hence `localhost` communication — and can share volumes
- They **share a lifecycle** — created, scheduled, and destroyed together

Same-process or same-container would defeat the isolation that makes the pattern valuable in the first place: if the sidecar shared a process with the app, a sidecar crash would take the app down, and you couldn't use a different language for each.

The course's other framing — **same lifespan, shares resources, communicates locally** — is correct. Everything else below holds.

Part 3 — The problem: monolithic application management

Lack of isolation

In a monolith all components are closely intertwined, so **a failure in one component can bring down the entire application**. The course's image: all your eggs in one basket — if the basket falls, all

the eggs break. This lack of fault isolation makes monoliths vulnerable and risky.

Scalability — moving the mountain

Scaling a monolith is like moving a mountain. **You cannot scale a particular component or functionality; you must scale the entire application**, even if only one component needs it. This leads to unnecessary resource consumption and inefficient scaling.

Complexity — a tangled web

As a monolith grows, so does its complexity. More features mean the codebase becomes a tangled web that's increasingly difficult to understand, manage, or modify. Maintenance becomes a nightmare and development slows significantly.

Technology constraint — one size does not fit all

With a monolith you're largely **stuck with the technology stack chosen at the beginning**. Want to integrate a new technology or switch to a more efficient one? That's a mammoth task. This lack of flexibility is a significant constraint given how rapidly technology evolves.

Part 4 — The solution

Pair each service with a secondary component — a **sidecar** — that takes over some of the main service's responsibilities. The two run alongside each other and communicate directly, creating a self-contained module that's easy to manage and scale.

Flexibility. By separating the main service from its secondary functions, it becomes easier to update, modify, or replace individual components without affecting the others. To change a specific piece of functionality, you **update the relevant sidecar and leave the rest of the application untouched** — no full redeploy.

Enhanced scalability. Since each sidecar is a separate entity, you can scale each independently. Surge in user logins? Scale up the **authentication sidecar**. High load on data processing? Scale up the **data-processing sidecar**. This is more efficient and saves resources.

Fault isolation. Each sidecar is independent, so a failure in one does not affect the others. Separate baskets for each egg — if one falls, the rest are safe.

Reduced complexity. Breaking the application into self-contained modules, each with its own sidecar, makes it easier to understand, manage, and maintain. Like clearing a path through dense forest — you can finally see where you're going.

Technology freedom. The independent, flexible structure makes it possible to **use different technologies for different sidecars**, letting you take advantage of the latest advances rather than being locked in.

Part 5 — Architecture

Co-located deployment

The main application and its sidecar are deployed **together**, sharing resources and communicating efficiently. This shared environment lets them **operate as a single unit while maintaining their individuality**.

Multiple sidecars

You are not restricted to one sidecar per application. You can have **several, each addressing a specific functionality**, letting you modularise the application.

> A motorbike with several sidecars — one for extra storage, one for a passenger, one carrying navigation aids. Each serves a distinct purpose; together they make one smooth ride.

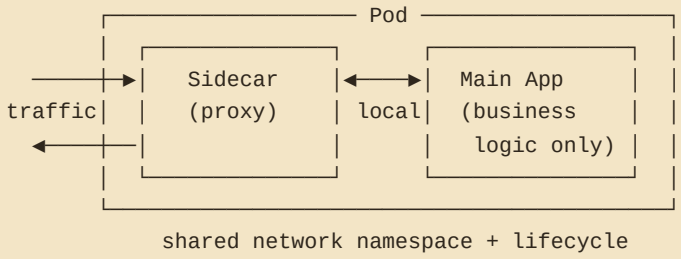
The network navigator — the sidecar as proxy

This is the most important architectural role, and the one that makes service meshes work.

The sidecar **intercepts all incoming and outgoing network traffic** from the main application, effectively acting as a **proxy**. This means it can handle:

- **Load balancing**
- **Circuit breaking** *(see Circuit Breaker)*
- **Security** — including mutual TLS
- **Service discovery** *(see Service Discovery)*

...leaving the main application **free to focus on delivering the primary service**.



> **Why "intercepts" is the key word.** The application doesn't call the sidecar and doesn't know it exists. It makes what it thinks is a normal HTTP call; traffic is transparently routed through the sidecar. That's what allows you to add mTLS, retries, and tracing to an application **without changing a line of its code** — even for an application whose source you can't easily modify.

Shared lifespan

The sidecar and main application, while separate entities, **share the same lifespan** — created, deployed, and destroyed together. This ensures the main application **always has its sidecar at its side**, ready to serve.

That guarantee matters. If the sidecar could be scheduled elsewhere or restarted independently, the application would need to handle its absence — reintroducing exactly the complexity you were removing.

Part 6 — Worked example

The main application — a Spring Boot REST service

```
@SpringBootApplication
@RestController
public class MainApplication {

    public static void main(String[] args) {
        SpringApplication.run(MainApplication.class, args);
    }

    @GetMapping("/api")
    public String getData() {
        return "Hello, Sidecar!";
    }
}
```

A simple web service returning a string whenever `/api` is hit.

The sidecar — a logging helper

```
@SpringBootApplication
@RestController
public class LoggingSidecar {

    public static void main(String[] args) {
        SpringApplication.run(LoggingSidecar.class, args);
    }

    @GetMapping("/log")
    public String logRequest() {
        System.out.println("Request received at /api endpoint");
        return "Logged!";
    }
}
```

This sidecar prints a log message every time `/log` is hit. Note it's also a Java application here — but that's incidental, and in practice it usually wouldn't be.

Wiring them together

```
@GetMapping("/api")
public String getData() {
    ResponseEntity<String> response =
        restTemplate.getForEntity("http://localhost:8081/log", String.class);
    System.out.println("Response from sidecar: " + response.getBody());
    return "Hello, Sidecar!";
}
```

`RestTemplate` is Spring's utility for calling RESTful services. Now whenever `/api` is hit, the main application calls the sidecar's `/log` endpoint, the sidecar logs the request, and the main application continues processing.

Notice `localhost:8081` — that's the shared network namespace at work. The main app doesn't need service discovery to find its sidecar; it's always right there at a known local port.

> ⚠️ **This example is pedagogically useful but architecturally backwards.** The main application *explicitly calling* the sidecar defeats the central benefit — the application now knows about the sidecar, contains code coupling it to the sidecar, and would break if the sidecar changed. Real sidecars are **transparent**, intercepting traffic without the application's knowledge (as Istio and Envoy do below). Read this as "how two co-located processes communicate," not as the intended usage pattern.

The deployment factor

The course is explicit that the Java example doesn't cover deployment. In production you would:

- Use a container platform like **Docker** to package the main application and sidecar
- Use **Kubernetes** to orchestrate deployment and scaling

Kubernetes has **built-in support for sidecar containers**, making it the natural platform for implementing this pattern.

Part 7 — Performance implications

"Does adding a sidecar to my application slow things down?" A legitimate question. It's a trade-off worth understanding rather than dismissing.

Resource cost

Each sidecar is an **additional process** running alongside your main application. That means more CPU, more memory, more network bandwidth.

Individually, the sidecar's resource usage may not be substantial. **But multiply that by the number of instances of your application.** With hundreds or thousands of instances each carrying its own sidecar, **cumulative resource usage becomes significant**.

> Made concrete: a sidecar using 100 MB of memory across 1,000 pods is 100 GB of RAM spent on infrastructure rather than application work. That's a real budget line, and it's the most common objection to adopting a service mesh.

Latency

Having a sidecar introduces an **additional network hop**, since requests may go through the sidecar before reaching the main application or an external service.

The latency introduced is **usually negligible** — it's a loopback connection within the same Pod, not a real network traversal. But in **high-performance, low-latency systems this could become an issue**, and it's worth measuring rather than assuming.

The verdict

The slight performance impact often **pales in comparison to the benefits**. The sidecar enhances functionality and maintainability and improves the overall development and operational experience. You need to balance these trade-offs for your situation.

Part 8 — Special considerations

1. Sidecar and main application versioning

Versioning becomes more complex. You must manage versions of **both** the main application and the sidecar. What if a new sidecar version is **incompatible with an older application version**, or vice versa? How do you roll out updates across a fleet? These questions need answering before you adopt the pattern, not after.

2. Shared resources

Sidecars share resources with the main application — **network and disk**. It's important the sidecar **doesn't hog these resources**, leaving nothing for the main application. Manage this using **resource limits and quotas at the container level**.

> Without limits, a misbehaving sidecar — say, one buffering logs during an outage of the log collector — can starve the very application it's meant to support.

3. Security

Sidecars often need **similar security permissions to the main application** to perform their tasks. This **increases the attack surface** and needs careful consideration when designing your sidecar.

A sidecar handling mTLS holds private keys. A sidecar collecting logs can read application output. Compromising the sidecar can be as valuable to an attacker as compromising the application.

4. Complexity

Sidecars solve many problems, but they **also add complexity**. Developers and operators must now understand and manage **multiple parts instead of a single application**. Debugging a request means reasoning about the application *and* the proxy in front of it.

> The pattern is not a one-size-fits-all solution. Like any architectural pattern it has strengths and weaknesses, and its effectiveness depends on your specific use case.

Part 9 — Use cases

Monitoring and metrics collection

You're running an application with multiple instances and want metrics from each. **Embedding monitoring code in each application would make things complex.** A sidecar handles metrics collection, leaving your application code clean and focused on core functionality.

Logging

A sidecar can **collect logs from the main application and forward them to a central logging system**. This abstracts away logging complexity, making the application easier to manage — and means changing your log destination doesn't require touching any application.

Security and authentication

Instead of embedding security and authentication logic into your main application, **delegate it to a sidecar**. This simplifies the application and makes security measures **more consistent and easier to update** — you patch one sidecar image rather than auditing dozens of services.

Networking features

Networking is complex. A sidecar can handle **service discovery, connection pooling, circuit-breaking, intelligent routing**, and more, letting your application focus on business logic without worrying about networking intricacies.

> The unifying test: **is this concern identical across all your services, and unrelated to what any of them actually does?** If yes, it's a sidecar candidate.

Part 10 — System design examples

Microservices architecture

Each microservice, as a main application, can have a sidecar providing ancillary services such as logging, monitoring, and service discovery.

> Picture a fleet of ships, each ship a microservice. The sidecars are little boats accompanying each ship, helping with navigation, communication, and maintenance — while the ships focus on their main task.

Kubernetes and Istio — the canonical example

Istio is a service mesh for Kubernetes, and it uses this pattern extensively. Istio deploys a **sidecar proxy — Envoy — alongside each service** to:

- Handle **all network communication**
- **Enforce policies**
- **Collect telemetry data**

> Picture a busy highway system where each car (service) has a dedicated assistant (the Envoy sidecar) for navigation, communication, and safety — enabling the driver to concentrate on driving.

This is the pattern at its purest. The application code contains **zero** networking, retry, mTLS, or observability logic; all of it lives in infrastructure, configured centrally and applied uniformly.

Part 11 — Sidecar vs. shared library

The real decision most teams face:

	Shared library	Sidecar
Languages	One per language, maintained separately	One, language-agnostic
Upgrade	Rebuild and redeploy every service	Update the sidecar image
Runtime cost	Negligible — same process	An extra process per instance
Latency	None	A local hop
Failure isolation	A library bug crashes the app	Sidecar can fail separately
Best when	One language, few services, latency-critical	Polyglot, many services, frequent policy changes

Rule of thumb: with a single language and a handful of services, a library is simpler and cheaper. With several languages, many services, or a need to change cross-cutting behaviour without redeploying everything, the sidecar earns its overhead.

Part 12 — Common beginner misconceptions

"The sidecar and app are the same container." No — separate containers in the same Pod. See Part 2.

"The app calls the sidecar." In the course's toy example, yes. In real deployments, no — the sidecar transparently intercepts traffic. That transparency is the whole value.

"A sidecar is a microservice." No. A microservice serves many clients and owns a domain. A sidecar serves exactly **one** application instance and owns no domain — its lifecycle is bound to its host.

"Sidecars are only for service meshes." Meshes are the most famous use, but log shippers, metrics agents, config reloaders, and secret-fetchers are all sidecars.

"It's free because it's just localhost." It costs one process worth of CPU and memory **per instance**, which at scale is the main reason teams hesitate.

"You need Kubernetes." Kubernetes makes it easy with Pods, but the pattern predates it — any system that can co-schedule two processes with shared networking works.

Part 13 — Check your understanding

1. What makes a concern a "cross-cutting" one?
2. Why doesn't a shared library solve cross-cutting concerns in a polyglot organisation?
3. What Kubernetes feature makes the pattern practical, and which two properties of it matter most?
4. Why is "intercepts" a more important word than "calls"?
5. What's the main objection to adopting a service mesh at scale?
6. When is a shared library the better choice?

(Answers: 1 — it's needed by many components but isn't the core purpose of any of them; 2 — you'd need a separate implementation per language, and they inevitably drift; 3 — the Pod, because it co-schedules containers on one machine and gives them a shared network namespace and lifecycle; 4 — interception means the application needs no knowledge or code changes, which is what lets you add mTLS and tracing to services you can't modify; 5 — cumulative resource cost, one extra process per instance across thousands of instances; 6 — a single language, few services, and latency-critical paths.)

Part 14 — Interview soundbite (30 seconds)

> "The sidecar pattern attaches a helper process alongside each service instance to handle cross-cutting concerns — logging, metrics, auth, networking — so the service itself contains only business logic. It's co-located: a separate container in the same Kubernetes Pod, sharing the network namespace and lifecycle, so they communicate over localhost. The key role is acting as a proxy that transparently intercepts all inbound and outbound traffic, which lets it do load balancing, circuit breaking, mTLS, and service discovery without the application knowing it exists. That transparency is the point — you can add mTLS and distributed tracing to a service without changing a line of its code. The canonical example is Istio, where an Envoy sidecar runs next to every service and the app has no networking code at all. The big advantage over a shared library is that it's language-agnostic, so a polyglot estate gets consistent behaviour and you can upgrade policy without redeploying every service. The costs are real though: one extra process per instance multiplies across thousands of pods, so a sidecar using 100 MB across a thousand pods is 100 GB of RAM; there's an extra local hop; you now version the sidecar against the app; and the sidecar needs permissions that widen your attack surface."

Part 15 — Implementation

A. Runnable demo

Full source: `demos/SidecarDemo.java`

```
cd demos && ./gradlew runCh09
```

The whole point in two class definitions. Before:

```
String handleRequest(String payload) {
    requestCount.incrementAndGet();                    // metrics
    System.out.println("[app] logging inbound request"); // logging
    if (!payload.startsWith("signed:")) return "401";    // security
    String result = businessLogic(payload);             // <- the only real work
    // ... emit latency metric
}
```

After:

```
static class CleanApp {
    String handleRequest(String payload) {
        return "order-created:" + payload;              // business logic ONLY
    }
}
```

Everything removed from the app now lives in the sidecar, which **intercepts** rather than being called:

```
String handleInbound(String rawRequest, String clientCert) {
    if (!"valid-cert".equals(clientCert)) {
        return "403 Forbidden";                          // app never invoked
    }
    String response = app.handleRequest(rawRequest);      // plain call, no plumbing
    // ... record latency
}
```

B. What you should see

```
=== 3. mTLS enforced by the sidecar ===
  -> 403 Forbidden
  >> rejected before the application was ever invoked

=== 4. Retry + backoff, also in the sidecar ===
    [sidecar] upstream failed (connection reset), retrying
    [sidecar] upstream ok on attempt 3
  upstream was actually called 3 times; the app asked once
  >> CleanApp contains no retry code whatsoever

=== 5. Telemetry collected without any app instrumentation ===
      requests_ok        1
      requests_rejected  1
      upstream_retries   2

=== 7. The cost: resource multiplier ===
      1,000 pods x 100MB sidecar = 100,000 MB (97.7 GB) of RAM
      5,000 pods x 100MB sidecar = 500,000 MB (488.3 GB) of RAM
```

Section 4 is the pattern in one line: the upstream was called three times; the application asked once. Retry, backoff, and failure handling happened entirely outside the code you wrote.

Section 7 is the honest counterweight — at a thousand pods you're spending ~98 GB of RAM on infrastructure. That's the number that shows up in the meeting where someone asks whether the mesh is worth it.

C. Production — Istio

Injection — label the namespace and every pod gets an Envoy sidecar automatically:

```
kubectl label namespace production istio-injection=enabled
```

Your Deployment is unchanged; the sidecar appears at admission time:

```
spec:
  containers:
    - name: order-service          # your app - you wrote this
      image: order-service:1.4.2
    # - name: istio-proxy          # injected automatically, you did NOT write this
```

mTLS across the whole mesh — no application code, no certificate handling:

```yaml
apiVersion: security.istio.io/v1beta1
kind: PeerAuthentication
metadata:
  name: default
  namespace: production
spec:
  mtls:
    mode: STRICT       # every service-to-service call is encrypted and authenticated
```

Retries, timeouts, and circuit breaking as configuration:

```yaml
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: inventory
spec:
  hosts: [inventory]
  http:
    - timeout: 3s
      retries:
        attempts: 3
        perTryTimeout: 1s
        retryOn: 5xx,reset,connect-failure
      route:
        - destination: { host: inventory }
---
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata:
  name: inventory
spec:
  host: inventory
  trafficPolicy:
    connectionPool:
      tcp:  { maxConnections: 100 }
      http: { http1MaxPendingRequests: 50, maxRequestsPerConnection: 10 }   # a bulkhead
    outlierDetection:                       # this IS a circuit breaker
      consecutive5xxErrors: 5
      interval: 10s
      baseEjectionTime: 30s
      maxEjectionPercent: 50
```

> Read that last block carefully: **you just configured Bulkhead, Retry, and Circuit Breaker in YAML, for every service, in every language, without touching application code.** That is the argument for a service mesh in a single manifest.

Non-mesh sidecars you'll meet just as often:

```
containers:
  - name: app
    image: my-app:1.0
    volumeMounts: [{ name: logs, mountPath: /var/log/app }]
  - name: log-shipper                    # sidecar: tails and ships logs
    image: fluent-bit:2.2
    volumeMounts: [{ name: logs, mountPath: /var/log/app }]
  - name: cloud-sql-proxy                # sidecar: secure DB tunnel
    image: gcr.io/cloud-sql-connectors/cloud-sql-proxy:2.8
volumes:
  - name: logs
    emptyDir: {}                         # shared between the two containers
```

Kubernetes native sidecars (1.29+) — solves the classic startup/shutdown ordering problem:

```
initContainers:
  - name: istio-proxy
    image: istio/proxyv2
    restartPolicy: Always        # <- makes it a true sidecar: starts first, stops last
```

D. Implementation pitfalls

1. The app starting before the sidecar is ready. The classic failure: your app makes its first outbound call, the proxy isn't up, and it fails at startup. Use native sidecars (`restartPolicy: Always` on an initContainer), or `holdApplicationUntilProxyStarts: true` in Istio.

2. The sidecar stopping before the app finishes. The mirror image at shutdown — in-flight requests fail because the proxy died first. Native sidecars fix this too.

3. No resource limits on the sidecar. An unbounded proxy can starve the application it exists to serve. Always set them:

```
annotations:
  sidecar.istio.io/proxyCPU: "100m"
  sidecar.istio.io/proxyMemory: "128Mi"
```

4. Adopting a full mesh for three services. The resource multiplier and operational complexity are fixed costs. Below roughly a dozen services in one or two languages, a shared library is cheaper and simpler. Be able to justify the mesh.

5. Forgetting the sidecar is in the network path when debugging. A 503 may come from Envoy, not your service. Learn to read the proxy's logs and its response flags before blaming application code.

6. Duplicating retries in app and mesh. If your HTTP client retries 3× and the mesh retries 3×, you get 9 requests. Pick one layer — usually the mesh — and disable the other.

7. Assuming mTLS means you're done with authorization. mTLS proves *who* is calling. It does not say *what they may do*. You still need `AuthorizationPolicy` resources.

Related patterns

- **Service Discovery** — meshes absorb discovery into the sidecar entirely
- **Circuit Breaker**, **Retry**, **Bulkhead** — all commonly enforced in the sidecar rather than in application code
- **API Gateway** — the gateway handles north-south (external) traffic; the mesh of sidecars handles east-west (internal)

- **Config Externalization** — a sidecar can fetch config and expose it locally to the app

Saga Pattern

Category: Data consistency / distributed transactions **One-liner:** Break a business transaction that spans several services into a sequence of local transactions, each with a compensating action that undoes it — achieving eventual consistency without distributed locks.

Part 0 — Concepts you need first

What is a transaction?

A **transaction** is a single logical operation made up of one or more data-manipulation steps, which **succeed or fail together as a unit**.

The classic example is a bank transfer: subtract £100 from account A, add £100 to account B. If only the first step runs, £100 has vanished. The two steps must be **one indivisible unit**.

What is ACID?

The four properties a traditional database transaction guarantees.

Atomicity — the transaction is treated as a single unit that either **succeeds completely or fails completely**. If any part fails, the whole thing fails and the database is left unchanged. *"Atomic" means indivisible — you can't have half of it.*

Consistency — the transaction brings the database from one **valid state to another**, respecting all its rules and constraints.

Isolation — concurrent transactions leave the database in the same state as if they had run **one after another**. Your half-finished transaction is invisible to everyone else.

Durability — once committed, it **stays committed**, even if the machine loses power a millisecond later.

Together: once a transaction completes, you can be confident it **either fully occurred or didn't occur at all**.

What is a "rollback"?

Undoing a transaction's changes, returning the database to its state before the transaction began. In a single database this is automatic and free — the database keeps a log and reverses it.

This is the capability you lose when the transaction spans services, and replacing it manually is what the Saga pattern is about.

What is a database lock?

While a transaction modifies a row, the database **locks** it so nobody else can modify it simultaneously. That's what makes isolation work.

The relevant consequence: **locks are held until the transaction commits.** A short transaction holds a lock for milliseconds. A transaction waiting on a network call to another continent holds it for seconds — during which nobody else can touch that row.

What is "database per service"?

A core microservices principle: **each service owns its own database**, and no other service reads or writes it directly. If Order Service needs inventory data, it *asks* Inventory Service; it does not query Inventory's tables.

This gives independence — Inventory can change its schema freely — but it's precisely what removes the option of one database transaction spanning both.

What is eventual consistency?

A model where the system **may temporarily be in an inconsistent state**, but will **eventually converge** to a consistent one.

Strong consistency: the moment your transfer completes, every observer sees both accounts updated. Eventual consistency: for a brief window, one observer might see the debit but not yet the credit — though it will resolve shortly.

Distributed systems trade strong for eventual consistency because strong consistency across a network requires locks and coordination that cost availability and speed.

Part 1 — The problem

It works fine in a monolith

An e-commerce checkout: adjust inventory, create the order, take payment. In a monolithic application with a single database, this is one ACID transaction. If payment fails, the database rolls back inventory and the order automatically. **You get atomicity for free.**

Why it breaks in microservices

Coordination. Inventory Service, Order Service, and Payment Service run on different servers, each with its **own database**. Enforcing atomicity means undoing all previous steps when any step fails — and **coordinating that rollback across independent services is genuinely challenging**. There is no shared transaction log, no single database to tell "undo everything."

Network latency. Services may be distributed across different geographic locations, so a transaction can take a significant amount of time. **Holding resources locked for that long is impractical** and causes performance problems.

> Made concrete: a database transaction normally holds locks for a few milliseconds. A distributed transaction across three services with network calls between them might hold them for **several seconds**. During that window, every other customer trying to buy the same product is blocked. At any real traffic level, the system grinds to a halt.

The conclusion: the traditional transaction model isn't designed for the complexity and demands of distributed systems. ACID's principles, well-intentioned as they are, **don't translate to a world where services operate independently and communicate over networks.**

Part 2 — The solution

A saga is a sequence of local transactions.

Each local transaction:

1. **Updates its own database** (a normal, fully ACID transaction — just local)
2. **Publishes an event** that triggers the next local transaction in the saga

If a local transaction **fails because it violates a business rule**, the saga executes a series of **compensating transactions** that undo the changes made by the preceding local transactions.

Rather than trying to enforce strict consistency as ACID does, Saga **acknowledges** that distributed transactions span multiple services and databases, and opts for the more practical **eventual consistency** model.

The guarantee it actually gives

Either **all transactions complete**, or **compensating transactions run to roll back** the ones already executed. The system maintains integrity through explicit compensation rather than automatic rollback.

This makes sagas ideal for **long-lived, high-level business transactions spanning multiple microservices**, across globally distributed and highly available databases.

The cost: designing sagas is complex. They require intricate coordination between participating services and **extensive error handling**. You are hand-writing what the database used to do for you.

The metaphor worth keeping

> If your distributed system is a city and each service is a building, **ACID transactions are the internal wiring of each building** — ensuring electricity flows where needed inside. But when you want to connect two buildings, you can't just extend the internal wiring; it wouldn't be safe or efficient. **You need a power line.** The Saga Pattern is that power line — connecting the buildings so they work together, while maintaining safety and consistency.

Part 3 — The two types

Choreography — decentralized

Every local transaction **publishes domain events** that trigger local transactions in other services. **There is no central coordination.** Each service produces events and listens to other services' events, deciding for itself whether to act.

> An orchestra with **no conductor** — each musician listens to the other instruments and plays their part in harmony. Or an ant colony, where each ant knows its tasks and performs them without being told.

What it looks like in practice: Order Service publishes `OrderCreated`. Payment Service is listening, sees it, charges the customer, publishes `PaymentCompleted`. Inventory Service is listening for that, reserves stock, publishes `StockReserved`. Nobody is in charge; the flow emerges from who subscribes to what.

Orchestration — centralized

A central **saga orchestrator** is responsible for executing the saga's steps. It tells each participant when to execute its local transaction.

> The same orchestra, **with** a conductor guiding all the musicians to create a symphony. Or a busy airport where an air traffic controller tells each plane when to take off and land.

What it looks like in practice: the orchestrator calls Order Service, waits, calls Payment Service, waits, calls Inventory Service. If Inventory fails, the orchestrator calls the compensations in reverse. The whole flow is readable in one place.

Choosing between them

	Choreography	Orchestration
Coordination	None — fully decentralized	Central orchestrator
Service autonomy	Higher	Lower — services do as told
Visibility of the flow	Poor — scattered across services	Good — in one place
Error handling	Harder	Easier
Coupling	Services coupled to events	Services coupled to the orchestrator
Best for	**Simple** transactions	**Complex** transactions

The rule of thumb the course gives: consider the complexity of your business transaction. If it's simple, choreography is a good option as it requires less coordination. As it becomes more complex, **orchestration provides better visibility and control**.

> The practical failure mode of choreography: with six services each reacting to events, **nobody can answer "what happens when a customer places an order?"** without reading six codebases. The business process exists but is written nowhere. That's why complex flows migrate to orchestration.

Part 4 — Architecture

Local transactions — the backbone

A saga is composed of multiple local transactions, each executed **within a single service and its corresponding database**. Each carries its own responsibilities, but strung together they work toward maintaining data consistency across the system.

Every local transaction has an associated compensating transaction capable of undoing its changes.

Compensating transactions

If a local transaction fails, the saga executes compensating transactions for the previously successful ones, bringing the system back to a consistent state.

> Walking along a path and realising you've gone the wrong way — you retrace your steps back to where you took the wrong turn.

The Saga Log

To track the saga's progress and state, a **saga log** is maintained. It records **every local transaction and compensating transaction executed** in the saga.

Its purpose: if the saga is interrupted — a service crashes mid-flow — the log lets it **resume from where it left off**, or know exactly which compensations to run.

> Hansel and Gretel's breadcrumb trail — if you get lost (a system failure), follow the breadcrumbs (the saga log) back home (a consistent state).

Database per service

Each service **owns its database**, ensuring the failure of one doesn't directly impact others. This prevents a single point of failure and makes the system more robust and reliable.

> A team of rock climbers, each with their own safety rope. If one slips, their rope catches them without pulling the rest down.

Eventual consistency

The ultimate goal. The system **may pass through inconsistent states**, but will **eventually reach a consistent one**.

> Juggling: at any given moment some balls are in the air, but a skilled juggler ensures all get handled correctly without dropping any.

Part 5 — Worked example: online shopping (orchestration)

The scenario: you add items to your cart and check out, creating an order. The Inventory Service checks whether items are available. If yes, it updates inventory and the order completes. If not, the order is cancelled.

The services — each with its own database

```
public class OrderService {
    private OrderDatabase orderDatabase = new OrderDatabase();

    public void createOrder(Order order) {
        orderDatabase.save(order);
    }

    public void cancelOrder(String orderId) {
        // COMPENSATING TRANSACTION for createOrder
    }
}
```

```
public class InventoryService {
    private InventoryDatabase inventoryDatabase = new InventoryDatabase();

    public boolean checkAndReduceInventory(Order order) {
        // local transaction — returns false if stock is insufficient
    }

    public void addBackInventory(Order order) {
        // COMPENSATING TRANSACTION for checkAndReduceInventory
    }
}
```

Notice the structure — this is the pattern in miniature. Each service has its **own database** (`OrderDatabase`, `InventoryDatabase`), and each forward operation is **paired with an inverse**:

Forward action	Compensating action
`createOrder`	`cancelOrder`
`checkAndReduceInventory`	`addBackInventory`

That pairing *is* the Saga pattern. Everything else is plumbing to invoke them in the right order.

The orchestrator

```
public class OrderOrchestrator {
    private OrderService orderService = new OrderService();
    private InventoryService inventoryService = new InventoryService();

    public void processOrder(Order order) {
        orderService.createOrder(order);

        if (!inventoryService.checkAndReduceInventory(order)) {
            orderService.cancelOrder(order.getId());   // compensate
        }
    }
}
```

Reading it: create the order first. Then attempt to check and reduce inventory. If that returns `false` — insufficient stock — **undo the order** by cancelling it.

The forward path and the compensation path are both visible in one method. That readability is orchestration's main selling point.

Handling mid-saga failures

The course raises the obvious question: *what if a service fails partway through?*

That's what the **Saga Log** is for. You expand the services to record each step. If a service fails, consult the log to see where it left off and execute the necessary compensating transactions.

> Notice this example has **no saga log at all**. If the process crashes between `createOrder` and `checkAndReduceInventory`, the order exists forever in a limbo state with no inventory reserved and nothing to fix it. That's why real implementations persist saga state before every step.

What the example is and isn't

It's a basic illustration. A real application involves more services, more complexity, and more failure modes — no retries, no asynchronous messaging, no log here. But the value of the pattern holds: it lets you **split a large transaction into manageable, independent parts that can be executed and compensated for separately.**

Part 6 — Issues and considerations

Complexity

Splitting a large transaction into several smaller ones ensures consistency but **adds complexity**. Each service must now handle **both** its local transaction **and** provide a compensating transaction.

The course's concrete difficulty: what happens if a **discount was applied** to the order, and the order then gets cancelled because items were unavailable? The Order Service's compensating transaction

must handle unwinding the discount too — perhaps returning a single-use voucher to the customer's account. Complexity creeps in quickly.

The pattern requires **careful design and implementation to avoid creating a tangled web of services.**

Longer latency

In a distributed transaction, **each service call is a network call**, and network calls are slower than local ones. Sagas therefore have **longer latency than traditional transactions**.

In a monolith, placing an order and updating inventory happens quickly inside a single database transaction. With a saga, these operations happen over the network, introducing delay.

Application consistency — not just data consistency

This is a subtle point and the most important one in the chapter.

Saga ensures **data** consistency by executing compensating transactions. But what about the **business process**?

> If the inventory check fails, the order gets cancelled. **But from a business perspective, cancelling an order is not the same as never having placed it.**

The customer may have already received a confirmation email. There's an audit record showing an order existed. Analytics counted it. A warehouse worker may have seen it on a screen.

Compensation is not time travel. It gets the data back to a consistent state, but it cannot un-happen the visible intermediate state. These considerations must be thought through when designing the saga.

Idempotency

An operation could **fail after executing but before reporting success**. In that case it may be retried — so the operations must be **idempotent** (performing them multiple times yields the same result as once).

The example: if `checkAndReduceInventory` reduces inventory but fails before reporting success, a retry must **not reduce the inventory a second time**. So that method needs to be idempotent — typically by recording the saga ID and refusing to apply the same operation twice. *(See Retry for idempotency keys.)*

Increased load

Each service must maintain a **Saga Log** to track the saga. That means **extra read and write operations for every service**, increasing load and potentially slowing response times. The Order Service now handles orders *and* manages its saga log.

It's a trade-off

As with most things in software architecture, this is about balance. Would you choose consistency over complexity, or speed over distributed transactions? Understanding the trade-offs is what lets you decide well.

Part 7 — Use cases

E-commerce systems

Order placement is a textbook distributed transaction. An **Order service** creates the order, a **Payment service** charges the customer, an **Inventory service** checks and reduces stock.

If the inventory check fails, the order must be cancelled **and the customer's payment refunded**. That refund is a compensating transaction — and note it's not a rollback. The charge really happened and really appears on the customer's statement; the refund is a second, separate movement of money.

Travel booking systems

Booking a trip involves reserving a flight, booking a hotel, and perhaps renting a car. These need to **succeed together or fail as a whole**. But they are **independent services, possibly owned by different companies**.

> This is the clearest case for Saga. There is no shared database — you cannot possibly run one ACID transaction across an airline, a hotel chain, and a car rental firm. Compensation (cancel the flight, cancel the hotel) is the *only* available mechanism.

Part 8 — System design examples

Online food delivery

When a user places an order, several steps are initiated: the order is created, the restaurant confirms it, a delivery partner is assigned, and payment is processed.

But what if the restaurant runs out of an item? The order needs to be cancelled and the payment refunded. Saga maintains data consistency across services, handles the failure scenario gracefully, and keeps the user experience seamless.

Internet banking

A user transfers money from savings to checking. The transaction involves two operations — **debiting savings** and **crediting checking**. They must happen in unison; if one fails, the other must not proceed.

> Without the Saga Pattern, a failure in the crediting operation could lead to **money disappearing into thin air** — debited from one account and never arriving in the other.

Part 9 — Saga vs. two-phase commit (2PC)

Not in the course, but you'll be asked to compare them.

2PC is the traditional way to make a transaction atomic across multiple databases. A coordinator asks every participant "can you commit?" (phase 1), and if all say yes, tells them all "commit now" (phase 2).

	2PC	Saga
Consistency	Strong (genuinely ACID)	Eventual
Locks	Holds locks across all participants until commit	No distributed locks
Availability	**Blocks** if the coordinator fails mid-protocol	Stays available
Rollback	Automatic	Manual — you write the compensating logic
Suits	Short transactions in one trust domain	Long-lived, cross-service, cross-company

Why microservices generally avoid 2PC: it holds locks for the duration of the whole distributed operation (the very problem described in Part 1), and if the coordinator dies between phases, participants are left holding locks with no instruction — a genuinely stuck system. It also requires every participant to support the protocol, which external companies won't.

Part 10 — Common beginner misconceptions

"A compensating transaction is a rollback." No. A rollback erases history; a compensation is a **new forward action that offsets the previous one**. The refund appears on the statement alongside the charge. Both are real.

"Every step can be compensated." Some can't. You cannot un-send an email, un-ship a package, or un-launch a rocket. For those, either put them **last** in the saga, or design a real-world compensation (a "please ignore that email" follow-up).

"Sagas give you ACID across services." They explicitly don't. You give up isolation entirely — other transactions **can see** your half-finished saga. That's the price of avoiding distributed locks.

"Choreography is simpler, so always start there." It's simpler for two or three services. Beyond that the invisible business process becomes a serious maintenance problem.

"The orchestrator is a single point of failure, so choreography is safer." The orchestrator can be made highly available and its state persisted. Choreography's risk isn't availability — it's comprehensibility.

"Eventual consistency means the data might just be wrong." No — it means there's a *window* during which observers may see intermediate state. It converges. The design question is whether your business can tolerate that window.

Part 11 — Check your understanding

1. Why can't you use one ACID transaction across three microservices?
2. Why is holding database locks across a distributed transaction impractical?
3. What is the pairing that defines the Saga pattern?
4. Why is "cancelling an order is not the same as never placing it" an important point?
5. Why must saga steps be idempotent?
6. Why does travel booking make the strongest case for Saga?

(Answers: 1 — each service owns its own database, so there's no single transaction manager or shared log to coordinate commit and rollback; 2 — the transaction spans network calls taking seconds, during which locked rows block every other user; 3 — each forward local transaction paired with a compensating transaction that undoes it; 4 — Saga restores data consistency but cannot undo visible side effects like confirmation emails and audit records, so the business process must account for the intermediate state; 5 — a step can succeed but fail before reporting success, then be retried,

and must not apply its effect twice; 6 — the participants are separate companies, so a shared database transaction is not merely difficult but impossible.)

Part 12 — Interview soundbite (30 seconds)

> "Saga solves distributed transactions when ACID can't cross service boundaries. Each service owns its own database, so there's no shared transaction manager to coordinate a commit — and holding locks across network calls for seconds would block every other user anyway. Instead you break the business transaction into a sequence of local transactions, each paired with a compensating transaction that undoes it. If step three fails, you run the compensations for steps two and one. You get eventual consistency rather than immediate consistency, and you give up isolation completely — other transactions can see your half-finished saga. There are two flavours: choreography, where each service emits events others react to, which is decentralized and fine for simple flows but scatters the business process across six codebases so nobody can read it; and orchestration, where a central coordinator drives the steps, giving much better visibility and error handling for complex flows. A saga log persists progress so you can resume or compensate after a crash. The classic examples are e-commerce checkout — order, payment, inventory, with a refund as compensation — and travel booking across flight, hotel, and car from different companies, where a shared transaction is simply impossible. The hard parts are that steps must be idempotent, since a step can succeed but fail to report success and then get retried, and that compensation isn't rollback: the refund is a second real movement of money, and cancelling an order isn't the same as never placing it, because the confirmation email already went out."

Part 13 — Implementation

A. Runnable demo

Full source: `demos/SagaDemo.java`

```
cd demos && ./gradlew runCh10
```

Compensation driven by the log, in reverse order:

```
void compensate(String sagaId) {
    for (String step : log.stepsNeedingCompensation(sagaId)) {   // newest first
        switch (step) {
            case "reserveStock"  -> inventory.release(sagaId);
            case "chargePayment" -> payments.refund(sagaId);
            case "createOrder"   -> orders.cancelOrder(sagaId);
        }
        log.record(sagaId, step, StepState.COMPENSATED);
    }
}
```

Idempotency using the saga ID as the key:

```
void charge(String sagaId, int amount) {
    if (charges.containsKey(sagaId)) {
        return;                      // duplicate - do NOT charge again
    }
    charges.put(sagaId, amount);
    ledger.add("CHARGE  +" + amount);
}
```

B. What you should see

```
=== 2. Step 3 fails -> compensations run in REVERSE ===
      OrderService: order created
      PaymentService: charged 500
      InventoryService: INSUFFICIENT STOCK (have 3, need 99)
    !!! step failed: out of stock -> compensating
      PaymentService: REFUNDED 500 (compensation)
      OrderService: order CANCELLED (compensation)
  order status: CANCELLED | stock: 3

=== 3. Idempotency: the same step retried ===
      PaymentService: charged 150
      PaymentService: duplicate detected, NOT charging again
      PaymentService: duplicate detected, NOT charging again

=== 4. Compensation is NOT rollback ===
  payment ledger: [CHARGE +300, CHARGE +500, REFUND -500, CHARGE +150]
```

Section 4 is the whole conceptual point, made concrete. Look at the ledger: the £500 charge and the £500 refund **both exist**. A database rollback would have erased the charge as though it never happened. Compensation adds an **offsetting entry** — and the customer sees both lines on their statement. That's why "cancelling an order is not the same as never placing it."

C. Production — orchestration

Hand-rolling saga orchestration is possible but you'll reinvent persistence, timeouts, and retries. Real options: **Temporal**, **Camunda/Zeebe**, **AWS Step Functions**, or **Axon Framework**.

Temporal is the most popular code-first choice. Your saga is ordinary code; the framework makes it durable — if the process crashes mid-saga, it resumes from the last completed step on another worker:

```java
public class OrderWorkflowImpl implements OrderWorkflow {

    private final OrderActivities activities = Workflow.newActivityStub(
        OrderActivities.class,
        ActivityOptions.newBuilder()
            .setStartToCloseTimeout(Duration.ofSeconds(10))
            .setRetryOptions(RetryOptions.newBuilder()
                .setMaximumAttempts(3)
                .setBackoffCoefficient(2.0)
                .build())
            .build());

    @Override
    public void processOrder(Order order) {
        Saga saga = new Saga(new Saga.Options.Builder().build());
        try {
            activities.createOrder(order);
            saga.addCompensation(activities::cancelOrder, order.id());   // register the undo

            activities.chargePayment(order);
            saga.addCompensation(activities::refundPayment, order.id());

            activities.reserveStock(order);                              // may fail
        } catch (ActivityFailure e) {
            saga.compensate();      // runs registered compensations in reverse
            throw e;
        }
    }
}
```

The pattern to copy even without Temporal: register the compensation **immediately after** each step succeeds. That way the list of things to undo is always exactly correct, with no bookkeeping.

Choreography with Kafka — no orchestrator; services react to each other's events:

```java
@KafkaListener(topics = "order-created", groupId = "payment-service")
public void onOrderCreated(OrderCreated event) {
    try {
        paymentService.charge(event.orderId(), event.amount());
        kafka.send("payment-completed", new PaymentCompleted(event.orderId()));
    } catch (PaymentFailedException e) {
        kafka.send("payment-failed", new PaymentFailed(event.orderId()));   // triggers compensation
    }
}
```

Note the shape: **every step publishes either success or failure**, and compensation is just another consumer reacting to a failure event. Nobody is in charge — which is exactly why complex flows become hard to follow.

D. Implementing the saga log properly

The demo's log is in memory, so a crash loses everything. Real implementations persist it **in the same transaction as the business change**, using the **Transactional Outbox** pattern:

```sql
CREATE TABLE saga_state (
    saga_id       UUID PRIMARY KEY,
    current_step  VARCHAR(50)  NOT NULL,
    state         VARCHAR(20)  NOT NULL,    -- STARTED / COMPLETED / COMPENSATING / DONE
    payload       JSONB        NOT NULL,
    updated_at    TIMESTAMP    NOT NULL
);

CREATE TABLE outbox (
    id          UUID PRIMARY KEY,
    aggregate   VARCHAR(50),
    event_type  VARCHAR(100),
    payload     JSONB,
    published   BOOLEAN DEFAULT FALSE
);
```

```java
@Transactional
public void completeStep(String sagaId, String step, Event next) {
    sagaRepo.markCompleted(sagaId, step);      // both writes
    outboxRepo.save(next);                     // in ONE local transaction
}
// A separate poller (or Debezium CDC) publishes unpublished outbox rows to Kafka
```

> **Why the outbox exists.** If you write to the database and then publish to Kafka as two separate operations, a crash between them leaves the two permanently inconsistent — the step completed but nobody was told. Writing the event to the same database in the same transaction makes them atomic; a separate process then relays it. This is the standard solution and worth knowing by name.

E. Implementation pitfalls

1. Compensations that aren't idempotent. Compensation gets retried too. Refunding twice is as bad as charging twice. Guard with the saga ID.

2. Compensations that can fail permanently. What if the refund API is down? You need retries with backoff, and ultimately a **dead-letter queue plus human intervention**. A saga stuck half-compensated is the worst possible state — plan for alerting on it.

3. Putting an uncompensatable step in the middle. You cannot un-send an email or un-ship a parcel. Order your steps so **irreversible actions come last**, after everything that could fail has succeeded.

4. An in-memory saga log. As in the demo. A crash mid-saga leaves orphaned state with nothing to recover from. Persist before each step.

5. Writing to the database and publishing separately. Use the outbox. This is the single most common correctness bug in event-driven sagas.

6. Assuming isolation. Sagas have **no isolation** — other transactions see your intermediate state. A customer may see an order that's about to be cancelled. Design the UI for it ("processing" rather than "confirmed").

7. Choreography for a complex flow. Six services reacting to each other's events means nobody can answer "what happens when an order is placed?" without reading six codebases. Move to orchestration once you pass three or four steps.

8. No timeouts on saga steps. A step that never responds leaves the saga hanging forever. Every step needs a deadline, after which it's treated as failed and compensation begins.

Related patterns

- **Event-Driven Architecture** — the messaging substrate choreography sagas run on
- **CQRS** — frequently paired with sagas in event-sourced systems
- **Retry** — idempotency concerns are shared; retries happen within saga steps
- **Circuit Breaker** — protects individual saga participants

Event-Driven Architecture (EDA) Pattern

Category: Data / messaging **One-liner:** Components communicate by publishing and reacting to events rather than calling each other directly — giving loose coupling, asynchrony, independent scaling, and real-time responsiveness.

Part 0 — Concepts you need first

What is an "event"?

An **event** is a record that **something happened**. It's a statement of fact about the past.

The naming convention tells you a lot: events are named in the **past tense** — `OrderPlaced`, `PaymentReceived`, `UserRegistered`. This isn't cosmetic. It reflects that an event is a *notification of something already done*, not a request for something to be done.

Contrast with a **command**, which is an instruction, named imperatively — `PlaceOrder`, `ChargeCard`. A command can be refused. An event cannot; it already happened.

Synchronous vs. asynchronous

Synchronous — you make a request and **wait** for the response before doing anything else. Like a phone call: both parties are engaged for the duration.

Asynchronous — you send a message and **carry on immediately**, handling the response whenever it arrives. Like email: you send it and get on with your day.

The practical difference for a server: in a synchronous call, the calling thread is **blocked and useless** for the whole wait. In an asynchronous one, it's free immediately.

What is a message queue / event bus?

Middleware that accepts messages from senders and delivers them to receivers. **Kafka** and **RabbitMQ** are the common ones.

The important properties:

- The **sender doesn't need to know who receives** — it publishes to the bus
- Messages are usually **persisted**, so if a receiver is down, the message waits
- **Many receivers** can consume the same message independently

What is publish-subscribe?

A messaging style where **publishers** send messages to a named channel (or "topic") without knowing who's listening, and **subscribers** register interest in a channel and receive everything published to it.

The essential property: **the publisher doesn't hold a list of subscribers.** Adding a new subscriber requires no change to the publisher — which is the entire source of EDA's flexibility.

What is "coupling"?

How much one component depends on knowledge of another.

Tight coupling: Service A calls `serviceB.doThing()`. A must know B exists, where it lives, its API, and its response format. Change B and you may break A.

Loose coupling: A publishes `ThingHappened`. It doesn't know whether anyone is listening. B may consume it, or five services may, or none. **A is unaffected either way.**

Part 1 — The problem

As applications grew, developers adopted distributed systems for scalability and resilience. But that shift brought challenges.

Managing complex interactions

A simple system has a small number of components interacting predictably. **Scale up and this becomes much harder.** A distributed system has a multitude of components that must interact and communicate. How do they communicate effectively? How do you ensure changes in one part are reflected in others?

The course's grounding example: consider how an email notification system works. When you receive an email saying a product has shipped, or that a blog you follow has a new post — that's a system efficiently handling complex interactions.

The call for asynchronous communication

In a synchronous system, a component makes a request and waits for the response before proceeding. This works for small, simple systems, but **quickly becomes a bottleneck** as the system grows.

> The question that makes it click: *what happens when you have thousands or even millions of components all waiting for responses?* Every waiting component is holding resources and doing nothing.

Asynchronous communication is often the answer — make a request, continue with other tasks, process the response when it arrives. Far more efficient, but **managing it effectively is a considerable challenge**.

The everyday example: why can you keep browsing an online store and adding items to your cart while previous requests are still being processed? Efficient asynchronous communication.

The demand for real-time responsiveness

Users expect immediate responses. Post a status update and you expect it visible to friends immediately. In online multiplayer games, any delay produces a poor experience.

Designing a system that manages complex interactions between components **and does so in real time** is no small feat.

Data consistency

With multiple components potentially reading and writing data concurrently, how do you ensure consistency? If one component updates a piece of data, how do others become aware of the change?

The example: an online banking system. When you make a transaction it's critical your balance updates promptly and accurately — but what if another transaction is being processed on your account at the same moment?

Part 2 — The solution

EDA revolves around the **production, detection, consumption of, and reaction to events**.

An **event** is a change in state, or an update, produced by a component — the **event producer**. It's then detected and consumed by one or more **event consumers**.

> **The courier analogy:** a package (event) is sent by the sender (producer) and received by the recipient (consumer). The delivery service (event bus) takes care of delivering it. Scale it up

> to many senders and receivers and the core concept is unchanged.

Part 3 — The benefits

Decoupling

Event producers and consumers **aren't tied together and don't need to be aware of each other's existence**. This makes the system more flexible and adaptable to change.

The illustration that makes this concrete: a user uploads a photo to a social media app. That **single event** can trigger multiple actions:

- Notifying the user's followers
- Updating the user's profile
- Alerting the moderation team if the content violates policy

With EDA, **each of these is handled independently**. The system efficiently manages multiple tasks triggered by one event.

> The part worth pausing on: **six months later you want to add "generate a thumbnail."** With EDA you write a new consumer subscribing to `PhotoUploaded` and deploy it. **You never touch the upload code.** In a direct-call design you'd modify the upload service, retest it, and redeploy it — and repeat that for every new reaction, forever. That difference in change cost is the pattern's real value.

Asynchrony

The producer simply produces the event and **carries on with its operations without waiting** for consumers to process it. This non-blocking operation greatly enhances the system's overall responsiveness and throughput.

Scalability

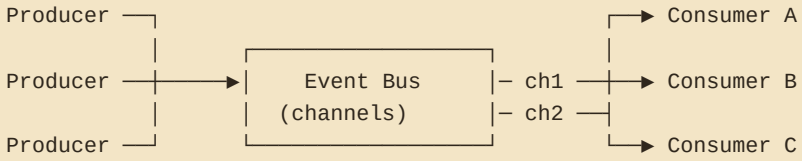
Since producers and consumers are decoupled, they can be **scaled independently** based on their workload. A surge of events from producers? Increase the number of consumer instances to handle the load — and vice versa.

Real-time processing

As soon as an event is produced, it can be **immediately handled** by the consumer. This is particularly useful where instant action is required — **fraud detection in banking transactions**, or real-time analytics.

Part 4 — The components

Component	Role
Event Producers	Create events. They define what an event is, encapsulate the relevant data, and send it to the Event Bus
Event Bus (message queue)	The backbone. Receives events from producers and transmits them to consumers. **This is what decouples them**
Event Channels	Pathways on the bus. They categorize events by nature, or by the kind of consumers they cater to, so each consumer listens only to the channel relevant to it
Event Consumers	Listen to events via the bus and react. The reaction could be starting a new process, or merely recording the event



Part 5 — Three structural variants

The choice depends on system requirements, complexity, and how much control you need over event handling.

Mediator EDA

A **central mediator** takes charge of event handling.

Producers send their events to the mediator. The mediator — **aware of all producers and consumers** — determines where to route them. It performs **event aggregation, filtering, and transformation**, then delivers the processed events to the appropriate consumers.

- **Advantage:** control. It allows complex routing and transformation logic, giving a high degree of control over event processing.
- **Disadvantage:** the mediator could become a **bottleneck**, limiting the system's scalability.

Broker EDA

No central mediator. Event producers send events directly to Event Channels on the Event Bus. Consumers subscribe to those channels and consume events directly. **The broker essentially *is* the event bus.**

- **Advantage:** scalability and simplicity. With no central mediator, it's easier to scale.
- **Disadvantage:** less control over event routing and processing.

Hybrid EDA

Combines elements of both. Adopted when a system requires a mix of **centralized control and high scalability** — for example, a mediator for a few complex flows and direct broker delivery for everything else.

Part 6 — Inner workings

Event generation and propagation

An event's life starts with a **change of state** — a user action, a sensor reading, an internal system update. An event is generated to encapsulate that change: essentially **a bundle of data representing a significant occurrence**.

Once generated it must be propagated. The **Event Bus** is the spinal cord, carrying events from producers to consumers via **Event Channels** — pathways that categorize and route events by type or nature, ensuring the right events reach the right consumers.

Routing

- **In Mediator EDA:** a central component decides routing based on event type and known consumers. It may also aggregate, filter, or transform events before sending them on.
- **In Broker EDA:** there's no central decision-maker. Producers place events directly into channels; consumers subscribe and consume. Simpler routing, but less controlled.

> Without routing, events would simply float aimlessly in the system. With it, events know precisely where to go.

Event processing and state change

Once an event reaches its consumer, it is processed. The consumer takes the event data and performs actions based on it, **leading to a change in system state** — anything from changing a value in a database to triggering a physical action in an IoT device.

How consumers know when to act

Event consumers are **always listening** — monitoring their subscribed channels on the bus, waiting. When an event arrives, they act.

This listening mechanism is usually implemented using the **Observer** or **Publish-Subscribe** patterns.

Real-time illustration — traffic management

Various sensors act as producers, generating an event for each vehicle detected. The events travel via the bus, through appropriate channels, to the consumers — traffic signals. Based on the events received (for example vehicle count), the signals **adjust their timing**, changing the system state: traffic flow.

Throughout, the system reacts in real time.

Event Sourcing and CQRS

Two significant related concepts:

Event Sourcing means changes to application state are stored as a **sequence of events**, not just as a snapshot of the current state. This allows:

- **Full auditability** — every change is recorded
- **Versioning**
- The ability to **"time-travel"** — see the system's state at any past point

> Worth grasping the shift: instead of storing "balance = £70," you store `Deposited £100`, `Withdrew £30`. The balance is **derived** by replaying those events. You've kept the history rather than just the conclusion — which is exactly how accounting ledgers have always worked.

CQRS involves **separating the read and write operations** of a system. This is especially useful in an EDA, as it lets you model your event handlers (write operations) separately from your views (read operations).

Part 7 — Worked example: traffic management (Java)

The event

```
public class VehicleEvent {
    private final String vehicleId;
    private final String type;

    public VehicleEvent(String vehicleId, String type) {
        this.vehicleId = vehicleId;
        this.type = type;
    }

    public String getVehicleId() { return vehicleId; }
    public String getType()      { return type; }
}
```

This represents a vehicle's action — such as 'entering' or 'leaving' a zone.

Note the `final` fields and absence of setters — the event is immutable. That's deliberate and correct. An event describes something that **already happened**; allowing a consumer to modify it would be meaningless (you can't change the past) and dangerous (other consumers would see the altered version).

The producer — a sensor

```
public class Sensor implements EventProducer {
    private EventBus eventBus;

    public Sensor(EventBus eventBus) {
        this.eventBus = eventBus;
    }

    public void vehicleDetected(String vehicleId, String action) {
        VehicleEvent event = new VehicleEvent(vehicleId, action);
        eventBus.publish(event);
    }
}
```

Sensors at traffic signals act as producers. Every time a vehicle passes, the sensor generates a `VehicleEvent` and publishes it to the bus.

Notice what's absent: the sensor has no reference to any traffic light. It doesn't know consumers exist. It publishes and returns — **fire and forget**.

The bus and channels

```
public interface EventBus {
    void publish(VehicleEvent event);
    void addChannel(EventChannel channel);
}

public class SimpleEventBus implements EventBus {
    private List<EventChannel> channels = new ArrayList<>();

    @Override
    public void publish(VehicleEvent event) {
        for (EventChannel channel : channels) {
            channel.add(event);
        }
    }

    @Override
    public void addChannel(EventChannel channel) {
        channels.add(channel);
    }
}

public class SimpleEventChannel implements EventChannel {
    private List<EventConsumer> consumers = new ArrayList<>();

    @Override
    public void add(VehicleEvent event) {
        for (EventConsumer consumer : consumers) {
            consumer.onEvent(event);
        }
    }

    @Override
    public void addConsumer(EventConsumer consumer) {
        consumers.add(consumer);
    }
}
```

When an event is published to the bus, it is added to all the channels; each channel then passes it to all its consumers.

The consumer — a traffic light

```
public class TrafficLight implements EventConsumer {
    @Override
    public void onEvent(VehicleEvent event) {
        if ("entering".equals(event.getType())) {
            System.out.println("Vehicle " + event.getVehicleId()
                + " is entering. Adjusting traffic light timings.");
        } else if ("leaving".equals(event.getType())) {
            System.out.println("Vehicle " + event.getVehicleId()
                + " has left. Resetting traffic light timings.");
        }
    }
}
```

The traffic light reacts to vehicle events by adjusting its timings.

What this example simplifies — read before copying it

The structure is right; the mechanics are not production-shaped:

- **It's entirely synchronous and in-process.** `publish` calls consumers **directly on the caller's thread**. The sensor doesn't return until every traffic light has finished processing — which destroys the asynchrony that is the pattern's main benefit. A real EDA uses Kafka or RabbitMQ, with consumers running in **separate processes** and the producer returning immediately.
- **No persistence.** If the process dies, in-flight events are gone. Real buses persist events so they survive restarts and can be replayed.
- **No error handling.** If one consumer throws, the loop breaks and later consumers never see the event.
- **The bus broadcasts to every channel** rather than routing by type, so channels don't actually categorize anything here.

Use it to understand the **roles** — producer, bus, channel, consumer — not the implementation.

Part 8 — Performance implications and considerations

Scalability

Each component is decoupled, so you can **scale them independently** based on demand. High volume of events but consumers can't keep up? Scale up consumers without affecting producers or channels. However, **ensuring balance to avoid overwhelming certain parts of the system requires careful consideration.**

Processing speed and latency

EDA commonly boosts performance because of its asynchronous nature — producers continue producing without waiting for consumers.

The flip side is latency. As the system scales and event volume increases, **the time from an event being produced to it being processed can grow**, delaying responses. Overall speed is also affected by the choice of event channel — synchronous or asynchronous.

Error handling and message durability

What happens when an event fails to be processed? How do you ensure messages aren't lost in transit?

If a consumer fails to process an event, the system needs a mechanism to **retry processing, log the failure, or alert an operator**. Ensuring **message durability** might require **persistent storage** to hold events until they're successfully processed.

Note the trade-off the course flags: **adding these mechanisms complicates the system and impacts performance.**

System complexity

EDA undoubtedly introduces complexity. **Tracking the flow of events and debugging issues is challenging** due to the asynchronous and distributed nature of the system. **Special tools and practices** may be needed to monitor and debug effectively. Designing a system where every component understands the format and meaning of events is itself complex.

> ⚠️ **This is the cost beginners most underestimate.** In a synchronous system, a stack trace tells you the whole story of what called what. In EDA, causation is spread across services *and across time* — the event that caused today's bug was published twenty minutes ago by a service you weren't looking at. You need **correlation IDs** (a shared identifier threaded through every event

> in a causal chain) and **distributed tracing** from day one, not bolted on after your first incident.

Event schemas and backward compatibility

Since all communication is via events, **all components must understand the events' structure**. This means events should follow a **well-defined schema**.

If the schema changes — new fields added, existing ones removed or altered — you must ensure **backward compatibility** so consumers expecting the old schema don't break.

> Practical rule: **adding an optional field is safe; removing or renaming one is not.** Old consumers ignore fields they don't know about, but break when a field they depend on vanishes. Schema registries exist to enforce this.

Order of events

In a system where order of operations matters, EDA presents a challenge. Events are **asynchronous by nature and could be processed in any order**. If your application has dependencies between events, you'll need a way to maintain the required order — which introduces further complexity.

> Concretely: if `AccountCreated` and `AccountDeleted` arrive out of order, a consumer might delete an account that doesn't exist yet and then create one that should be gone. Kafka guarantees ordering *within a partition*, which is why choosing a partition key (like account ID) matters.

Testing

How do you test an event-driven system? **Unit testing individual components isn't enough** — it won't cover the interactions between parts. You'll need comprehensive **integration and end-to-end tests**, which ensure the system works as a whole but bring added complexity in writing and maintaining them.

None of these should deter you — they're a guide to navigating the challenges. Every architecture has trade-offs; the trick is understanding them and making informed decisions.

Part 9 — Use cases

Real-time analytics

In an increasingly data-driven world, processing and analysing data in real time is a powerful advantage. As events are generated they can be **instantly consumed and processed**, giving real-time insights. Especially useful in **finance**, where real-time stock market data drives instantaneous trading decisions.

IoT systems

IoT devices generate a multitude of events — a sensor detecting an environmental change, a smart device being activated, a security system triggered. The **asynchronous nature of EDA is perfectly suited** to handling these. In a smart home, an increase in room temperature triggers an event consumed by the air conditioning system, which adjusts accordingly.

E-commerce systems

An e-commerce platform comprises subsystems like inventory, billing, and shipping. When a customer places an order it triggers a series of events: inventory verifies stock, billing processes payment,

shipping handles delivery. With EDA these operations are carried out **asynchronously and efficiently**.

Microservices architecture

Services need to communicate. **Instead of direct communication — which leads to tightly coupled systems — services emit events that other services consume.** This keeps services loosely coupled and able to **evolve independently over time**.

Part 10 — System design example: a ride-sharing app

Designing something like Uber or Lyft with EDA. Various events drive the entire process.

1. **A rider requests a ride** → generates an event containing the necessary details such as pickup location and destination. The system **broadcasts** this event.
2. **Drivers in the vicinity** act as consumers of this event.
3. **A driver accepts the ride** → triggers another event, consumed by the rider's app, informing them of the match.
4. **During the ride**, the driver's app continuously sends events about the **current location**. The rider's app consumes these and **updates the display in real time**.
5. **The ride ends** → the driver's app sends a **ride completion event**.
6. **The billing service** consumes it, calculates the fare, and **triggers a payment event**.

This showcases EDA handling complex workflows while promoting real-time, asynchronous processing and loosely coupled design.

> Notice how naturally this maps: a ride genuinely *is* a sequence of things that happened. When your domain is already event-shaped, EDA stops feeling like a technical choice and starts feeling like the obvious model.

Part 11 — Common beginner misconceptions

"Events are just asynchronous function calls." No. A function call has one specific target and expects a return. An event has **zero or many** targets, unknown to the publisher, and expects nothing.

"EDA means no more direct calls anywhere." No. When you need an immediate answer — "is this user authorized?" — a synchronous call is right. EDA is for notification and reaction, not for queries.

"The event bus makes my system reliable." It makes it *decoupled*. Reliability requires persistence, retries, dead-letter queues, and idempotent consumers on top.

"If I publish an event, it definitely gets processed." Only with durable messaging and acknowledgements. Fire-and-forget over an in-memory bus loses events on restart.

"Consumers can assume they see each event exactly once." Most systems guarantee *at-least-once* delivery, meaning **duplicates happen**. Consumers should be idempotent. *(See Retry.)*

"Ordering is guaranteed." Generally not across the whole system — only within a partition or channel, if your broker offers even that.

Part 12 — Check your understanding

1. Why are events named in the past tense?

2. What's the change-cost argument for EDA, using the photo upload example?
3. What's the difference between Mediator and Broker EDA, and what does each trade away?
4. Why is debugging harder in EDA, and what two tools address it?
5. Which schema changes are safe and which break consumers?
6. What's wrong with the example's `publish` method, relative to real EDA?

(Answers: 1 — an event records something that already happened and cannot be refused, unlike a command; 2 — adding a new reaction means writing a new consumer without ever modifying the upload service, whereas direct calls require changing and redeploying the producer each time; 3 — Mediator centralizes routing and transformation, gaining control but risking a bottleneck, while Broker publishes straight to channels, gaining scalability but losing routing control; 4 — causation is spread across services and time so there's no single stack trace, addressed by correlation IDs and distributed tracing; 5 — adding optional fields is safe, removing or renaming is not; 6 — it calls consumers synchronously on the producer's thread, so the producer blocks until all consumers finish, eliminating the asynchrony that is the point.)

Part 13 — Interview soundbite (30 seconds)

> "Event-driven architecture has components communicate by publishing events rather than calling each other directly. A producer publishes a state change to an event bus, and any number of consumers subscribe to the channels they care about. The producer doesn't know who's listening, which is the whole point — when a user uploads a photo, that one event can trigger follower notifications, a profile update, and content moderation, and six months later you add thumbnail generation by writing a new consumer without ever touching the upload code. You also get asynchrony, so producers don't block, and independent scaling of producers and consumers. There are three variants: broker, where producers publish straight to channels — simple and scalable; mediator, where a central component routes, filters, and transforms — more control but a potential bottleneck; and hybrid. Ride-sharing is a natural example, with ride requests, driver acceptance, live location updates, and completion all as events. The costs are real: ordering isn't guaranteed across the system, you have to version event schemas carefully because removing a field breaks old consumers, delivery is usually at-least-once so consumers must be idempotent, and debugging is much harder because causation spans services and time — so you need correlation IDs and distributed tracing from the start. It pairs naturally with event sourcing, where state is stored as the sequence of events rather than a snapshot, and with CQRS."

Part 14 — Implementation

A. Runnable demo

Full source: **`demos/EventDrivenDemo.java`**

```
cd demos && ./gradlew runCh11
```

Failure isolation — one consumer throwing must not affect the others:

```
for (Subscriber s : subs) {
    Runnable task = () -> {
        try { s.handler().accept(e); }
        catch (Exception ex) {
            System.out.printf("      [%s] FAILED: %s (isolated)%n", s.name(), ex.getMessage());
        }
    };
    if (async) pool.submit(task); else task.run();
}
```

Idempotent consumer, using a set of seen event IDs:

```
void handle(Event e) {
    String eventId = e.payload().get("eventId");
    if (!processed.add(eventId)) return;      // add() is false if already present
    chargesApplied++;
}
```

B. What you should see

```
=== 1. SYNCHRONOUS bus: producer waits for every consumer ===
  producer blocked for 114ms  <- the sum of all consumers

=== 2. ASYNCHRONOUS bus: producer returns immediately ===
  producer returned in 0ms  <- did NOT wait
      [moderation] FAILED: moderation service unavailable (isolated)

=== 4. Add a NEW consumer - producer code untouched ===
    [bus] publishing PhotoUploaded (cid=cid-3) to 4 subscriber(s)
  >> 4 reactions now. The upload service never changed.

=== 5. At-least-once delivery -> idempotent consumers ===
      [billing]    charge applied (total=1)
      [billing]    duplicate evt-abc ignored
      [billing]    duplicate evt-abc ignored
  charges actually applied: 1 (must be 1)
```

114ms → 0ms is the value of asynchrony as a measured number. And section 4 is the whole architectural argument: the subscriber count went from 3 to 4 and **the producer's code was never edited**.

C. Production — Kafka

```
spring:
  kafka:
    bootstrap-servers: kafka-1:9092,kafka-2:9092,kafka-3:9092
    producer:
      acks: all                         # wait for all replicas - no silent data loss
      retries: 3
      properties:
        enable.idempotence: true        # broker de-duplicates producer retries
        max.in.flight.requests.per.connection: 5
    consumer:
      group-id: notification-service
      auto-offset-reset: earliest
      enable-auto-commit: false         # commit only AFTER successful processing
      isolation-level: read_committed
```

Producer — the partition key is the important decision:

```
@Service
public class PhotoEventPublisher {
    private final KafkaTemplate<String, PhotoUploaded> kafka;

    public void publish(String photoId, String userId) {
        PhotoUploaded event = new PhotoUploaded(
                UUID.randomUUID().toString(),      // eventId, for consumer de-duplication
                MDC.get("correlationId"),          // trace the causal chain
                photoId, userId, Instant.now());

        // Key = userId -> all events for one user land on one partition,
        // and Kafka guarantees ordering WITHIN a partition.
        kafka.send("photo-uploaded", userId, event);
    }
}
```

Consumer — manual acknowledgement plus idempotency:

```
@KafkaListener(topics = "photo-uploaded", groupId = "thumbnail-service")
public void onPhotoUploaded(PhotoUploaded event, Acknowledgment ack) {
    if (processedEvents.existsById(event.eventId())) {   // at-least-once => duplicates happen
        ack.acknowledge();
        return;
    }
    try {
        thumbnailService.generate(event.photoId());
        processedEvents.save(new ProcessedEvent(event.eventId(), Instant.now()));
        ack.acknowledge();                                // commit only on success
    } catch (TransientException e) {
        // Do NOT ack - Kafka redelivers
        throw e;
    }
}
```

Dead-letter topic — where messages go after retries are exhausted:

```
@Bean
public DefaultErrorHandler errorHandler(KafkaTemplate<Object, Object> template) {
    var recoverer = new DeadLetterPublishingRecoverer(template,
            (record, ex) -> new TopicPartition(record.topic() + ".DLT", record.partition()));
    return new DefaultErrorHandler(recoverer,
            new ExponentialBackOff(1000L, 2.0));   // 1s, 2s, 4s...
}
```

Schema evolution — register schemas so a breaking change is rejected at build time, not discovered in production:

```
spring.kafka.properties:
  schema.registry.url: http://schema-registry:8081
  value.subject.name.strategy: io.confluent.kafka.serializers.subject.TopicNameStrategy
```

Compatibility mode `BACKWARD` means **new consumers can read old events**. Adding an optional field passes; removing a field fails the build.

D. Implementation pitfalls

1. Auto-commit enabled. With `enable-auto-commit: true`, Kafka commits the offset on a timer — possibly *before* your handler finishes. Crash mid-processing and the message is **silently lost**. Always commit manually after success.

2. Consumers that aren't idempotent. At-least-once delivery is the norm, so duplicates are guaranteed, not hypothetical. Store processed event IDs.

3. No partition key, then expecting ordering. Kafka orders **within a partition only**. Without a key, events for the same entity scatter across partitions and arrive out of order. Key by entity ID.

4. Publishing before committing the database transaction. Classic bug: you write to the DB, then publish, and crash in between — the state changed but nobody was told. Use the **Transactional Outbox** *(see Saga Part 13D)*.

5. Events carrying too much or too little. Too little and every consumer must call back to the producer, recreating coupling. Too much and the event becomes a schema everyone depends on. Carry **identifiers plus the facts that actually changed**.

6. Removing or renaming event fields. Old consumers break. Additive changes only; use a schema registry to enforce it.

7. No correlation ID. Without one you cannot reconstruct what caused what. Generate at the edge, propagate through every event, log on every line.

8. No dead-letter queue. A permanently-failing message blocks its partition forever, stalling every message behind it. Always configure a DLT with alerting.

Related patterns

- **CQRS** — separate read/write models, commonly built on EDA
- **Saga** — choreography sagas are EDA applied to distributed transactions
- **Retry** — requeueing and dead-letter queues are retry in an async context
- **Bulkhead** — a queue between components is a natural isolation boundary

CQRS — Command Query Responsibility Segregation

Category: Data / messaging **One-liner:** Split a system's operations into commands (writes) and queries (reads) with separate models, so each can be optimised, scaled, and evolved independently.

Part 0 — Concepts you need first

What is CRUD?

Create, Read, Update, Delete — the four basic operations on stored data. Most applications start here: one `User` class, one `users` table, one set of methods that create, read, update, and delete users.

CRUD is simple and intuitive, and for most applications it's the right answer. This chapter is about what happens when it stops being enough.

What is a "model"?

In this chapter, a **model** means the code and data structures representing part of your domain — the `User` class, its validation rules, its database table, the code that loads and saves it.

CQRS's central claim is that **one model trying to serve both reads and writes ends up serving neither well**, because the two have genuinely opposed requirements.

Reads vs. writes — why they differ

This is the intuition the whole pattern rests on.

	Writes	Reads
Must be	Correct, validated, secure	Fast
Volume	Usually lower	Usually far higher
Complexity	Business rules, constraints	Joining and formatting
Cost of error	Severe — corrupt data	Mild — a stale display
Optimal storage	Normalised (no duplication)	Denormalised (pre-joined)

That last row is the deep conflict. **Normalised** storage stores each fact once — excellent for writes, because updating a fact means changing one row. **Denormalised** storage duplicates data into ready-made shapes — excellent for reads, because the answer is already assembled with no joins.

You cannot have both in one schema. Every design is a compromise between them — and CQRS says: stop compromising, have two.

What is a DTO?

A **Data Transfer Object** — a simple object whose only job is to carry data to whoever asked for it. It has no business logic. In CQRS, the read side returns DTOs shaped for the screen, rather than returning domain objects.

What does "side effect" mean?

A change to state that persists after a method returns. Reading data has no side effects; saving data does. This distinction is what CQRS formalises.

Part 1 — The problem

In a monolithic application we usually use **one database for everything**. That single database handles both read and write operations — meaning it's simultaneously serving **complex join queries** *and* performing **CUD** (Create, Update, Delete) operations. As the application gets more complicated, **handling these read and write tasks becomes really hard to manage.**

The scale mismatch

In an ideal world, CRUD operations provide a simple and intuitive method for handling data. But complex applications demand handling numerous concurrent operations, diverse data requirements, and efficient scaling — and traditional CRUD struggles to keep up.

The e-commerce picture: thousands of users browsing products (**read** operations) while concurrently there are inventory updates, order placements, and product additions (**write** operations).

Can a single model efficiently handle these varied requirements? That's where the cracks show.

Note the asymmetry: **read operations are significantly more numerous, while write operations carry more complexity.** Can we scale them independently based on their different needs? Not with a single data model handling both.

The optimisation conflict — the banking example

A banking application handles two main kinds of operation:

- **Transactions** — depositing or withdrawing money
- **Balance inquiries** — checking the current account balance

Their needs are quite different:

Writing data (transactions) involves creating new records or updating existing ones. It needs to be **accurate and secure**, as it directly affects a user's finances.

Reading data (balance inquiries) is about fetching data **quickly and efficiently**. Users expect to see their balance in real time, without delay.

> **The trap, stated plainly:** if you focus too much on making transactions fast and secure, you end up making balance inquiries slower. If you optimise for quick balance checks, you compromise the efficiency or security of processing transactions. **One model cannot be tuned for both.**

This is a classic case where the traditional approach falls short. It becomes difficult to maintain, scale, and perform well under different types of load.

Part 2 — The solution

Break the term down:

- **Command** — an instruction to *do* something: creating, updating, or deleting data. In the banking example, processing a deposit or a withdrawal.
- **Query** — a request to *retrieve* data **without changing it**. Checking your account balance.
- **Responsibility Segregation** — dividing responsibilities: separating the handling of commands (writes) from queries (reads).

The two sides

The Command Side handles all write operations. It's responsible for updating data — processing transactions in the banking app.

The Query Side deals with all read operations. It's optimised for fetching data quickly — showing your current account balance.

The formal rule

This is the definition that gives the pattern its rigour, and it's worth memorising. The separation of concerns distinguishes between methods that change state and those that don't. **Each method of an object falls into one of these categories, but never both:**

	Returns a value?	Changes state?
Query	✅ Returns a result	❌ No state change, no side effects
Command *(also called modifiers or mutators)*	❌ Only an indication of success or failure	✅ Changes system state

> **Why "never both" matters.** A method that both changes state *and* returns data is impossible to call safely — you can't retry it (it might apply twice), you can't cache it, and you can't reason about it. Enforcing the split makes queries freely repeatable and cacheable, and makes commands explicit about being consequential.

Why this is useful

Optimization — by separating reads and writes you can optimise each independently. The command side focuses on security and accuracy; the query side is optimised for speed.

Simplicity — it simplifies the design. Each side is developed and maintained separately, reducing complexity.

Scalability — as the application grows, scale read and write operations independently based on their specific demands.

Flexibility in technology choice — different technology stacks can be used for the read side and the write side if that suits their needs better. A relational database for writes with its constraints and transactions; a document store or search index for reads.

In essence: CQRS acknowledges that the needs for reading and writing data are different, and lets you treat them separately.

Part 3 — Architecture: three levels

CQRS is not one thing — it's a spectrum. Knowing which level you mean is important, because the costs differ enormously.

Level 1 — Separate models, one database

The Command Model handles all write operations. It is responsible for maintaining the **integrity and consistency** of data. It receives commands, performs validations, applies business rules, and alters system state. *The course's image: a disciplined librarian, ensuring every book is in its proper place.*

The Query Model handles all read operations, providing the current state of the system to users. It's optimised for reads, ensuring users get data efficiently and quickly. *A friendly librarian, always ready to fetch the book you need.*

This is the simplest useful form — two models, one database. **Most teams should stop here.** You get most of the design clarity with almost none of the operational cost.

Level 2 — Separate read and write databases

To enhance flexibility and scalability, you can use **separate databases** for reads and writes.

> ⚠️ However, this **significantly increases complexity**, as it requires careful consideration of how to propagate changes from the write database to the read database.

The specific problems this creates, all of which the course lists:

- **No atomic transactions across the read and write models** — so you must rely on **asynchronous messaging and events** to achieve **eventual consistency**
- **Out-of-order event propagation**
- **Event loss**
- **Sync discrepancies**
- **Read Model update failures**
- **The timing of UI updates after writing** — what does the user see immediately after they hit save, if the read model hasn't caught up?

Implementing and managing this style **demands a high degree of complexity and a deep understanding of distributed transactions**. It's suitable **only if your application's non-functional requirements genuinely demand such an intricate solution.**

Level 3 — Event Sourcing

This exists to solve one specific problem from Level 2: **ordering**.

Using separate databases makes maintaining sync with eventual consistency hard, and **the sequence of events from write side to read side is crucial**. If updates on a write model occur quickly one after another, **receiving events out of order leaves the read model with outdated information.**

The complication: **many asynchronous message buses prioritise availability and performance, and often don't guarantee the order of message delivery.**

Event Sourcing addresses this with a different method of storing write models: **append-only event stores** that record every action on a model. When updating, the model is refreshed by **replaying its entire event history**.

In this approach each write model instance exists as an **independent event stream**, which enables:

- **Various data views**, by replaying the events differently
- **Rebuilding read models** from scratch by retrieving all events from the write side — the direct fix for synchronisation problems
- **Simplified auditing**, since the event streams document every change to every model
- **Creating new read models** by replaying past events — you can add a new view of historical data that never existed before

> **However:** implementing event sourcing **adds complexity and is a significant undertaking**, especially for anyone new to CQRS, event sourcing, or distributed systems. Do not adopt Level 3 because you adopted Level 1. They are separate decisions with very different costs.

Part 4 — Inner workings

The journey of a command

A command signifies an **intention to change state**. Follow one through: a user wants to update their address in an e-commerce application. The command is `UpdateUserAddress`.

Step 1 — Validation. Before anything else, the command is validated. This can involve checks for **data format, permissions, and other business rules**. Is the new address in a valid format? Does the user have permission to change it? Pass, and it proceeds; fail, and it's **rejected**.

Step 2 — Handling. Once validated, the command is handled by the **command handler** — the workhorse of the command side. It's responsible for updating system state based on the command; here, updating the user's address.

Step 3 — Generating events. Every command leads to a change in state, represented by an **event** — here, `UserAddressUpdated`. This event is stored in the **event store**, serving as a historical record of all changes in the system.

The query's quest

A query is simply a request for data. Our user now wants to view their updated address, issuing `GetUserAddress`.

Unlike commands, queries require no validation and generate no events. They're simply processed by the **query handler**, which fetches the required data and returns it.

> That asymmetry is worth noticing — it's a direct consequence of the formal rule. Because a query changes nothing, there's nothing to validate against business rules and nothing to record. This is why the read path can be made so much simpler and faster than the write path.

Synchronising the two sides

How do the two sides stay in sync? This is where **Event Sourcing** comes in.

The `UserAddressUpdated` event generated when the user updated their address is **replayed on the query side** to update its state. The user's updated address is thereby reflected in the query model. Event Sourcing ensures that despite working separately, **the command and query sides remain in sync**.

Part 5 — Worked example: user profile management

A simple application managing user profiles. You can **create a new user** (command), **update their email** (command), and **view user details** (query).

The pieces

Command Model (Write Model):

- **Command Class** — defines an action to perform, e.g. `CreateUserCommand` with `userId`, `name`, `email`
- **Command Handler** — processes commands: takes the command, performs business logic, persists changes
- **Domain Model** — represents the state of the data (the `User` class)
- **Repository** — handles data persistence

Query Model (Read Model):

- **DTO** — represents the data we want to read, e.g. `UserProfileDTO`

- **Query Class** — defines a query, e.g. `GetUserProfileQuery` with a `userId`
- **Query Handler** — handles the query and returns the data, often using **read-optimised storage**

Infrastructure:

- **Command Bus** — routes commands to their handlers
- **Query Bus** — routes queries to their handlers
- **Event Store** — in a more advanced setup, stores the events resulting from commands

Command classes

```
public class CreateUserCommand {
    private String userId;
    private String name;
    private String email;
}

public class UpdateUserEmailCommand {
    private String userId;
    private String newEmail;
}
```

Note these are **just data** — an intention expressed as an object, with no behaviour. Making the intent an explicit object is part of the point: you can log it, queue it, or replay it.

Command handlers

```
public class CreateUserCommandHandler {
    private UserRepository userRepository;

    public CreateUserCommandHandler(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    public void handle(CreateUserCommand command) {
        User user = new User(command.getUserId(), command.getName(), command.getEmail());
        userRepository.save(user);
    }
}

public class UpdateUserEmailCommandHandler {
    private UserRepository userRepository;

    public UpdateUserEmailCommandHandler(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    public void handle(UpdateUserEmailCommand command) {
        if (!isValidEmail(command.getNewEmail())) {
            throw new IllegalArgumentException("Invalid email format");
        }
        User user = userRepository.findById(command.getUserId());
        if (user != null) {
            user.setEmail(command.getNewEmail());
            userRepository.save(user);
        }
    }

    private boolean isValidEmail(String email) {
        return email != null && email.contains("@");
    }
}
```

Two things to notice. First, `handle` returns `void` — matching the formal rule that commands don't return data. Second, **validation lives here**, in the command handler, before any state change. This is the "accuracy and security" focus of the write side made concrete.

Domain model and repository

```
public class User {
    private String id;
    private String name;
    private String email;
}

public interface UserRepository {
    void save(User user);
    User findById(String id);
}
```

The read side

```
public class UserProfileDTO {
    private String id;
    private String name;
    private String email;
}

public class GetUserProfileQuery { private String userId; }
public class ReadUserEmailQuery  { private String userId; }
```

```
public class GetUserProfileQueryHandler {
    private UserRepository userRepository;

    public UserProfileDTO handle(GetUserProfileQuery query) {
        User user = userRepository.findById(query.getUserId());
        return new UserProfileDTO(user.getId(), user.getName(), user.getEmail());
    }
}

public class ReadUserEmailQueryHandler {
    private UserRepository userRepository;

    public String handle(ReadUserEmailQuery query) {
        User user = userRepository.findById(query.getUserId());
        return user != null ? user.getEmail() : null;
    }
}
```

Notice the granularity. `ReadUserEmailQuery` returns just a `String` — not a whole object. Read models can be **as narrow as the screen actually needs**, and that's precisely the optimisation opportunity CQRS opens up. Each query gets its own purpose-built shape rather than everyone sharing one bloated object.

Also note the handlers return **DTOs, not domain objects**. The read side is deliberately not exposing the write side's model.

Infrastructure

```
public class SimpleCommandBus implements CommandBus {
    private CreateUserCommandHandler handler;

    @Override
    public void dispatch(CreateUserCommand command) {
        handler.handle(command);
    }
}

public class SimpleQueryBus implements QueryBus {
    private GetUserProfileQueryHandler handler;

    @Override
    public UserProfileDTO dispatch(GetUserProfileQuery query) {
        return handler.handle(query);
    }
}

public class SimpleEventStore implements EventStore {
    private List<UserEvent> events = new ArrayList<>();

    @Override
    public void storeEvent(UserEvent event) {
        events.add(event);
    }
}
```

The buses route to handlers; the event store captures changes as a series of events which, in a full application, can be **replayed to rebuild the state of an entity**.

> **Where this example actually sits:** it's **Level 1**. Both sides still share the same `UserRepository` and the same database. The event store is present but unused by the read path. That's not a flaw — Level 1 is where most real systems should live, and it already delivers the design clarity that is CQRS's main benefit.

Part 6 — Issues, considerations, and performance

Complexity

Implementing CQRS introduces **a fair amount of complexity**. Maintaining two separate models — command and query — is a complex task. This can be a roadblock for developers unfamiliar with the pattern, and **the learning curve is steep**.

> The course's image: like weaving a basket, the more intricate the design, the more complex the weaving process.

Managing data consistency

One of the significant challenges is maintaining consistency between the command model and the query model.

The scenario: the command model changes, but the query model hasn't been updated yet. **What happens if a query arrives during that interval? It returns outdated data.**

> This is the central trade-off of CQRS, and the one to raise in an interview. In UI terms: a user saves their profile, the page reloads, and their old name is still showing. Common mitigations include having the UI optimistically display what the user just submitted, or routing reads immediately after a write to the write model.

Handling transactions

In traditional CRUD, handling transactions is relatively straightforward. In CQRS it gets complicated.

The scenario: you need to perform multiple commands as part of a single logical transaction. What if one command fails? **How do you roll back the changes from previous commands?** Remember, commands change system state.

> The course's image: like painting a wall — if you make a mistake with one colour you can't simply erase it; you may need to repaint the entire wall.

(This is exactly the problem the Saga pattern addresses with compensating transactions.)

Dealing with asynchronicity

CQRS often comes with asynchronous operations, making **error handling and debugging more complex**. In a system where operations are performed asynchronously, **tracking down the source of an error is challenging**.

The performance upside

On the brighter side, a compelling reason to use CQRS is its **potential to improve performance**. By segregating commands and queries you optimise each side for its specific purpose.

For instance, if you find queries outnumber commands, you can **allocate more resources to the query side** to ensure smooth operation.

> The course's image: putting more trains on your busiest routes while maintaining fewer services where demand is low.

Part 7 — Use cases

High-performance applications

The most compelling case: applications where **the number of reads significantly outweighs the number of writes**. An e-commerce platform where millions of users browse products (queries) compared to a much smaller number of transactions (commands). It makes obvious sense to optimise the system for the higher volume of reads.

Complex business logic

Where a system has **complex business logic that modifies state**, separating read and write responsibilities simplifies that complexity.

> The course's image: division of labour on a factory assembly line — each worker focuses on their specific task, leading to improved efficiency and higher quality output.

The practical benefit: your write model can be rich with rules and invariants without that complexity slowing down or complicating every read.

Collaborative domains

Where **many users can update the same data**, CQRS helps manage conflicts. Picture a shared document edited by several people simultaneously. **If updates are treated as commands that can fail or be rejected based on rules**, it becomes much easier to handle this collaborative environment.

Part 8 — System design examples

Event-driven microservices

An e-commerce system handling orders. Each order goes through stages: **placed, paid, shipped, delivered**.

Each stage is a **state change driven by commands**. Meanwhile, different microservices need to **query** the state of an order — a shipment tracking service, a customer notification service. Using CQRS here leads to a cleaner, more efficient system design.

Real-time analytics

A social media platform where users are constantly **posting updates (commands)** while the platform simultaneously provides **real-time analytics about user engagement (queries)**.

Their performance needs are genuinely opposite:

- **Commands must be fast and reliable** to provide a smooth user experience
- **Queries require heavy computation** but are **not time-critical**

By segregating these responsibilities, the system can **optimise resource allocation to meet these differing needs**. This is a textbook fit — the two workloads would actively harm each other sharing one model.

Part 9 — Common beginner misconceptions

"CQRS requires two databases." No — that's Level 2. Level 1 is two models over one database, and it's where most systems should live.

"CQRS requires event sourcing." No. They're frequently discussed together and complement each other, but they're independent choices. You can do CQRS with no events at all.

"CQRS is the same as read replicas." Read replicas are copies of the *same* schema for load distribution. CQRS means a *different model* shaped for reading. You can use both.

"Commands are just HTTP POSTs." Commands are a design concept, not a transport detail. A command is an explicit object representing intent — which is what lets you queue, log, and replay it.

"It's always faster." It adds indirection. It's faster when read and write workloads genuinely conflict, and pure overhead when they don't.

"Apply it to the whole application." Almost always wrong. Apply it to the **specific parts** where read and write needs genuinely diverge, and leave simple CRUD as CRUD.

Part 10 — Check your understanding

1. Why can't one database schema be optimal for both reads and writes?
2. What are the two halves of the formal rule, and why does "never both" matter?
3. Which level of CQRS should most teams adopt, and why?
4. What specific problem does event sourcing solve that Level 2 has?
5. What is the central trade-off, expressed as something a user would notice?
6. Why is the social media analytics example a textbook fit?

(Answers: 1 — writes want normalised storage where each fact lives once, reads want denormalised pre-joined shapes, and these are directly opposed; 2 — queries return data and change nothing, commands change state and return only success or failure; "never both" makes queries safely repeatable and cacheable and makes commands explicitly consequential; 3 — Level 1, two models over one database, because it gives the design clarity with almost none of the operational cost; 4 — out-of-order event delivery leaving the read model stale, solved by replaying an append-only event stream; 5 — the consistency window: a user saves their profile and the page still shows the old value; 6 — posting must be fast and reliable while analytics is computation-heavy but not time-critical, so the two workloads would degrade each other if they shared a model.)

Part 11 — Interview soundbite (30 seconds)

> "CQRS separates the write path from the read path — commands that change state and return nothing, queries that return data and change nothing. The motivation is that reads and writes usually have opposite requirements: in a banking app, transactions need accuracy and security while balance inquiries need raw speed, and at the storage level writes want normalised data while reads want denormalised pre-joined shapes, so you can't tune one model for both. It's a spectrum, and being clear about which level you mean matters. The simplest version is two models over one database, which is where most systems should stop — you get the design clarity with almost no operational cost. The next level is separate read and write databases, which buys independent scaling but costs you atomic transactions, so you're on eventual consistency via async events and dealing with out-of-order delivery and lost messages. The furthest level is event sourcing, where the write side is an append-only event log and you rebuild read models by replaying it. That solves ordering and gives you audit history and the ability to build entirely new read models from past events, but it's a big undertaking and a separate decision from adopting CQRS. The main downside throughout is the consistency window — a user saves their profile and the page still shows the old name — plus genuine complexity and harder debugging. It pairs naturally with event-driven architecture, and with sagas when one logical transaction spans multiple commands."

Part 12 — Implementation

A. Runnable demo

Full source: `demos/CqrsDemo.java`

```
cd demos && ./gradlew runCh12
```

The formal rule made mechanical — command handlers return `void`:

```
void handle(ChangeEmail cmd) {                      // <- returns void
    User u = writeStore.get(cmd.userId());
    if (!cmd.newEmail().contains("@")) throw new IllegalArgumentException("invalid email");
    String old = u.email;
    u.email = cmd.newEmail();
    publish(new UserEvent.EmailChanged(cmd.userId(), old, cmd.newEmail()));
}

UserProfileView get(String id) { return store.get(id); }   // <- returns data, changes nothing
```

Rebuilding a read model from the event log — the Level 3 superpower:

```
fast.clear();                              // wipe the read model entirely
eventStore.all().forEach(fast::apply);     // replay history
```

B. What you should see

```
=== 3. Level 2: separate read store -> CONSISTENCY WINDOW ===
  write store immediately: carol@new.com
  read model immediately:  carol@old.com   <- STALE
  read model after 300ms:  carol@new.com   <- converged

=== 4. Several read models from ONE write ===
  profile view : UserProfileView[id=u4, name=Dave, email=dave@x.com]
  directory view: [UserDirectoryEntry[name=Dave, initial=D], ...]

=== 5. Level 3: rebuild the read model by replaying events ===
  after wiping the read model:  null
  after replaying the log:      UserProfileView[id=u1, name=Alice, ...]
```

Section 3 is the trade-off you must be able to describe in an interview, and here it's observable: the write store has the new email while the read model still has the old one. That's not a bug — it's the deal you accepted. Section 5 shows the compensating power: **read models are disposable**, because the event log is the truth.

C. Production — Axon Framework

```
@Aggregate
public class UserAggregate {

    @AggregateIdentifier private String userId;
    private String email;

    @CommandHandler
    public UserAggregate(CreateUserCommand cmd) {
        if (!cmd.email().contains("@")) throw new IllegalArgumentException("invalid email");
        AggregateLifecycle.apply(new UserCreatedEvent(cmd.userId(), cmd.name(), cmd.email()));
    }

    @CommandHandler
    public void handle(ChangeEmailCommand cmd) {
        if (cmd.newEmail().equals(this.email)) return;              // no-op, no event
        AggregateLifecycle.apply(new EmailChangedEvent(cmd.userId(), this.email, cmd.newEmail()));
    }

    // State is derived by applying events - never assigned directly
    @EventSourcingHandler
    public void on(UserCreatedEvent e) { this.userId = e.userId(); this.email = e.email(); }

    @EventSourcingHandler
    public void on(EmailChangedEvent e) { this.email = e.newEmail(); }
}
```

The projection — the read side:

```
@Component
public class UserProfileProjection {

    private final UserProfileRepository repo;

    @EventHandler
    public void on(UserCreatedEvent e) {
        repo.save(new UserProfileView(e.userId(), e.name(), e.email()));
    }

    @EventHandler
    public void on(EmailChangedEvent e) {
        repo.findById(e.userId()).ifPresent(v -> { v.setEmail(e.newEmail()); repo.save(v); });
    }

    @QueryHandler
    public UserProfileView handle(FindUserProfileQuery q) {
        return repo.findById(q.userId()).orElseThrow();
    }
}
```

Plain Spring, no framework — Level 1, which is where most systems should live:

```java
// Write side: JPA against a normalised schema
public interface UserRepository extends JpaRepository<User, String> {}

// Read side: a projection interface or native query against a denormalised view
public interface UserProfileReadRepository extends Repository<UserProfileView, String> {
    @Query(value = """
        SELECT u.id, u.name, u.email, COUNT(o.id) AS order_count
        FROM users u LEFT JOIN orders o ON o.user_id = u.id
        WHERE u.id = :id GROUP BY u.id
        """, nativeQuery = true)
    UserProfileView findProfile(@Param("id") String id);
}
```

> **You do not need Axon, Kafka, or event sourcing to do CQRS.** Two repositories and two model classes over one database is legitimate CQRS and delivers most of the benefit.

Handling the consistency window in the UI — three practical options:

```java
// 1. Read-your-own-writes: route reads right after a write to the write model
if (recentlyWrittenBy(userId)) return writeModel.find(userId);
return readModel.find(userId);

// 2. Return the new state from the command endpoint so the UI can render optimistically
// 3. Return a version token; the client polls the read model until it catches up
```

D. Implementation pitfalls

1. Jumping straight to Level 3. Event sourcing is a large, hard commitment — schema evolution for events, snapshotting for long streams, replay tooling. Adopting it *because* you adopted CQRS is the most common way teams get into trouble. Start at Level 1.

2. Commands that return data. The moment `handle()` returns the updated entity, the rule collapses and you're back to one model. Return void, or at most an ID.

3. Business logic in the read model. Projections should only *shape* data. Any rule that decides something belongs on the write side, or the two sides will disagree.

4. Ignoring the consistency window in the UI. Users who save and immediately see stale data file bugs. Design for it explicitly with one of the three techniques above.

5. Not being able to rebuild projections. The main benefit of event sourcing is disposable read models — but only if replay is *tested*. Make rebuilding a routine operation, not a theoretical one.

6. Unbounded event streams. An aggregate with 100,000 events takes forever to load by replay. Use **snapshots** — persist state every N events and replay only from there.

7. Applying CQRS everywhere. Simple CRUD entities should stay CRUD. Apply it where read and write needs genuinely diverge, and be able to say *why* for each place you use it.

8. Events named as commands. `UpdateEmail` is a command; `EmailChanged` is an event. Getting the tense wrong signals a design where events are really disguised instructions, which reintroduces coupling.

Related patterns

- **Event-Driven Architecture** — the substrate that syncs command and query sides
- **Saga** — for transactions spanning multiple commands, where rollback isn't automatic

- **BFF** — read models shaped per client is the same instinct as BFF applied to data

Configuration Externalization Pattern

Category: Operations **One-liner:** Move configuration out of application code and deployment packages into a centralized store that services fetch at runtime — so changing config doesn't require rebuilding and redeploying.

Part 0 — Concepts you need first

What is "configuration"?

Configuration is the set of values that control how an application behaves, but which **aren't the application's logic**. Typical examples:

- Database connection strings and credentials
- API keys and endpoint URLs for external services
- Timeout durations, retry counts, thread pool sizes
- Feature flags — switches turning functionality on and off
- Region-specific values: currency, tax rate, language

The test for whether something is configuration: **would you want a different value in development versus production, or in one region versus another, without changing the code?** If yes, it's configuration.

What does "hardcoded" mean?

A value written directly into the source code:

```
String dbUrl = "jdbc:postgresql://prod-db-01:5432/orders";
```

Changing it means editing the source, recompiling, and redeploying. The value is **frozen into the built artifact**.

What is a "deployment package"?

The bundle you actually ship — a JAR, a container image, a zip. "Bundled configuration" means the config file travels *inside* that package. Better than hardcoding (you can read it without recompiling), but **still frozen at build time**: to change it you rebuild the package and redeploy.

What is a "deployment cycle"?

The full process of getting a change into production: build → test → review → deploy → verify. In a well-run organisation this might take an hour. In a regulated one it might take days.

The core complaint of this chapter: that entire process, for changing one number.

What is an environment?

A separate copy of your system for a purpose — **development** (engineers experimenting), **staging/testing** (rehearsal), **production** (real users). Same code, different configuration: different databases, different API keys, different scale.

What is a secret?

Configuration that must not be disclosed — database passwords, API keys, encryption keys. **Secrets are configuration, but with much stricter handling requirements**, which becomes a recurring theme below.

Part 1 — The framing

Picture yourself as an architect designing a city. Would you place the blueprint of every building **inside** each one, hard-coded into the walls? Or keep it separate, accessible, and modifiable, so each building can adapt to changing needs?

The answer is obvious for physical structures. The same principle applies to software.

In microservices, configuration determines how efficiently applications run and adapt. Whether it's network settings, database parameters, or feature toggles, **each microservice has a set of configurations governing its behaviour**.

Traditionally these are hardcoded into the application or bundled within the deployment package. But is that efficient? **What happens when you need to change a parameter? A new deployment cycle for a minor tweak?**

Part 2 — The problem

Volume and diversity

Picture your architecture as a bustling city, each microservice a building with its own purpose and configuration. Now imagine **each building is from a different era, built with diverse architectural styles and materials.** That's what a microservices architecture often looks like — many services, possibly written in **different languages using different technology stacks**.

Managing configuration across all of them quickly becomes a nightmare, because each stack has its own conventions for where config lives and how it's read.

The drawbacks of hardcoding

Hardcoding is **like permanently etching the blueprint onto a building's facade**. It seems fine initially — everything's in one place. The downsides appear when you need to change something.

Hardcoding ties configuration **tightly to the application code**. Consequences:

- **Any change, no matter how minor, requires a new version of the application followed by a complete redeployment**
- This **slows the adaptability** of your microservices
- **It introduces risk.** If a configuration error slips into production, you face **another round of building and deployment** — wasting resources and, in the worst case, causing **significant downtime**

> That last point is the one worth dwelling on. A typo in a database hostname is a five-second fix conceptually, but a full deployment cycle in practice — and you're doing it while production is broken.

The burden of scale

These issues are magnified in microservices by the sheer scale of the system.

> Return to the city analogy: now imagine you need to modify the blueprint of **every single building**. That's what engineers face when configuration is hardcoded or bundled within each microservice.

In a **monolithic** architecture, configuration is usually **centralized and managed in one place**. In **microservices**, each service comes with its own configuration maintained independently. This lack of centralization **amplifies the workload** and **increases the risk of inconsistencies and errors** across the system.

> A concrete inconsistency: you rotate a database password and update it in fourteen services. You miss one. That service keeps working until its connection pool recycles — possibly hours later — and then fails in a way that looks unrelated to anything you did.

Inflexibility of deployments

Microservices are known for adaptability — different services deployed independently, enabling faster releases and efficient scaling. **Traditional configuration management severely hampers this flexibility.**

With hardcoded or bundled configuration, **each minor change could necessitate a full redeployment of a service**. This is inefficient, and it **negates the independent deployment advantage** that is microservices' main selling point. It also leads to **inconsistencies in configuration across environments** — development, testing, production.

Inability to adapt to dynamic environments

Microservices often operate in dynamic environments where conditions change rapidly, and those conditions influence what configuration gives optimal performance:

- **Network settings** may need adjusting based on **traffic patterns**
- **Feature toggles** may need flipping based on **user behaviour**

With traditional configuration management, **responding to these conditions in real time becomes nearly impossible.** Your services are **stuck with outdated configuration until the next deployment cycle**, impacting performance and user experience.

Security implications

Hardcoded configuration can lead to **sensitive information — database credentials, API keys — being exposed or leaked**. This is a security risk with serious consequences for your system and your users' data.

> The most common way this happens in practice: a credential hardcoded in source is committed to version control, and now it exists in the repository's history **forever**, visible to everyone with read access — and permanently, since rewriting Git history is painful and rarely complete. Rotating the credential is the only real remedy.

Part 3 — The solution

Apply **separation of concerns** to configuration management: **externalize configuration from application code**, storing and managing it in a **separate, centralized location**.

> Take all the blueprints of the city's buildings and keep them in a **central town hall**, away from the buildings themselves. The blueprints become easy to update and review without disturbing the buildings.

Mechanically: configuration is no longer part of the deployment package. It's stored externally in a **configuration server** or a file in a network-accessible location. The microservice **retrieves configuration at runtime**, ensuring it always has the most up-to-date values.

Why this is advantageous

A unified, centralized location for managing all configuration. This directly addresses the volume and diversity problem: **regardless of a microservice's language or technology stack, configuration is retrieved from the same place**, simplifying management and reducing errors.

Support for dynamic environments. Since configuration is retrieved at runtime it can be **updated in real time** as conditions change. Services **no longer wait for the next deployment cycle** to adapt — they're as dynamic as the environment they operate in.

Preserved independent deployment. Separating configuration from code upholds the microservices principle of independent deployment. Configuration changes **no longer necessitate a full redeployment**; they're updated centrally and become immediately available.

Enhanced security. With configuration externalized and centrally managed, it's **easier to enforce security policies and controls**. Sensitive information is **no longer scattered across microservices** — it's securely stored and managed in one place, reducing the risk of exposure or leaks.

Efficient scalability. The pattern scales with your architecture. As new microservices are added, their configuration is simply included in the centralized location. **No need to navigate through each service** to update configuration.

Consistency across environments. Moving from development to testing to production can feel like entering different countries with different rules. Configuration for all environments is **centrally managed**, keeping them in sync and reducing errors when promoting between them.

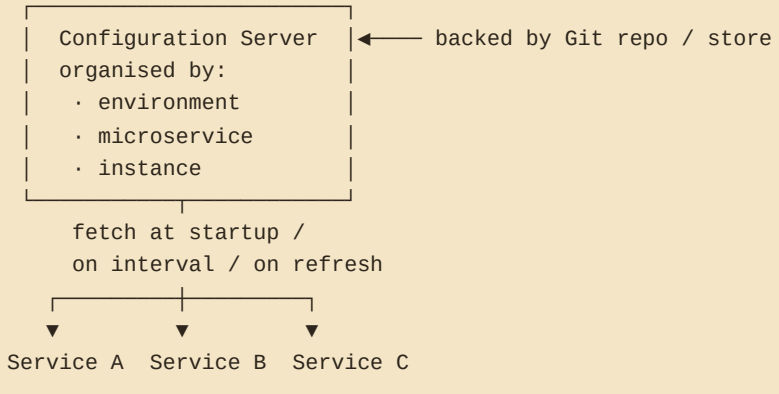
Part 4 — Architecture

At the heart of the pattern are two components: **the microservice** and the **configuration server** (or configuration file).

The interaction

The microservice, **at startup or at pre-defined intervals**, sends a request to the configuration server asking for its configuration. This can include **database connection settings, API keys, or any other environment-specific data**. The server responds with what that service needs.

What's notable is that this exchange happens **dynamically, at runtime**. The microservice can adapt in real time to new configuration **without needing a restart or redeployment**, reducing downtime and enhancing agility.



The configuration server

The **nerve centre** — think of it as the command centre overseeing and controlling all configuration. It stores, retrieves, and manages configuration for all microservices.

To streamline things, configuration can be organised:

- **By environment** — development, staging, production
- **By microservice**
- **By instance** of a microservice

> Like a well-organised library, with each configuration neatly categorised and easy to find. This organisation is what mitigates the challenges of traditional configuration management — it's not just centralized, it's *navigable*.

Dynamic adaptability in practice

Suppose you have a microservice interacting with a database, with connection parameters stored as configuration. Suddenly the database connection parameters change.

Instead of redeploying the microservice with the new parameters, you update the configuration server. The microservice, when it next retrieves configuration, **receives the updated parameters and continues its operation seamlessly.**

Security — safeguarding sensitive configuration

Returning to the library analogy: **not all books are open for everyone to read.** Some configuration contains sensitive data needing protection.

The configuration server can secure sensitive configuration through **encryption** or **access controls**. By centralizing management of these values, it's **easier to enforce strict security policies**, reducing the risk of inadvertent exposure or malicious attack.

Scalability and consistency

As you add more microservices, you simply add their configuration to the server — **no worrying about individually managing configuration for each new service.**

And by externalizing configuration you maintain **consistency across environments**. You **switch configuration as you move through environments** — no environment-specific code, no configurations getting mixed up.

Part 5 — Worked example: Spring Cloud Config

Spring Cloud Config is widely adopted in the Java community for centralized configuration management in microservices, due to its simplicity and efficiency.

The configuration server

```
@SpringBootApplication
@EnableConfigServer
public class ConfigServer {
    public static void main(String[] args) {
        SpringApplication.run(ConfigServer.class, args);
    }
}
```

With just these few lines you've turned an application into a configuration server. The `@EnableConfigServer` annotation does the work.

The configuration store — a Git repository

Now tell the server where to find the configuration:

```
spring.cloud.config.server.git.uri=https://github.com/yourusername/config-repo
```

This tells the configuration server to fetch configuration from the specified Git repository.

> **Using Git here is genuinely clever, and worth understanding rather than skimming.** By storing configuration in a version-controlled repository you get, for free: a full history of every change and who made it, diffs showing exactly what changed, pull requests and code review *for configuration*, and the ability to **roll back to any previous state** with a standard Git operation. Configuration becomes a first-class engineering artifact rather than a value someone typed into a form.

The client setup

In the microservice's build file, add the Spring Cloud Config Client dependency. Then annotate the main class with `@EnableConfigClient`. The microservice is now a configuration client, ready to consume configuration from the server.

Pulling configuration — the `@Value` annotation

```
@Value("${my.property}")
private String myProperty;
```

`my.property` is a property defined in the configuration store. `@Value` fetches its value and assigns it to the field. **Note how little the application code knows** — it references a property name, not a location or a mechanism.

Refreshing without restarting

```
POST /actuator/refresh
```

Sending a POST request to this endpoint triggers the configuration client to **re-fetch configuration from the server** — new configuration applied **without restarting the microservice.**

This is the payoff of the whole pattern in one endpoint: change a value centrally, hit refresh, done.

Encryption and decryption

```
encrypt.key=MySuperSecretKey
```

This sets the encryption key used to encrypt and decrypt properties. To encrypt a property use the `/encrypt` endpoint; to decrypt, `/decrypt`.

> ⚠️ **Notice the irony, because it matters in practice.** `encrypt.key` is itself a secret, sitting in a plain properties file. You've encrypted all your secrets and left the key to them in the open. In production this master key belongs in a dedicated secrets manager — **HashiCorp Vault, AWS Secrets Manager, or a cloud KMS** — with the config server retrieving it at startup. **The pattern**

> **relocates the secret problem; it doesn't eliminate it.** For secrets specifically, a purpose-built secrets manager usually beats a general configuration server.

Part 6 — Considerations and implications

Data security

While the pattern aids security, **it also introduces a new surface area for potential attacks**. You are now storing sensitive configuration data **remotely**. How secure is your storage? Is your data transfer secure?

To ensure security, consider **encryption**. The `MySuperSecretKey` example is just the tip of the iceberg — from using **secure and encrypted connections** to **regularly rotating keys**, you need to stay ahead in the security game.

The configuration server as a single point of failure

While centralizing simplifies management, it **introduces a single point of failure.**

> **What happens if your configuration server goes down? All your microservices would be paralysed, unable to fetch or refresh their configuration.**

Mitigation: introduce **redundancy**. Run **multiple instances** of your configuration server in a **high-availability configuration**. And **regular health checks are crucial.**

> Add to this the mitigation the course covers under performance: **client-side caching of last-known-good configuration**. This is what makes the whole trade acceptable. You've swapped "config changes require a redeploy" for "config server outage could take everything down" — and caching converts that second risk from fatal to merely inconvenient, because running services keep using the values they already have. This is the identical resilience trick used in Service Discovery.

Configuration versioning and rollbacks

The pattern allows easier scalability and quicker feature rollouts. But **what happens when you need to roll back?** Can you easily revert to a previous configuration if something goes wrong?

The good news: version control systems like **Git** provide the solution. Storing configuration in a version-controlled repository means you can **easily roll back to a previous state**.

But this **requires good configuration management practices**: Are your changes well documented? Are versions correctly labelled? Git gives you the capability; discipline is what makes it usable at 3am during an incident.

Service discovery and the config service

In a dynamic microservices environment, services come up and down and scale based on load. **How do new services discover the config service?**

Hardcoding the configuration server's address in each microservice isn't a scalable solution — and note the irony: you'd be hardcoding a location in order to avoid hardcoding configuration.

This is where **service discovery** comes in. Tools like **Netflix's Eureka** or **HashiCorp's Consul** dynamically discover services in your architecture, **including the configuration server**. *(See Service Discovery.)*

Performance implications

Fetching configuration remotely may introduce latency, especially if the configuration server is under heavy load or network issues arise.

Caching helps reduce this latency — Spring Cloud Config clients **cache remote properties by default**.

But caching brings its own considerations:

- **How often do you refresh the cache?** Too often and you hammer the server; too rarely and changes take a long time to take effect
- **What's the impact on memory usage?**

> It's a balancing act between **performance and freshness of configuration.**

Part 7 — Real-world use cases

E-commerce platform

Several microservices — Inventory, Order, User Management — each interfacing with various databases and APIs, each with its own configuration. With **hundreds of services running across production, staging, and development**, managing configuration is a nightmare.

The pattern externalizes all configuration to a central server. Each service fetches what it needs. **Changes are made in one place and reflected across the system.**

Financial services firm

A highly regulated environment. The firm must ensure **sensitive data like API keys and database passwords are not exposed**.

Using this pattern, all sensitive configuration is stored in the central configuration server, **possibly in encrypted form**. Services fetch configuration **over a secure channel** and use it. **No sensitive data is stored in the service codebase or containers** — which is exactly what auditors want to see.

Global fast-food chain

Outlets in numerous countries, each with an application managing orders, inventory, and logistics. But **each country has its own tax laws, currency, and language preferences.**

The chain uses the pattern to manage **country-specific configuration**. When an application boots up it **fetches the configuration for the country it operates in** — like having a personalized configuration per country, from one shared codebase.

Tech startup

A startup developing a streaming service, following agile practices with **multiple deployments each day**. Each deployment may require configuration changes — **feature flags or A/B testing parameters**.

The pattern lets them easily manage this dynamic configuration. Each service fetches updated configuration whenever it needs it, making **deployments faster and smoother**.

Telecommunications giant

Complex network infrastructure managed by numerous microservices. **Network parameters like bandwidth, routing information, and security protocols** need meticulous management.

All network configuration is stored centrally; each microservice fetches what it needs, giving efficient management of complex configuration.

Part 8 — Common beginner misconceptions

"Externalizing config means putting it in a file next to the app." That's still bundled config. Externalizing means it lives **outside the deployment artifact**, retrieved over the network.

"A config server is the right place for secrets." It's *better* than hardcoding, but a dedicated secrets manager (Vault, AWS Secrets Manager) gives you leasing, automatic rotation, fine-grained access policies, and audit logs that a general config server doesn't.

"Environment variables are the same thing." They're a good, simple form of externalization and often sufficient. What they don't give you is **central management, versioning, or refresh without a restart** — changing an environment variable means restarting the process.

"If the config server is down, everything stops." Only without client-side caching. With caching, **running services keep running**; you just can't start new ones or push changes. Design for that.

"Config changes are low risk because there's no deploy." The opposite concern applies — no deploy means **no build, no tests, no review** unless you deliberately add them. That's precisely why backing the store with Git and requiring pull requests matters.

"One config server for everything." Consider separating by trust boundary. Not every service should be able to read every other service's configuration.

Part 9 — Check your understanding

1. What's the difference between hardcoded, bundled, and externalized configuration?
2. Why does hardcoded configuration undermine the main benefit of microservices?
3. What does backing the config store with Git give you for free?
4. What's the central irony of the `encrypt.key` property?
5. Why do you need service discovery in order to do configuration externalization well?
6. What makes the config server's single-point-of-failure risk acceptable?

(Answers: 1 — hardcoded is in source and needs recompiling; bundled is in the deployment package and needs rebuilding and redeploying; externalized lives outside the artifact and is fetched at runtime; 2 — independent deployability, because a one-line config change forces a full redeploy of the service; 3 — change history, authorship, diffs, code review, and rollback to any previous state; 4 — it's a secret protecting all other secrets, stored in plaintext, so it belongs in a real secrets manager; 5 — hardcoding the config server's address in every service would reintroduce the exact problem you're solving; 6 — client-side caching of last-known-good values, so running services survive a config server outage.)

Part 10 — Interview soundbite (30 seconds)

> "Configuration externalization moves config out of application code and deployment packages into a central config server that services fetch at runtime. The problem it solves is that hardcoded config ties every change to a full build-and-deploy cycle, which destroys the independent deployability that's the whole point of microservices — and means you can't react to changing conditions until the next release. It also scatters secrets like database credentials across

every service and into version control history. With a config server you organise config by environment, service, and instance; services fetch at startup or on an interval, and Spring Cloud Config exposes an actuator refresh endpoint so you can apply changes without restarting. It's typically backed by a Git repo, which is the clever part — you get versioning, diffs, code review, and rollback on your configuration for free. The main trade-offs are that the config server becomes a single point of failure, so it needs HA plus client-side caching of last-known-good values so a config outage degrades rather than kills you; you've created a new attack surface holding all your secrets, so encryption in transit and key rotation matter; and services need service discovery to find the config server, because hardcoding its address would reintroduce the exact problem you're solving. For secrets specifically I'd use a dedicated secrets manager like Vault rather than a general config server, since you get leasing, rotation, and audit logging."

Part 11 — Implementation

A. Runnable demo

Full source: `demos/ConfigExternalizationDemo.java`

```
cd demos && ./gradlew runCh13
```

Layered resolution — later layers override earlier ones:

```
Map<String, String> fetch() {
    Map<String, String> merged = new LinkedHashMap<>();
    layers.values().forEach(merged::putAll);   // default <- environment <- instance
    return merged;
}
```

The cache that survives a server outage:

```
void refresh() {
    try {
        cache = server.fetch();
        cachedVersion = server.version();
    } catch (IllegalStateException e) {
        // server unreachable - KEEP the previous config rather than failing
    }
}
```

And the detail that makes runtime flips work — read config **at use time**, not at startup:

```
String checkout() {
    return config.flagEnabled("feature.newCheckout")   // read on every call
            ? "NEW checkout flow" : "legacy checkout flow";
}
```

B. What you should see

```
=== 3. Change a value + refresh, NO restart ===
  before refresh: ... timeout=1000ms ...
  after refresh:  ... timeout=5000ms ...
  >> same running process, new value

=== 4. Layering: instance overrides environment overrides default ===
  ... timeout=9000ms ...
  >> instance layer won (9000), overriding environment (5000)

=== 5. Feature flag flipped centrally ===
  checkout now: legacy checkout flow
  checkout now: NEW checkout flow
  and svc2 too: NEW checkout flow
  >> every service flipped at once, with no deployment

=== 6. Config server outage ===
    [client] server DOWN - keeping cached config (v8)
  checkout still: NEW checkout flow
  >> running services keep working on cached config
```

Section 5 is the payoff — one central change flipped behaviour in every running service simultaneously, with no build and no deploy. **Section 6 is the safety net** that makes centralizing acceptable: the config server died and nothing stopped.

C. Production — Spring Cloud Config

The server:

```
@SpringBootApplication
@EnableConfigServer
public class ConfigServerApplication {
    public static void main(String[] args) { SpringApplication.run(ConfigServerApplication.class,
args); }
}
```

```
server.port: 8888
spring:
  cloud.config.server.git:
    uri: https://github.com/acme/config-repo
    default-label: main
    clone-on-start: true
    search-paths: '{application}'          # per-service folders
  security.user:
    name: configuser
    password: ${CONFIG_SERVER_PASSWORD}    # itself injected, not committed
```

The repository layout — this is what gives you environment layering:

```
config-repo/
├── application.yml                # defaults for EVERY service
├── application-prod.yml           # defaults for every service in prod
├── order-service.yml              # all environments of one service
├── order-service-dev.yml
└── order-service-prod.yml         # most specific: wins
```

The client:

```yaml
spring:
  application.name: order-service          # picks which files apply
  config.import: "optional:configserver:http://config-server:8888"
  cloud.config:
    fail-fast: true                        # don't start with wrong config
    retry: { initial-interval: 1000, max-attempts: 6, multiplier: 1.5 }
```

Refresh without restart — the crucial bit is `@RefreshScope`:

```java
@RefreshScope                              // bean is recreated on refresh
@Component
public class CheckoutService {

    @Value("${feature.newCheckout:false}")
    private boolean newCheckoutEnabled;

    public String checkout() {
        return newCheckoutEnabled ? newFlow() : legacyFlow();
    }
}
```

```
curl -X POST http://order-service:8080/actuator/refresh
```

For fleet-wide refresh, **Spring Cloud Bus** broadcasts over Kafka or RabbitMQ so one call refreshes every instance:

```
curl -X POST http://any-instance:8080/actuator/busrefresh
```

Secrets belong in a secrets manager, not here:

```yaml
spring.cloud.vault:
  uri: https://vault:8200
  authentication: KUBERNETES            # pod identity, no static credentials
  kv: { enabled: true, backend: secret }
```

Vault gives you **leasing, automatic rotation, and audit logs** — none of which a general config server provides. Keep ordinary configuration in Git and secrets in Vault.

Kubernetes-native alternative — often all you need:

```yaml
apiVersion: v1
kind: ConfigMap
metadata: { name: order-service-config }
data:
  http.timeoutMs: "3000"
  feature.newCheckout: "true"
---
# Mounted as a volume, changes propagate to the file within ~1 min
volumes:
  - name: config
    configMap: { name: order-service-config }
```

Note: ConfigMaps mounted as **volumes** update live; ConfigMaps consumed as **environment variables** do **not** — the pod must restart. That trips people up constantly.

D. Implementation pitfalls

1. Reading config once into a `static final` field. Then refresh does nothing, because the value was captured at class-load time. Use `@RefreshScope`, or read through a provider at each use.

2. No client-side caching or `fail-fast` misconfigured. Without caching, a config server outage takes down every service. With `fail-fast: true` **and** retry configured, startup waits and retries rather than booting with defaults that are silently wrong.

3. Secrets in the Git config repo. Once committed they're in the history permanently. Use Vault or a sealed-secrets mechanism, and treat any leaked credential as needing rotation, not deletion.

4. The encryption key stored beside the encrypted values. The irony flagged in Part 5 — you've locked the door and left the key in it. The master key belongs in a KMS or in Vault, injected at startup.

5. No validation of config before it propagates. A typo in a Git commit reaches production in seconds with no build or test to catch it. Add a CI job that validates the config repo, and require pull requests — the speed of config changes is a hazard as well as a feature.

6. Changing config in production directly. If your repo is the source of truth, an out-of-band change gets silently reverted by the next refresh, producing a genuinely baffling incident.

7. A single config server instance. It's a single point of failure for the whole estate. Run at least two replicas behind a service, and monitor them.

8. Hardcoding the config server's address. Self-defeating. Use service discovery or a stable Kubernetes Service DNS name.

9. Assuming env-var ConfigMaps refresh. They don't. Only volume mounts propagate without a restart.

Related patterns

- **Service Discovery** — how services locate the config server; identical HA and caching concerns
- **Sidecar** — a sidecar can fetch config and expose it locally to the application
- **Strangler Fig** — the feature flags driving migration routing live in externalized config
- **Circuit Breaker** — breaker thresholds are configuration you'll want to tune without redeploying